



UNIVERSITAS INDONESIA

PREDIKSI INDEKS HARGA SAHAM LQ45 DENGAN MODEL GRU-ODE

SKRIPSI

FATTAN RADITYA ANGGORO

2206812533

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

PROGRAM STUDI SARJANA MATEMATIKA

DEPOK

JUNI 2026



UNIVERSITAS INDONESIA

PREDIKSI INDEKS HARGA SAHAM LQ45 DENGAN MODEL GRU-ODE

SKRIPSI

**Diajukan sebagai salah satu syarat memperoleh gelar
Sarjana Matematika**

FATTAN RADITYA ANGGORO

2206812533

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

PROGRAM STUDI SARJANA MATEMATIKA

DEPOK

JUNI 2026

HALAMAN PERNYATAAN ORISINALITAS

**Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.**

Nama : Fattan Raditya Anggoro

NPM : 2206812533

Tanda Tangan

Tanggal : Mei 2026

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh :

Nama : Fattan Raditya Anggoro
NPM : 2206812533
Program Studi : Sarjana Matematika
Judul Skripsi : Prediksi Indeks Harga Saham LQ45 dengan Model GRU-ODE

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Matematika pada Program Studi Sarjana Matematika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Indonesia.

DEWAN PENGUJI

Pembimbing I : Drs. Gatot Fatwanto Hertono, M.Sc., Ph.D. ()

Pembimbing II : Devvi Sarwinda, S.Si., M.Kom. ()

Penguji I : Prof. Dr. Suryadi, M.T. ()

Penguji II : Dr. Putri Zahra Kamalia, S.Si., M.Si. ()

Ditetapkan di: Depok
Tanggal : Juni 2026

HALAMAN PERNYATAAN PENGGUNAAN KECERDASAN ARTIFISIAL

Saya menyatakan bahwa dalam penyusunan Tugas Akhir ini, saya menggunakan teknologi Kecerdasan Buatan dengan *platform* Google Gemini untuk tujuan pemeriksaan dan perbaikan tata bahasa dan saya bertanggung jawab penuh atas keakuratan, orisinalitas, dan integritas seluruh isi Tugas Akhir ini.

Nama : Fattan Raditya Anggoro

NPM : 2206812533

Tanda Tangan :

Tanggal : Mei 2026

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa, karena atas berkat dan rahmat-Nya, saya dapat menyelesaikan skripsi ini. Penulisan skripsi ini dilakukan dalam rangka memenuhi salah satu syarat untuk mencapai gelar Sarjana Matematika pada Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Indonesia. Saya menyadari bahwa, tanpa bantuan dan bimbingan dari berbagai pihak, dari masa perkuliahan sampai pada penyusunan skripsi ini, sangatlah sulit bagi saya untuk menyelesaikan skripsi ini. Oleh karena itu, saya mengucapkan terima kasih kepada:

1. Dosen pembimbing yang telah menyediakan waktu, tenaga, dan pikiran untuk mengarahkan saya dalam penyusunan skripsi ini;
2. Dosen penguji yang telah mengkritisi serta memberikan saran dan masukan demi pengembangan penelitian ini;
3. Orang tua dan keluarga saya yang telah memberikan bantuan dukungan material dan moral; dan
4. Khansa.
5. BEM FMIPA UI 2023 dalam Departemen Pemkad
6. BEM FMIPA UI 2024 dalam Departemen Pemkad
7. Keluarga Besar PDM 2024
8. BEM FMIPA UI 2025 dalam Pengurus Inti dan Tim Satuan Pengendali Internal
9. BEM UI 2026 dalam Pengurus Inti dan Tim Satuan Pengendali Internal
10. Matek
11. Teman-teman Departemen Matematika

Akhir kata, saya berharap Tuhan Yang Maha Esa berkenan membalas segala kebaikan semua pihak yang telah membantu. Semoga skripsi ini membawa manfaat bagi pengembangan ilmu.

Depok, Mei 2026

Fattan Raditya Anggoro

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama : Fattan Raditya Anggoro
NPM : 2206812533
Program Studi : Sarjana Matematika
Departemen : Matematika
Fakultas : Matematika dan Ilmu Pengetahuan Alam
Jenis Karya : Skripsi

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

Prediksi Indeks Harga Saham LQ45 dengan Model GRU-ODE

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini, Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*data base*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Depok
Pada tanggal : Mei 2026
Yang menyatakan

(Fattan Raditya Anggoro)

ABSTRAK

Nama : Fattan Raditya Anggoro
Program Studi : Sarjana Matematika
Judul : Prediksi Indeks Harga Saham LQ45 dengan Model GRU-ODE

Indeks Harga Saham LQ45 merupakan indikator vital yang mencerminkan kinerja saham dengan likuiditas tinggi dan kapitalisasi pasar besar di Bursa Efek Indonesia. Namun, karakteristik data pasar saham *intraday* yang fluktuatif, non-linear, dan memiliki interval waktu tidak teratur (*irregular time-series*) menjadi tantangan signifikan bagi metode *forecasting* konvensional. Model *Deep Learning* standar yang beroperasi pada domain waktu diskrit, seperti *Gated Recurrent Unit* (GRU), memiliki keterbatasan dalam menangani iregularitas tersebut karena mengasumsikan interval waktu yang seragam antar observasi. Untuk mengatasi permasalahan ini, penelitian ini mengusulkan penerapan model *Gated Recurrent Unit – Ordinary Differential Equation* (GRU-ODE). Model *hybrid* ini memadukan mekanisme *gating* GRU untuk menangkap dependensi temporal jangka panjang dengan Neural ODE, yang memodelkan evolusi *hidden state* secara kontinu antar titik observasi melalui penyelesaian persamaan diferensial secara numerik. Penelitian menggunakan data harga penutupan (*closing price*) Indeks LQ45 dengan interval 5 menit pada periode 1 Agustus 2025 hingga 28 November 2025 sebanyak 4.736 sampel yang diperoleh dari Yahoo Finance. Kinerja model dievaluasi menggunakan metrik *Mean Absolute Error* (MAE), *Root Mean Square Error* (RMSE), dan *Mean Absolute Percentage Error* (MAPE). Berdasarkan *Grid Search* terhadap 16 kombinasi hyperparameter, konfigurasi optimal diperoleh pada 32 *hidden units*, *learning rate* 0,001, *batch size* 16, dan *solver* RK4, dengan MAE 0,7831, RMSE 1,0955, dan MAPE 0,0918% pada *test set*. Dibandingkan dengan GRU, model GRU-ODE menghasilkan reduksi *error* prediksi sebesar 29,42% (MAE), 24,52% (RMSE), dan 29,53% (MAPE), yang menunjukkan bahwa pemodelan *hidden state* secara kontinu lebih sesuai untuk prediksi Indeks Harga Saham LQ45 *intraday* dibandingkan GRU.

Kata kunci: *Gated Recurrent Unit*, *Neural Ordinary Differential Equations*, *Forecasting*, Indeks LQ45, GRU-ODE.

EXTENDED ABSTRACT

Undergraduate Thesis

Forecasting LQ45 Stock Price Index using GRU-ODE

Student Name : Fattan Raditya Anggoro

Student ID Number (NPM) : 2206812533

Study Program : Undergraduate Study Program of Mathematics

The LQ45 Stock Price Index is a key indicator reflecting the performance of stocks with high liquidity and large market capitalization on the Indonesia Stock Exchange. However, intraday stock market data are fluctuating, non-linear, and recorded at irregular time intervals due to lunch breaks, overnight closures, weekends, and public holidays. These characteristics pose significant challenges for conventional forecasting methods. Standard deep learning models that operate on a discrete time domain, such as the Gated Recurrent Unit (GRU), have limitations in handling this irregularity because they implicitly assume uniform time intervals between observations. This study therefore aims to develop a forecasting model that explicitly accounts for irregular intervals and to evaluate its performance against a discrete-time baseline.

To address these issues, this study proposes the application of the Gated Recurrent Unit – Ordinary Differential Equation (GRU-ODE) model. This hybrid model integrates the gating mechanism of GRU, which captures long-term dependencies, with a Neural ODE component that models the continuous evolution of the hidden state between observation points via a numerical ODE solver (Euler and Runge–Kutta 4th order). The elapsed time Δt between consecutive observations is incorporated as an explicit input, allowing the model to adapt to irregularly-spaced observations without information loss caused by forced discretization.

This study uses 5-minute closing-price data of the LQ45 Index retrieved from Yahoo Finance for the period August 1, 2025 to November 28, 2025, yielding 4,736 samples. The data are partitioned into 75% training, 15% validation, and 10% test sets. Model performance is evaluated on the test set using Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and Mean Absolute Percentage Error (MAPE), each of which measures a complementary aspect of prediction accuracy and the model's ability to track market volatility.

Based on a Grid Search over 16 hyperparameter combinations, the GRU-ODE model achieved its best performance with 32 hidden units, learning rate 0.001, batch size 16, and the RK4 solver, yielding MAE 0.7831, RMSE 1.0955, and MAPE 0.0918% on the test set. Compared with a GRU baseline trained under identical conditions, GRU-ODE

reduced prediction errors by 29.42% (MAE), 24.52% (RMSE), and 29.53% (MAPE). These results indicate that explicitly modeling the irregularity of time intervals and the continuous dynamics of the hidden state makes GRU-ODE substantially better suited to forecasting the rapidly fluctuating LQ45 Intraday Stock Price Index than the conventional discrete-time GRU.

Keywords: *Gated Recurrent Unit, Neural Ordinary Differential Equations, Forecasting, LQ45 Index, GRU-ODE.*

DAFTAR ISI

HALAMAN PERNYATAAN ORISINALITAS	iii
HALAMAN PENGESAHAN	v
HALAMAN PERNYATAAN PENGGUNAAN KECERDASAN ARTIFISIAL	vii
KATA PENGANTAR	ix
HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI	xi
ABSTRAK	xiii
EXTENDED ABSTRACT	xv
DAFTAR ISI	xvii
DAFTAR TABEL	xxi
DAFTAR GAMBAR	xxiii
DAFTAR LAMPIRAN	xxv
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	4
1.4 Manfaat Penelitian	4
1.4.1 Bagi Institusi	4
1.4.2 Bagi Peneliti	5
1.4.3 Bagi Dunia Pendidikan	5
1.5 Ruang Lingkup Penelitian	5
BAB 2 TINJAUAN LITERATUR	7

2.1	Indeks Harga Saham LQ45	7
2.2	<i>Time-Series (Irregular Time-Series)</i>	8
2.2.1	Pembagian Data <i>Time-Series</i>	8
2.3	Normalisasi Data	9
2.4	<i>Machine Learning</i>	10
2.5	<i>Deep Neural Network</i>	11
2.5.1	<i>Multilayer Perceptron (MLP)</i>	12
2.5.2	Fungsi Aktivasi	13
2.5.3	<i>Backpropagation</i>	14
2.6	Regularisasi	15
2.6.1	<i>Early Stopping</i>	16
2.6.2	<i>Dropout</i>	16
2.7	<i>Gated Recurrent Unit (GRU)</i>	17
2.8	<i>Ordinary Differential Equation</i>	18
2.8.1	Metode Euler	19
2.8.2	Metode Runge-Kutta Orde 4	19
2.9	<i>Neural Ordinary Differential Equation</i>	19
2.10	GRU-ODE	21
2.10.1	Integrasi Numerik <i>Hidden State</i>	22
2.10.2	<i>Backpropagation</i> pada GRU-ODE	23
2.10.2.1	<i>Discretize-then-Optimize (DTO)</i>	24
2.10.2.2	<i>Adjoint Method</i>	24

BAB 3 PEMBENTUKAN MODEL GATED RECURRENT UNIT- ORDINARY DIFFERENTIAL EQUATION (GRU-ODE) 25

3.1	Konsep Dasar <i>Gated Recurrent Unit – Ordinary Differential Equation</i> . .	26
3.2	Pemodelan <i>Gated Recurrent Unit</i>	26
3.3	Pemodelan GRU-ODE	27
3.4	Integrasi Numerik pada GRU-ODE	29
3.4.1	Integrasi Numerik dengan Metode Euler	30
3.4.2	Integrasi Numerik dengan Metode RK4	31
3.5	Ilustrasi Perhitungan GRU-ODE	32
3.5.1	Ilustrasi Perhitungan pada <i>Reset Gate</i>	33
3.5.2	Ilustrasi Perhitungan pada <i>Update Gate</i>	33
3.5.3	Ilustrasi Perhitungan <i>Candidate Hidden State (g)</i>	33
3.5.4	Ilustrasi Perhitungan Laju Perubahan <i>Hidden State</i>	34

3.5.5	Ilustrasi Perhitungan Penentuan <i>Hidden State</i> Baru	34
3.6	<i>Optimasi Parameter (Backpropagation Through the Solver)</i>	35
3.6.1	Turunan <i>Loss Function</i> terhadap <i>Hidden State</i>	36
3.6.2	Turunan <i>Loss Function</i> terhadap Parameter <i>Gate</i>	37
3.6.2.1	Turunan terhadap parameter <i>update gate</i> W_z dan b_z . . .	37
3.6.2.2	Turunan terhadap parameter <i>candidate gate</i> W_g dan b_g .	37
3.6.2.3	Turunan terhadap parameter <i>reset gate</i> W_r dan b_r . . .	38
3.6.2.4	Akumulasi <i>Gradient</i> Lintas Substep	38
3.6.3	Optimasi dengan Adam	39
3.6.3.1	Estimasi Momen Pertama (Rerata <i>Gradient</i>)	39
3.6.3.2	Estimasi Momen Kedua (Rerata Kuadrat <i>Gradient</i>) . . .	39
3.6.3.3	Koreksi Bias	40
3.6.3.4	Pembaruan Parameter	40
3.7	Hyperparameter Gated Recurrent Unit - Ordinary Differential Equation .	40
3.7.1	<i>Sequence Length</i>	40
3.7.2	Jumlah Unit <i>Hidden State (Hidden Units)</i>	41
3.7.3	<i>Learning Rate</i>	41
3.7.4	Jumlah <i>Epoch</i>	41
3.7.5	<i>Batch Size</i>	41
3.7.6	<i>ODE Solver</i>	42
3.7.7	Jumlah <i>Substep</i> Integrasi (<i>Number of Unfolds</i>)	42
3.7.8	<i>Optimizer</i> dan <i>Gradient Clipping</i>	42
3.7.9	<i>Data Splitting</i>	42
3.8	Perhitungan Metrik Evaluasi	43
3.8.1	<i>Root Mean Squared Error (RMSE)</i>	43
3.8.2	<i>Mean Absolute Error (MAE)</i>	43
3.8.3	<i>Mean Absolute Percentage Error (MAPE)</i>	44
BAB 4	HASIL PENELITIAN DAN PEMBAHASAN	45
4.1	Spesifikasi Perangkat dan Bahasa Pemrograman	45
4.2	Data Indeks Harga Saham LQ45	45
4.3	Statistika Deskriptif Variabel <i>Closing Price</i> Indeks Harga Saham <i>Intraday</i> LQ45	47
4.4	<i>Pre-processing</i>	47
4.4.1	<i>Data Cleaning</i> dan Standarisasi	47
4.4.2	<i>Feature Engineering</i>	47

4.4.3	Normalisasi	48
4.4.4	<i>Sequence Generation</i>	49
4.4.5	<i>Data Splitting</i>	49
4.5	Model GRU-ODE	50
4.6	Optimisasi <i>Hyperparameter</i> GRU-ODE	50
4.7	Implementasi Model GRU-ODE dalam Prediksi Indeks Harga Saham <i>In-</i> <i>traday</i> LQ45	51
4.8	Perbandingan Performa Model GRU-ODE dengan Model GRU	53
BAB 5	KESIMPULAN DAN SARAN	55
5.1	Kesimpulan	55
5.2	Saran	55
DAFTAR PUSTAKA		57

DAFTAR TABEL

Tabel 3.1	Faktor turunan parameter <i>update gate</i>	37
Tabel 4.1	Spesifikasi perangkat dan <i>library</i> yang digunakan.	45
Tabel 4.2	Sampel Data Indeks Harga Saham LQ45.	46
Tabel 4.3	Statistika Deskriptif <i>Closing Price</i> Indeks LQ45.	47
Tabel 4.4	Pembagian Data dan Periode Waktu Pemodelan.	49
Tabel 4.5	Konfigurasi <i>Hyperparameter</i> Model GRU-ODE.	51
Tabel 4.6	<i>Hyperparameter</i> Terbaik Hasil <i>Grid Search</i>	51
Tabel 4.7	Metrik Evaluasi Model GRU-ODE pada Setiap Set Data.	51
Tabel 4.8	Perbandingan Performa GRU-ODE vs GRU pada <i>Test Set</i>	53

DAFTAR GAMBAR

Gambar 2.1	Ilustrasi <i>Multilayer Perceptron</i>	12
Gambar 2.2	Ilustrasi <i>Sigmoid</i>	13
Gambar 2.3	Ilustrasi ReLU	14
Gambar 2.4	Ilustrasi <i>Multilayer Perceptron</i>	15
Gambar 2.5	Ilustrasi Struktur GRU	18
Gambar 3.1	Bagan Alur Penelitian	25
Gambar 4.1	Grafik Closing Price Indeks Harga Saham Intraday LQ45	46
Gambar 4.2	Hasil Prediksi pada Train Set	52
Gambar 4.3	Hasil Prediksi pada Validation Set	52
Gambar 4.4	Hasil Prediksi pada Test Set	52

DAFTAR LAMPIRAN

Lampiran 1 Kode Program Implementasi Model GRU-ODE	61
Lampiran 2 Kode Program Implementasi Model GRU	75

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dewasa ini, saham merupakan instrumen investasi yang sangat fundamental. Menurut Bursa Efek Indonesia (2022), saham adalah tanda penyertaan modal dalam suatu perusahaan terbatas yang merupakan satuan nilai pembukuan dalam berbagai instrumen finansial yang menunjukkan bagian kepemilikan atas sebuah perusahaan. Hak pemilik saham menurut Bursa Efek Indonesia (BEI) meliputi hak untuk menghadiri dan memberikan suara dalam Rapat Umum Pemegang Saham (RUPS), menerima dividen jika dibagikan, menerima sisa kekayaan jika perusahaan dilikuidasi, serta hak lainnya yang diatur oleh undang-undang dan anggaran dasar. Hak ini memberikan wewenang bagi pemegang saham untuk berpartisipasi dalam pengambilan keputusan penting perusahaan.

Seorang investor dapat melakukan jual beli saham dalam sebuah medium bernama pasar modal. Otoritas Jasa Keuangan (OJK) mendefinisikan pasar modal sebagai kegiatan yang berkaitan dengan penawaran umum dan perdagangan efek, serta perusahaan publik yang berhubungan dengan efek yang diterbitkannya, dan semua lembaga serta profesi yang terkait dengan efek tersebut. Meskipun segala kegiatan administratif di pasar saham diatur secara ketat oleh OJK, pergerakan harga yang terjadi di dalamnya sangatlah dinamis dan dipengaruhi oleh banyak faktor, mulai dari kondisi makroekonomi, performa bisnis, hingga psikologis individu. Teori *Complex Adaptive System* (CAS) adalah salah satu teori yang menjelaskan kompleksitas pasar. CAS berargumen bahwa pasar adalah sistem yang terus beradaptasi di mana elemen-elemen di dalamnya selalu memperbarui strategi mereka (Arthur, 1999), dan interaksi dinamis ini menciptakan pola statistik yang kompleks (LeBaron, 2000).

Untuk memantau dinamika pasar yang kompleks tersebut, investor memerlukan acuan yang terukur. Indeks saham adalah ukuran statistik yang mencerminkan keseluruhan pergerakan harga atas sekumpulan saham yang dipilih berdasarkan kriteria dan metodologi tertentu serta dievaluasi secara berkala. Di tengah pergerakan harga saham yang sulit diprediksi, indeks saham dapat menjadi acuan dalam mengukur sentimen pasar dan juga *benchmark* bagi portofolio investor. PT Bursa Efek Indonesia kini mengelola puluhan indeks saham, salah satunya ialah LQ45. Indeks Harga Saham LQ45 adalah indeks yang mengukur kinerja harga dari 45 saham yang memiliki likuiditas tinggi dan kapitalisasi pasar besar serta didukung oleh fundamental perusahaan yang baik (Bursa Efek Indonesia, 2025). Tingginya likuiditas pada saham-saham LQ45 menjadikannya sangat

sensitif terhadap perubahan informasi pasar, sehingga pergerakannya menjadi representasi utama aktivitas perdagangan di bursa (Susilowati et al., 2012). Karakteristik inilah yang menjadikan LQ45 sebagai objek yang sangat relevan untuk merepresentasikan dinamika pasar saham Indonesia, dan menjadi alasan pemilihannya dalam penelitian ini.

Data Indeks Harga Saham LQ45 yang digunakan dalam penelitian ini berupa data *intraday* dengan resolusi lima menit, yang termasuk dalam data tipe *time-series*. Menurut Cryer and Chan (2008), analisis data *time-series* bertujuan untuk memahami atau memodelkan mekanisme stokastik untuk melakukan prediksi data *time-series* mendatang berdasarkan data historis sebelumnya. Analisis data *time-series* telah diaplikasikan di berbagai bidang, contohnya seperti peramalan penjualan, peramalan analisis cuaca, prediksi tren ekonomi, hingga peramalan di bidang kesehatan.

Seiring dengan kemajuan dalam bidang komputasi dan ilmu data, terjadi transisi signifikan dari pendekatan tradisional ke model berbasis *machine learning* dan *deep learning*. Hal ini dikarenakan model-model tradisional seperti ARIMA kurang mampu menangkap pola non-linear yang menonjol pada data yang fluktuatif seperti pada kasus prediksi harga saham (Adebiyi et al., 2014). Di antara berbagai arsitektur *deep learning*, *Recurrent Neural Networks* (RNN) muncul sebagai paradigma yang secara fundamental dirancang untuk memproses data sekuensial atau data *time-series*. Meskipun secara konseptual sangat kuat, arsitektur RNN standar memiliki keterbatasan yang signifikan, yang dikenal sebagai masalah *vanishing/exploding gradient problem* (Bengio et al., 1994).

Untuk menjawab masalah *vanishing gradient* yang kerap menjadi *bottleneck* dalam kinerja RNN, berbagai varian arsitektur yang lebih canggih telah dikembangkan. Salah satu yang paling menonjol dan efisien adalah *Gated Recurrent Unit* (GRU), yang diperkenalkan oleh Cho et al. (2014). GRU secara spesifik dirancang untuk mempelajari dependensi jangka panjang dengan memperkenalkan *gating mechanism* yang adaptif. Salah satu keunggulan signifikan dari GRU adalah efisiensi komputasinya, terutama jika dibandingkan dengan arsitektur populer lainnya, *Long Short-Term Memory* (LSTM). GRU memiliki arsitektur yang lebih ramping karena menyatukan *input gate* dan *forget gate* LSTM menjadi satu *update gate* dan tidak menggunakan komponen *cell state* yang terpisah (Cho et al., 2014). Hal ini menghasilkan jumlah parameter yang lebih sedikit, sehingga mengurangi beban komputasi dan mempercepat proses pelatihan. Evaluasi empiris menunjukkan kinerja GRU sering kali sebanding dengan LSTM, meskipun keunggulan relatif keduanya bergantung pada karakteristik data dan jenis tugas yang dihadapi (Chung et al., 2014).

Meskipun demikian, arsitektur GRU beroperasi dalam waktu diskrit. Pada setiap langkah, *hidden state* diperbarui melalui satu *update* diskrit yang mengasumsikan bahwa

interval waktu antar observasi seragam dan bahwa dinamika sistem hanya didefinisikan pada titik-titik observasi tersebut. Asumsi ini adalah sebuah penyederhanaan yang signifikan dalam konteks prediksi indeks harga saham. Sebagaimana telah disinggung melalui teori CAS, pergerakan harga saham pada hakikatnya merupakan hasil dari berbagai faktor ekonomi yang berlangsung secara kontinu, meskipun hanya diobservasi pada interval diskrit. Antara dua titik observasi, harga sebenarnya terus berevolusi sebagai reaksi terhadap aliran informasi, sentimen pasar, dan transaksi yang terjadi tanpa henti. Lebih jauh, observasi pada data *intraday* BEI tercatat secara sporadis sehingga interval antar observasi seringkali tidak seragam. Selain interval reguler antar observasi, terdapat sejumlah *gap* struktural yang mengakibatkan asumsi keseragaman interval yang melekat pada GRU diskrit menjadi tidak realistis.

Sebagai solusi atas keterbatasan pemodelan waktu diskrit ini, Chen et al. (2018) memperkenalkan *Neural Ordinary Differential Equations* (Neural ODE), keluarga baru model RNN yang mengadaptasi pembaruan *hidden state* diskrit dengan sebuah proses dinamika waktu kontinu. Alih-alih memperbarui *hidden state* melalui langkah diskrit, Neural ODE memparameterisasi evolusi *hidden state* terhadap waktu menggunakan *neural network*. Hal ini memungkinkan Neural ODE menangani secara alami interval observasi yang tidak seragam tanpa perlu menyamakan atau menginterpolasi data terlebih dahulu (Chen et al., 2018).

Maka dari itu, penelitian ini mengusulkan penggunaan model *hybrid Gated Recurrent Unit – Ordinary Differential Equation* (GRU-ODE). Model ini menggabungkan *gating mechanism* GRU untuk mempelajari dependensi jangka panjang, dengan Neural ODE yang memodelkan evolusi *hidden state* secara kontinu di antara titik-titik observasi (De Brouwer et al., 2019). Dengan memparameterisasi turunan *hidden state* melalui *neural network*, GRU-ODE memungkinkan model untuk mempelajari dinamika yang berlangsung di antara observasi, alih-alih hanya pada titik observasi itu sendiri, sekaligus mengintegrasikan dinamika tersebut secara konsisten terhadap interval waktu antar observasi yang tidak seragam. Pendekatan ini memberikan *inductive bias* yang secara teori lebih sesuai dengan karakteristik data yang secara fundamental kontinu seperti harga saham, sehingga diharapkan mampu meningkatkan akurasi prediksi Indeks Harga Saham LQ45.

Data yang akan digunakan dalam penelitian ini adalah data Indeks Saham *Intraday* LQ45 yang dinilai sangat relevan dalam menggambarkan keadaan bursa saham Indonesia. Indeks Harga Saham LQ45 adalah salah satu indeks saham paling penting dan representatif di BEI. Indeks ini dirancang untuk mengukur kinerja 45 saham terpilih yang dievaluasi secara berkala sehingga sangat mencerminkan kinerja saham yang paling aktif

di pasar Indonesia (Bursa Efek Indonesia, 2024).

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, diperoleh rumusan masalah sebagai berikut.

1. Bagaimana memahami karakteristik pergerakan Indeks Harga Saham LQ45 *intraday* interval 5 menit?
2. Bagaimana membangun model GRU-ODE yang memodelkan evolusi *hidden state* secara kontinu dalam prediksi indeks harga saham LQ45 *intraday* interval 5 menit?
3. Bagaimana performa model GRU-ODE dibandingkan dengan model GRU *baseline* yang beroperasi pada waktu diskrit dalam *forecasting* indeks harga saham LQ45 *intraday* interval 5 menit?

1.3 Tujuan Penelitian

Berdasarkan permasalahan penelitian yang dipaparkan sebelumnya, berikut merupakan tujuan yang ingin dicapai dalam penelitian ini.

1. Memahami karakteristik pergerakan Indeks Harga Saham LQ45 berfrekuensi tinggi.
2. Membangun model GRU-ODE dalam prediksi Indeks Harga Saham LQ45 *intraday* interval 5 menit.
3. Mengukur performa model GRU-ODE dalam prediksi indeks harga saham LQ45 *intraday* interval 5 menit.

1.4 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan kontribusi pada bidang prediksi data *time-series* keuangan, khususnya pada konteks data *intraday* yang bersifat *irregular*. Manfaat dari penelitian ini dapat dijabarkan sebagai berikut.

1.4.1 Bagi Institusi

Penelitian ini diharapkan memberikan referensi tambahan bagi institusi keuangan dan akademik dalam memahami penerapan model *hybrid* berbasis ODE pada data pasar modal Indonesia, khususnya untuk prediksi indeks harga pada frekuensi *intraday*.

1.4.2 Bagi Peneliti

Penelitian ini menjadi sarana penerapan ilmu yang telah dipelajari dalam bidang matematika, khususnya persamaan diferensial biasa, metode numerik, dan *machine learning*, dalam konteks permasalahan dunia nyata di bidang keuangan.

1.4.3 Bagi Dunia Pendidikan

Penelitian ini diharapkan dapat menjadi rujukan akademik untuk penelitian lanjutan terkait *Neural ODE*, GRU-ODE, maupun *deep learning* berbasis waktu kontinu untuk data *time-series* keuangan.

1.5 Ruang Lingkup Penelitian

Penelitian ini memiliki beberapa batasan untuk memperjelas ruang lingkup serta memastikan penelitian yang dilakukan memiliki batasan yang jelas. Adapun batasan-batasan yang digunakan dalam penelitian ini adalah sebagai berikut.

1. Data yang digunakan adalah data harga *closing* indeks harga saham LQ45 dalam periode waktu 5 menit dari 1 Agustus 2025 hingga 28 Oktober 2025 yang diperoleh dari Yahoo Finance pada tautan: <https://finance.yahoo.com/quote/^JKLQ45>.
2. Pengukuran kinerja model diukur menggunakan metrik *Mean Absolute Error* (MAE), dan *Root Mean Squared Error* (RMSE) dan *Mean Absolute Percentage Error* (MAPE).
3. Simulasi dijalankan menggunakan Python versi 3.13.5 dengan perangkat komputasi NVIDIA Tesla T4 sebagai GPU.

BAB 2

TINJAUAN LITERATUR

Bab ini membahas mengenai landasan teori yang digunakan dalam penelitian, mulai dari Indeks Saham LQ45, konsep data *time-series*, *machine learning*, *deep neural network*, GRU, *Ordinary Differential Equation*, hingga model GRU-ODE.

2.1 Indeks Harga Saham LQ45

Menurut Bursa Efek Indonesia (Bursa Efek Indonesia, 2024), Indeks Harga Saham LQ45 merupakan indikator pasar yang mengukur performa harga dari 45 saham terpilih yang memiliki likuiditas tinggi, kapitalisasi pasar *free float* yang besar, serta didukung oleh fundamental perusahaan yang solid dan tingkat kepatuhan yang baik. Indeks yang diluncurkan pada 1 Februari 1997 ini merepresentasikan kumpulan saham yang paling mudah diperdagangkan (*liquid*) akibat tingginya permintaan pasar.

Proses seleksi konstituen LQ45 dilakukan secara ketat dengan menyaring saham-saham dari Indeks Harga Saham Gabungan (IHSG) yang memenuhi kriteria awal, antara lain:

1. Telah tercatat di bursa minimal enam bulan;
2. Termasuk dalam 150 saham dengan nilai transaksi tertinggi di pasar reguler selama satu tahun ke belakang;
3. Tidak dalam status *suspended* dan harus ditransaksikan dalam enam bulan terakhir;
4. Mematuhi batas minimum kapitalisasi *free float market* dan tidak sedang dalam status *suspended*;
5. Memiliki *free float ratio* sebesar 10%.

Dari kumpulan saham yang lolos kriteria awal (*universe*), dilakukan penyaringan bertahap untuk memilih 80 saham (IDX80), yang kemudian dikerucutkan kembali menjadi 45 saham final berdasarkan faktor likuiditas (nilai, frekuensi, dan kapitalisasi pasar) serta fundamental (kinerja keuangan dan kepatuhan) (Bursa Efek Indonesia, 2024).

Dalam perhitungan indeksnya, LQ45 menerapkan metodologi pembobotan *Capped Free Float Adjusted Market Capitalization Weighted* yang mendasarkan bobot indeks pada kapitalisasi pasar *free float* yang disesuaikan, dengan penerapan batasan (*cap*) tertentu untuk mencegah dominasi tunggal saham terhadap pergerakan indeks secara keseluruhan. Secara umum, pembobotan indeks dapat dinyatakan sebagai berikut:

$$\text{Index} = \sum_{i=1}^n \frac{\text{Market Cap}_i \times \text{Free Float Ratio}_i}{\text{Base Market Cap}_i} \times 100 \quad (2.1)$$

Dengan keterangan sebagai berikut:

Market Cap_{*i*} : total saham tercatat $\times$ harga pasar atas saham *i*;

Free Float Ratio_{*i*} : perbandingan jumlah saham *free float* terhadap jumlah saham tercatat atas saham *i*;

n : jumlah konstituen indeks;

Base Market Cap_{*i*} : kapitalisasi pasar pada Tanggal Dasar yang disesuaikan apabila ada perubahan jumlah saham yang dihitung untuk indeks.

2.2 Time-Series (Irregular Time-Series)

Penelitian ini bertujuan untuk memprediksi nilai indeks harga saham di masa depan berdasarkan pola historis. Permasalahan semacam ini termasuk dalam ranah *time-series forecasting*. *Time-series forecasting* adalah proses menggunakan model statistik atau komputasi untuk memprediksi nilai target variabel di masa depan berdasarkan data masa lalu yang telah diamati (Shi et al., 2022).

Time-series adalah sekumpulan observasi atau titik data yang dicatat secara berurutan dalam interval waktu (Cryer and Chan, 2008). Tipe data ini kerap digunakan untuk mengidentifikasi tren dan membuat prediksi dalam jangka waktu tertentu. Data tipe ini berbentuk titik/interval waktu yang sekuensial/berurutan. Data ini dapat dikumpulkan dalam interval harian, mingguan, bulanan, kuartalan, tahunan, atau bahkan dalam hitungan detik. Secara matematis, *time-series* dapat direpresentasikan sebagai himpunan pasangan nilai $\{(t, Y_t) : t \in T\}$, di mana *t* adalah indeks waktu dan *Y_t* adalah nilai observasi pada waktu *t*. Analisis menggunakan data *time-series* mempunyai banyak implementasi, diantaranya klasifikasi, *explanative analysis*, *descriptive analysis*, *segmentation*, dan *forecasting* (Cryer and Chan, 2008).

Dalam konteks data *intraday*, ketidakseragaman interval antar observasi menjadi hal yang perlu diperhatikan. Hal ini seringkali disebabkan oleh beberapa hal, seperti jeda istirahat makan siang, jeda *overnight*, atau bahkan sekedar kesalahan teknis pencatatan. Arsitektur *deep learning* seperti RNN mengasumsikan interval setiap langkah adalah seragam, hal ini adalah asumsi yang tidak terpenuhi pada data *intraday*.

2.2.1 Pembagian Data Time-Series

Pada pemodelan *machine learning*, data umumnya dibagi menjadi tiga himpunan, yaitu data *training*, *validation*, dan *test*. Data *training* digunakan untuk pembaruan bobot

model, data *validation* digunakan untuk pemilihan *hyperparameter* terbaik dan *regularization*, sedangkan data *test* digunakan hanya pada tahap akhir untuk evaluasi performa model yang *unbiased* (Goodfellow et al., 2016).

Pada data non-temporal, seperti pada kasus klasifikasi, pembagian umumnya dilakukan secara acak (*random split*) dengan asumsi observasi bersifat *independent and identically distributed* (i.i.d.). Namun, asumsi tersebut tidak berlaku pada data *time-series* karena terdapat dependensi temporal antar observasi (Bergmeir and Benítez, 2012). Pembagian acak pada data *time-series* akan menyebabkan *data leakage*, yaitu kondisi di mana informasi dari masa depan secara tidak sengaja digunakan untuk memprediksi masa lalu (Kaufman et al., 2012). Sebagai contoh, apabila sebuah observasi pada waktu t masuk ke data *training* sedangkan observasi pada waktu $t - 1$ masuk ke data *test*, maka model akan mempelajari informasi dari titik waktu yang lebih baru saat memprediksi titik waktu yang lebih lama, sehingga estimasi performa pada data *test* menjadi *inflated* dan tidak merefleksikan kemampuan generalisasi model pada data baru.

Untuk mengatasi hal tersebut, pembagian data *time-series* dilakukan secara kronologis (*chronological split*), yaitu data diurutkan berdasarkan waktu lalu dibagi menjadi tiga segmen berurutan tanpa pengacakan (Bergmeir and Benítez, 2012). Secara matematis, jika data *time-series* dinyatakan sebagai $\mathcal{D} = \{(t_i, Y_{t_i})\}_{i=1}^n$ dengan $t_1 < t_2 < \dots < t_n$, maka pembagian kronologis dengan proporsi $p_{\text{train}} : p_{\text{val}} : p_{\text{test}}$ menghasilkan:

$$\begin{aligned}\mathcal{D}_{\text{train}} &= \{(t_i, Y_{t_i}) : 1 \leq i \leq n_{\text{train}}\}, \\ \mathcal{D}_{\text{val}} &= \{(t_i, Y_{t_i}) : n_{\text{train}} < i \leq n_{\text{train}} + n_{\text{val}}\}, \\ \mathcal{D}_{\text{test}} &= \{(t_i, Y_{t_i}) : n_{\text{train}} + n_{\text{val}} < i \leq n\}.\end{aligned}$$

Skema pembagian tersebut menjamin tidak adanya *data leakage* pada proses *training*, karena seluruh observasi pada *training set* memiliki waktu yang lebih awal dibandingkan observasi pada *validation set* dan *test set*. Dengan demikian, evaluasi performa model pada *test set* merepresentasikan kemampuan model dalam memprediksi periode waktu yang benar-benar belum pernah diamati selama *training*.

2.3 Normalisasi Data

Normalisasi adalah salah satu teknik *pre-processing* data yang digunakan untuk mengubah nilai fitur ke dalam rentang nilai tertentu (Han et al., 2011). Pada model berbasis *deep learning*, normalisasi merupakan tahapan penting karena beberapa alasan. Pertama, fitur dengan rentang jauh berbeda dapat menyebabkan proses optimasi berlangsung lam-

bat atau tidak stabil. Fitur dengan skala yang besar akan mendominasi perhitungan *gradient*, sebaliknya fitur dengan skala kecil memberikan kontribusi yang lebih sedikit. Kedua, fungsi aktivasi seperti sigmoid dan tanh memiliki daerah saturasi pada nilai input yang sangat besar atau sangat kecil, yang dapat menyebabkan *vanishing gradient* dan menghambat proses pembelajaran (Goodfellow et al., 2016).

Salah satu metode normalisasi yang umum digunakan pada model *deep learning* untuk *time-series* adalah *Min-Max Scaling* (Han et al., 2011). Metode ini melakukan transformasi linier pada nilai data sehingga seluruh nilai berada dalam rentang $[0, 1]$ atau rentang lain yang ditentukan. Persamaan *Min-Max Scaling* untuk rentang $[0, 1]$ dinyatakan oleh Persamaan 2.2:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (2.2)$$

dengan keterangan:

x' : nilai hasil normalisasi;

x : nilai data aktual;

$x_{\min}$: nilai minimum data;

$x_{\max}$: nilai maksimum data.

Setelah model menghasilkan prediksi dalam rentang ternormalisasi, hasil prediksi tersebut perlu dikembalikan ke skala aslinya melalui proses denormalisasi. Persamaan denormalisasi diperoleh dengan menyelesaikan persamaan (2.2) terhadap x :

$$x = x' \cdot (x_{\max} - x_{\min}) + x_{\min} \quad (2.3)$$

Pada konteks penelitian *time-series forecasting*, perhitungan parameter $x_{\min}$ dan $x_{\max}$ harus dilakukan hanya pada data *training* untuk mencegah *data leakage* (Kaufman et al., 2012). Nilai $x_{\min}$ dan $x_{\max}$ yang diperoleh dari data *training* kemudian diterapkan untuk menormalisasi data *validation* dan *test*, sehingga estimasi performa model pada data baru bersifat *unbiased*.

2.4 Machine Learning

Machine learning merupakan salah satu subset dari *Artificial Intelligence* (AI) yang terfokus untuk memungkinkan komputer mempelajari data historis melalui pengenalan pola dan teori pembelajaran dengan metode komputasi (Simon and Singh, 2015). Metode *Machine learning* diprogram mirip dengan cara kerja otak manusia. Hal yang membedakan adalah manusia belajar dari pengalaman hidup sebelumnya, sementara komputer,

dalam konteks ini *machine learning*, belajar dari data historis. *Machine learning* menjadi krusial karena efektivitasnya untuk menemukan pola dan hubungan yang tidak terlihat secara intuitif oleh manusia. Dikutip dari Rebala et al. (2019), *Machine learning* dapat dikelompokkan menjadi 4 kategori:

- *Supervised Learning*

Supervised Learning adalah metode *machine learning* di mana *training* data yang digunakan pada model adalah data yang berlabel. *Training* data terdiri atas pasangan $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, dengan x_i sebagai fitur input dan y_i adalah label/target output. Tujuan dari *supervised learning* adalah untuk mempelajari pola output y untuk setiap input x yang baru (*test data*). Umumnya, *supervised learning* dibagi menjadi dua, yaitu klasifikasi (apabila variabel y diskrit atau kategorik) dan regresi (apabila variabel y kontinu).

- *Unsupervised Learning*

Unsupervised Learning adalah metode *machine learning* yang bekerja pada data yang tidak memiliki variabel target yang eksplisit. Dengan kata lain, *training* datanya terdiri dari input berupa $x_1, x_2, \dots, x_n$. Tujuan dari *unsupervised learning* adalah untuk menemukan pola tersembunyi atau struktur intrinsik pada *dataset*. Metode *unsupervised learning* ini meliputi *clustering* dan *dimensionality reduction*.

- *Semi-supervised Learning*

Semi-supervised learning mengombinasikan kedua metode, yaitu *unsupervised* dan *supervised learning* dengan menggunakan data berlabel dan tidak berlabel sebagai input *training*. Algoritma ini secara konsep mencoba memperbaiki *decision boundary* yang dihasilkan oleh metode *supervised learning* dengan memanfaatkan struktur topologi dari data *unsupervised learning*.

- *Reinforcement Learning*

Reinforcement learning merupakan metode yang belajar lewat proses *trial-and-error*, dibanding metode-metode *machine learning* lainnya yang belajar dari data historis. Masalah ini sering dimodelkan secara matematis dengan *Markov Decision Process* (MDP).

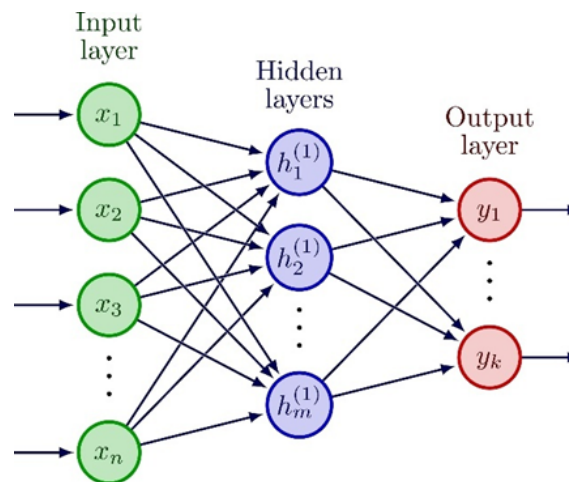
2.5 Deep Neural Network

Deep Neural Network (DNN) merupakan sekumpulan neuron buatan yang tersusun dalam beberapa lapisan. DNN adalah subset dari *machine learning* berdasarkan repre-

sentasi data hierarkis dan merupakan evolusi dari *Artificial Neural Network* yang secara konsep meniru struktur dan fungsi otak manusia (LeCun et al., 2015).

2.5.1 *Multilayer Perceptron (MLP)*

MLP merupakan bentuk paling dasar dari *neural network*, yaitu sebuah model komputasi sederhana yang mengambil input, menghitung *weighted sum*, dan mengaktivasi fungsi aktivasi untuk kemudian menghasilkan output (Haykin, 2009).



Gambar 2.1: Ilustrasi *Multilayer Perceptron* (Sumber: Goodfellow et al., 2016)

Pada dasarnya, *Deep Neural Network* merupakan *Multilayer Perceptron* dengan beberapa lapisan (Goodfellow et al., 2016). Untuk MLP, lapisan tersebut terdiri atas:

- *Input Layer*: lapisan ini menerima fitur sebagai input. Maka, jumlah neuron pada lapisan ini sama dengan jumlah fitur yang digunakan dalam *dataset*.
- *Hidden Layer*: lapisan ini merupakan lapisan yang memproses input yang diperoleh sebelum diteruskan ke lapisan selanjutnya. Pada lapisan inilah, sebagian besar proses komputasi terjadi. Dalam konteks *deep neural network*, terdapat lebih dari satu *hidden layer* dalam arsitektur.
- *Output Layer*: lapisan ini mendapat informasi dari *hidden layer* untuk mendapatkan hasil akhir dari model. Jumlah neuronnya bergantung pada jenis modelnya (misalnya, pada model regresi berarti *output layer* berjumlah satu neuron).

Ilustrasi *Multilayer Perceptron* menunjukkan aliran informasi dari kiri (input) ke kanan (output) melalui *hidden layer*, dengan setiap garis menunjukkan bobot (w). Setiap neuron terhubung ke semua neuron di lapisan sebelumnya dan direpresentasikan kekuatannya oleh *weight* (w). Setiap neuron juga mengandung *bias* (b) yang memengaruhi fungsi aktivasi untuk bergerak positif atau negatif.

2.5.2 Fungsi Aktivasi

Peran utama dari fungsi aktivasi adalah untuk memperkenalkan sifat non-linearitas. Tanpa fungsi aktivasi yang non-linear, sekompleks apa pun tumpukan *hidden layer* secara matematis akan setara dengan satu lapisan linear (Goodfellow et al., 2016).

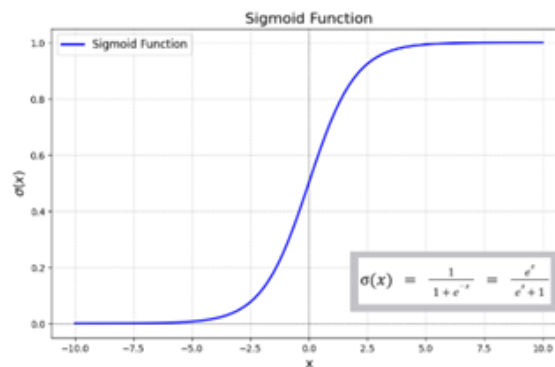
Fungsi aktivasi σ yang diterapkan pada jumlah input dan bias ($z = \sum w_i x_i + b$) dapat dinyatakan sebagai:

$$\text{Output} = \sigma(z) \quad (2.4)$$

Diantara beberapa fungsi aktivasi yang populer untuk digunakan:

- **Sigmoid:** Fungsi aktivasi ini menghasilkan output antara 0 dan 1. Popularitasnya menurun karena *vanishing gradient problem*. Pada penelitian ini, fungsi aktivasi sigmoid akan digunakan untuk menentukan perubahan laju *hidden state*, tepatnya pada *reset gate* dan *update gate*. Secara matematis, fungsi sigmoid dinyatakan sebagai berikut:

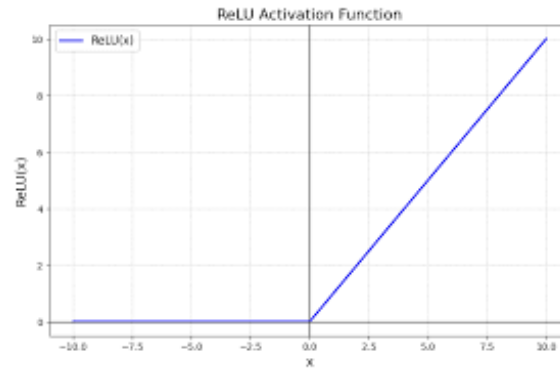
$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.5)$$



Gambar 2.2: Ilustrasi *Sigmoid* (Sumber: GeeksforGeeks, 2026)

- **ReLU:** *Rectified Linear Unit* (ReLU) adalah fungsi aktivasi yang paling populer saat ini karena efisiensi komputasinya dan efektif dalam mengatasi *vanishing gradient problem* (Nair and Hinton, 2010). Pada penelitian ini, fungsi aktivasi ReLU digunakan pada *output layer*. Secara matematis, fungsi ReLU dinyatakan sebagai berikut:

$$f(x) = \max(0, x) \quad (2.6)$$



Gambar 2.3: Ilustrasi ReLU

- Tanh: *Hyperbolic Tangent* (Tanh) adalah fungsi aktivasi nonlinier yang memetakan nilai input ke dalam interval $(-1, 1)$. Secara matematis, tanh dapat dituliskan sebagai berikut:

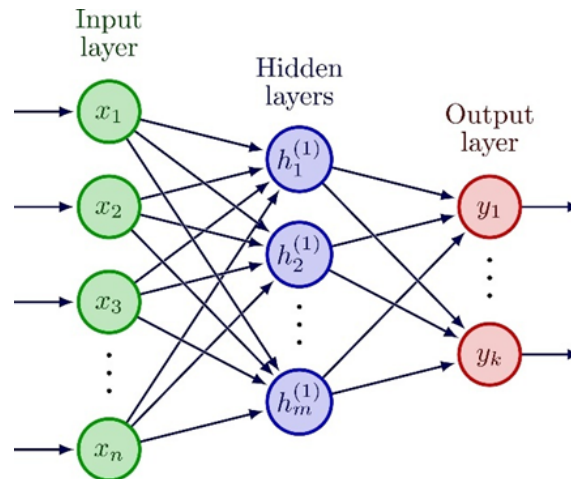
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2.7)$$

Keunggulan fungsi aktivasi ini dibanding sigmoid adalah karakteristiknya yang *zero-centered*. Hal ini membantu proses optimasi selama fase *backpropagation* yang mencegah terjadinya osilasi pada pembaruan bobot yang sering terjadi pada fungsi yang tidak *zero-centered* (LeCun et al., 2012). Pada penelitian ini, fungsi aktivasi *tanh* akan digunakan pada model GRU-ODE untuk menentukan derivatif dari *hidden state*, tepatnya pada gerbang *candidate hidden state*.

2.5.3 Backpropagation

Pembelajaran pada *Neural Network* pada dasarnya adalah proses iteratif untuk menyesuaikan parameter, yaitu bobot dan bias, guna meminimalkan kesalahan prediksi dan mencapai tingkat akurasi yang terbaik (Rumelhart et al., 1986). Kesalahan prediksi diukur menggunakan sebuah fungsi obyektif yang disebut *loss function*. Proses ini merupakan masalah optimasi yang diselesaikan melalui dua tahapan utama:

- *Forward Pass*: Pada tahap ini, data input berupa fitur masuk melalui *input layer* dan diteruskan secara sekuensial ke *hidden layer* hingga mencapai *output layer* untuk menghasilkan nilai prediksi ($\hat{y}$). *Forward pass* dapat diilustrasikan dengan sinyal input (x) yang merambat di mana setiap neuron melakukan operasi linier ($z = wx + b$) dan kemudian dilewatkan pada operasi non-linier melalui fungsi aktivasi. Di akhir tahapan ini, nilai prediksi ($\hat{y}$) dievaluasi terhadap nilai aktual target menggunakan *loss function* untuk menghitung besarnya kesalahan (*error*).



Gambar 2.4: Ilustrasi *Multilayer Perceptron* (Sumber: Goodfellow et al., 2016)

- *Backpropagation*: Setelah besaran *error* diketahui, informasi tersebut dikembalikan mundur dari *output layer* menuju *input layer*. Algoritma *backpropagation* memanfaatkan prinsip aturan rantai (*chain rule*) dari kalkulus untuk menghitung gradien dari *loss function* terhadap setiap bobot dan bias di seluruh jaringan secara efisien (Rumelhart et al., 1986). Gradien ini menunjukkan arah penyesuaian yang diperlukan untuk memperkecil *error*. Selanjutnya, sebuah algoritma optimasi (seperti *Gradient Descent* atau *Adam*) menggunakan nilai gradien ini beserta parameter laju pembelajaran (*learning rate*) untuk memperbarui nilai bobot dan bias secara sistematis.

Meskipun arsitektur *feedforward* seperti *Deep Neural Network* sangat tangguh dalam mempelajari representasi fitur statis, arsitektur ini memiliki keterbatasan karena tidak memiliki memori mengenai input sebelumnya, sehingga kurang optimal untuk memodelkan data sekuensial. Keterbatasan inilah yang menjadi cikal bakal pengembangan *Recurrent Neural Network* (RNN), sebuah arsitektur yang dirancang dengan memori internal sehingga sangat populer dan efektif digunakan untuk analisis data berurutan seperti data *time-series*.

2.6 Regularisasi

Dalam pelatihan model *deep learning*, terdapat risiko terjadinya *overfitting*, yaitu kondisi ketika model mempelajari pola data *training* secara berlebihan, termasuk *noise* di dalamnya, sehingga performanya menurun ketika dihadapkan pada data baru yang belum pernah diamati (Goodfellow et al., 2016). Kondisi *overfitting* ditandai oleh selisih yang

besar antara *error* pada data *training* dan data *validation* (*generalization gap*). Regularisasi adalah sekumpulan teknik yang dirancang untuk menurunkan *generalization error* tanpa harus menaikkan *training error* secara signifikan, sehingga kemampuan generalisasi model menjadi lebih baik (Goodfellow et al., 2016).

Model berbasis *recurrent neural network* seperti GRU memiliki jumlah parameter yang besar sehingga rentan mengalami *overfitting*, terlebih pada data *time-series* finansial yang umumnya memiliki rasio *signal-to-noise* yang rendah. Oleh karena itu, penerapan regularisasi menjadi penting untuk menjaga stabilitas dan kemampuan generalisasi model. Pada penelitian ini digunakan dua teknik regularisasi, yaitu *early stopping* dan *dropout*.

2.6.1 *Early Stopping*

Early stopping adalah teknik regularisasi yang menghentikan proses pelatihan sebelum model mengalami *overfitting* (Goodfellow et al., 2016). Selama pelatihan, *training error* cenderung terus menurun seiring bertambahnya *epoch*, sedangkan *validation error* pada umumnya menurun pada awal *training*, lalu kembali meningkat ketika model mulai mengalami *overfitting*. *Early stopping* memanfaatkan fenomena tersebut dengan memantau nilai *loss* pada data *validation* di setiap *epoch* dan menyimpan bobot model yang menghasilkan *validation loss* terendah. Selain berperan sebagai regularisasi, *early stopping* juga menurunkan biaya komputasi karena pelatihan tidak perlu dijalankan hingga seluruh *epoch* maksimum tercapai (Goodfellow et al., 2016).

2.6.2 *Dropout*

Dropout adalah teknik regularisasi yang bekerja dengan menonaktifkan sebagian neuron secara acak pada setiap iterasi pelatihan (Srivastava et al., 2014). Pada setiap *forward pass*, setiap neuron memiliki probabilitas tertentu untuk dinonaktifkan, yaitu output-nya diatur menjadi nol, sehingga model dipaksa untuk tidak bergantung secara berlebihan pada neuron tertentu. Mekanisme ini setara dengan melatih banyak sub-jaringan (*thinned networks*) yang berbeda dan merata-ratakan prediksinya, sehingga menyerupai metode *ensemble* (Srivastava et al., 2014). Penelitian ini menggunakan *inverted dropout* yang dituangkan pada algoritma berikut:

Algorithm 1 *Dropout*

Require: Aktivasi a , probabilitas *dropout* p , mode (*training* atau *inference*)

if mode = *training* **then**

Sampel untuk *dropout*:

$$v \sim \text{Bernoulli}(1 - p)$$

Terapkan *dropout*:

$$a \leftarrow a \odot v$$

Lakukan *inverted dropout scaling*:

$$a \leftarrow \frac{a}{1 - p}$$

else

$a \leftarrow a$ ▷ Tidak ada *dropout* saat inferensi

end if

return a

2.7 Gated Recurrent Unit (GRU)

Gated Recurrent Unit atau biasa disebut GRU adalah suatu adaptasi dari model *Long-Short Term Memory* (LSTM) yang diperkenalkan oleh (Cho et al., 2014). GRU, yang merupakan keluarga dari RNN, menyederhanakan arsitektur LSTM dengan menggabungkan *forget gate* dan *input gate* menjadi satu *update gate*. Dengan demikian, GRU hanya memiliki dua *gate* dalam arsitekturnya:

- *Update Gate* (z_t). *Update gate* berperan dalam menentukan informasi dari *hidden state* sebelumnya (h_{t-1}) yang akan ditinggalkan. *Gate* ini mengevaluasi relevansi dari *hidden state* sebelumnya untuk meng-*update hidden state* yang baru.
- *Reset Gate* (r_t). *Reset gate* berperan dalam menentukan informasi dari *hidden state* sebelumnya (h_{t-1}) yang akan diteruskan ke *hidden state* h_t . Selain itu, *reset gate* juga berperan menentukan kandidat *hidden state* baru ($\tilde{h}_t$) yang akan ditambahkan.

Secara matematis, algoritma model *Gated Recurrent Unit* dapat dituliskan sebagai (Cho et al., 2014):

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (2.8)$$

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (2.9)$$

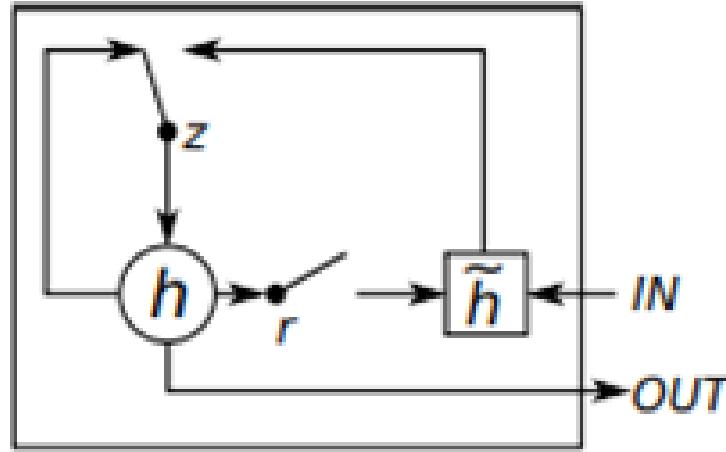
$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \quad (2.10)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (2.11)$$

Dengan:

z_t : *update gate*;

r_t : *reset gate*;



Gambar 2.5: Ilustrasi Struktur GRU (Sumber: Chung et al., 2014)

$\tilde{h}_t$: kandidat *hidden state* baru;

h_{t-1} : *hidden state* dari langkah waktu sebelumnya;

h_t : *hidden state* final;

σ : fungsi aktivasi sigmoid dengan output 0 sampai 1;

$\tanh$: fungsi aktivasi tangen hiperbolik dengan output -1 sampai 1;

$\odot$: fungsi perkalian *element-wise* (Hadamard product);

W_h, W_r, W_z : matriks bobot yang dipelajari model untuk setiap *gate*;

U_h, U_r, U_z : matriks bobot *recurrent* yang menghubungkan *hidden state* lalu (h_{t-1}) dengan masing-masing *gate*;

b_h, b_r, b_z : vektor bias pada setiap *gate*;

x_t : input data aktual pada waktu t .

2.8 Ordinary Differential Equation

Ordinary Differential Equation (ODE) adalah persamaan yang melibatkan sebuah fungsi satu variabel independen (umumnya waktu, t) dan satu atau lebih turunannya (derivatif) terhadap variabel tersebut (Zill, 2013). Bentuk umum dari ODE orde pertama dapat direpresentasikan sebagai berikut:

$$\frac{dy}{dt} = f(t, y) \quad (2.12)$$

Dengan fungsi $y = y(t)$ merepresentasikan fungsi yang ingin dicari, sedangkan f merupakan fungsi yang mendefinisikan perubahan y terhadap t .

Dalam konteks Neural ODE, diperlukan metode *solver* ODE secara numerik untuk memodelkan evolusi kontinu dari *hidden state* dan kedua titik observasi. Pemilihan metode yang sesuai juga sangat penting demi keseimbangan komputasi dan akurasi. Dalam konteks metode numerik, yang digunakan dalam mengaproksimasi solusi masalah nilai awal, terdapat beberapa metode yang populer untuk digunakan:

2.8.1 Metode Euler

Metode Euler merupakan cara mengaproksimasi ODE yang paling sederhana dan cepat secara komputasi, namun sangat rawan dengan error yang besar jika nilai h semakin besar. Metode Euler bekerja dengan memperkirakan nilai dengan *slope* (kemiringan) fungsi f pada titik saat ini (Zill, 2013):

$$y_{n+1} = y_n + h f(t_n, y_n) \quad (2.13)$$

Dengan $h = t_{n+1} - t_n$ adalah *stepsize* waktu.

2.8.2 Metode Runge-Kutta Orde 4

Metode Runge-Kutta orde 4 (RK4) populer karena mempunyai keseimbangan yang baik dalam akurasi dan biaya komputasi. Metode ini bekerja dengan mengevaluasi fungsi f pada titik-titik berikut (Zill, 2013):

$$k_1 = f(t_n, y_n) \quad (2.14)$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \quad (2.15)$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \quad (2.16)$$

$$k_4 = f(t_n + h, y_n + hk_3) \quad (2.17)$$

Lalu, nilai berikutnya dinyatakan sebagai berikut:

$$y_{n+1} = y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4) \quad (2.18)$$

2.9 Neural Ordinary Differential Equation

Neural Ordinary Differential Equation (Neural ODE) diperkenalkan oleh Chen et al. (2018) sebagai pendekatan yang secara fundamental berbeda dari arsitektur *deep learning* pada umumnya. Sebagian besar model *deep learning* sekuensial seperti RNN, LSTM, dan GRU memproses data secara diskrit, yaitu *hidden state* diperbarui langkah demi langkah

pada setiap titik observasi. Sedangkan, Neural ODE memodelkan dinamika sistem sebagai proses yang kontinu dengan memparameterisasi derivatif dari *hidden state* terhadap waktu menggunakan *neural network*. Dengan cara ini, model mempelajari fungsi dinamika langsung dari data, sehingga cocok digunakan untuk pemodelan *irregular time-series*, baik untuk *forecasting*, interpolasi, maupun imputasi *missing values*.

Neural ODE secara matematis memparameterisasi derivatif dari *hidden state* $h(t)$ melalui masalah nilai awal berikut (Chen et al., 2018):

$$\begin{aligned}\frac{dh(t)}{dt} &= f_{\theta}(h(t), t), \\ h(t_0) &= h_0,\end{aligned}\tag{2.19}$$

dengan f_{θ} merepresentasikan fungsi dinamika yang diparameterisasi oleh *neural network* dengan parameter θ . *Neural network* konvensional memetakan *input* langsung ke nilai target secara diskrit, sedangkan Neural ODE bekerja dengan menghasilkan laju evolusi *hidden state* terhadap waktu, yaitu $\frac{dh(t)}{dt}$, yang kemudian diintegrasikan untuk memperoleh $h(t)$.

Karena f_{θ} umumnya berbentuk nonlinier dan kompleks, solusi analitik dari Persamaan (2.19) tidak mungkin diperoleh melalui integrasi langsung. Oleh sebab itu, nilai *hidden state* pada rangkaian titik waktu dievaluasi menggunakan *solver* numerik (*ODE Solver*) yang secara matematis dinyatakan sebagai (Chen et al., 2018):

$$h(t_0), h(t_1), \dots, h(t_N) = \text{ODESolve}(f_{\theta}, h(t_0), (t_0, t_1, \dots, t_N)).\tag{2.20}$$

Operator ODE Solver menyelesaikan masalah nilai awal dengan tiga input utama: fungsi dinamika f_{θ} , kondisi awal *hidden state* $h(t_0)$, dan rangkaian titik waktu $(t_0, t_1, \dots, t_N)$. Outputnya berupa nilai-nilai *hidden state* yang telah terintegrasi sepanjang lintasan waktu, sehingga model mampu mengestimasi *state* pada sembarang titik waktu secara kontinu meskipun observasinya hanya tersedia pada titik waktu yang diskrit.

Untuk kasus *forecasting* pada data *time-series*, Chen et al. (2018) memperluas persamaan di atas menjadi sebuah model variabel laten (*latent variable model*). Pada formulasi ini, *hidden state* diperlakukan sebagai *latent state* z_t , sehingga setiap *time-series* di-representasikan sebagai sebuah lintasan di ruang laten (*latent trajectory*). Lintasan tersebut ditentukan oleh dua komponen: keadaan awal lokal z_{t_0} yang spesifik untuk masing-masing deret, dan fungsi dinamika laten f_{θ} yang berlaku global bagi seluruh *dataset*.

Secara matematis, model generatif ini dinyatakan sebagai (Chen et al., 2018):

$$z_{t_0} \sim p(z_{t_0}), \quad (2.21)$$

$$z_{t_0}, z_{t_1}, \dots, z_{t_N} = \text{ODESolve}(f_\theta, z_{t_0}, (t_0, t_1, \dots, t_N)), \quad (2.22)$$

$$x_{t_i} \sim p(x \mid z_{t_i}, \theta). \quad (2.23)$$

Persamaan (2.21) menyatakan bahwa keadaan awal z_{t_0} ditarik dari distribusi *prior*, dinamika laten kemudian diintegrasikan oleh ODESolve untuk memperoleh seluruh lintasan, dan setiap observasi x_{t_i} di-generate dari *latent state* pada waktu tersebut melalui distribusi *likelihood*. Keterangan simbol yang digunakan adalah sebagai berikut:

$h(t)$: *hidden state* kontinu pada waktu t ;

z_t : *latent state* pada formulasi generatif/forecasting;

x_{t_i} : observasi pada waktu t_i ;

f_θ : fungsi dinamika yang diparameterisasi *neural network*;

θ : parameter *neural network*;

ODESolver : *solver* numerik untuk menyelesaikan ODE;

p : fungsi densitas probabilitas (*prior* dan *likelihood*).

2.10 GRU-ODE

Model *Gated Recurrent Unit-Ordinary Differential Equation* (GRU-ODE) yang diperkenalkan oleh De Brouwer et al. (2019) merupakan perluasan kontinu dari *Gated Recurrent Unit* (GRU) yang dirancang untuk menjawab keterbatasan GRU dalam memodelkan dinamika antar observasi yang tidak seragam pada data *time-series*. Pada GRU klasik, *hidden state* hanya diperbarui pada setiap titik observasi sehingga interval waktu antar observasi tidak diperhitungkan secara eksplisit. GRU-ODE mengatasi hal tersebut dengan merumuskan evolusi *hidden state* sebagai sebuah *Ordinary Differential Equation* (ODE) yang menghitung laju evolusi *hidden state* dari *gating mechanism* GRU.

Secara umum, GRU-ODE mendefinisikan laju evolusi *hidden state* $h(t)$ melalui persamaan diferensial:

$$\frac{dh(t)}{dt} = (1 - z_t) \odot (g_t - h(t)), \quad t \in (t_{i-1}, t_i], \quad (2.24)$$

dengan kondisi awal $h(t_{i-1}) = h_{i-1}$. Komponen z_t (*update gate*) dan g_t (kandidat *hidden state*) dihitung dari *input* x dan *state* terkini $h(t)$ menggunakan *gating mechanism* GRU

sebagai berikut:

$$r_t = \sigma(W_r[x, h(t)] + b_r), \quad (2.25)$$

$$z_t = \sigma(W_z[x, h(t)] + b_z), \quad (2.26)$$

$$g_t = \tanh(W_g[x, r_t \odot h(t)] + b_g), \quad (2.27)$$

dengan keterangan:

$h(t)$: *hidden state* kontinu pada waktu t ;

x : *input* yang berlaku pada interval $(t_{i-1}, t_i]$;

r_t : *reset gate*;

z_t : *update gate*;

g_t : kandidat *hidden state*;

W_r, W_z, W_g : matriks bobot;

b_r, b_z, b_g : vektor bias;

$\sigma, \tanh$: fungsi aktivasi sigmoid dan tangen hiperbolik;

$\odot$: Hadamard *product*.

Bentuk $\frac{dh(t)}{dt} = (1 - z_t) \odot (g_t - h(t))$ merupakan limit kontinu dari persamaan GRU diskrit $h_i = z_t \odot h(t) + (1 - z_t) \odot g_t$ ketika selang waktu antar observasi mendekati nol (De Brouwer et al., 2019). Perlu dicatat bahwa konvensi pembobotan *update gate* pada GRU-ODE merupakan kebalikan dari konvensi GRU. Pada formulasi GRU, $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$, sehingga $z_t \rightarrow 1$ berarti *hidden state* sepenuhnya diperbarui ke kandidat baru, sebaliknya pada GRU-ODE, $z_t \rightarrow 1$ justru berarti *hidden state* hampir tidak berubah. Kedua konvensi ekuivalen melalui hubungan $z_t^{(\text{GRU-ODE})} = 1 - z_t^{(\text{Cho})}$, dengan kandidat *hidden state* yang dinotasikan $\tilde{h}_t$ pada GRU dinyatakan sebagai g_t pada GRU-ODE.

2.10.1 Integrasi Numerik *Hidden State*

Karena solusi analitik dari persamaan diferensial di atas umumnya tidak dapat diperoleh secara eksplisit, evolusi *hidden state* dari waktu t_{i-1} ke t_i diaproksimasi menggunakan *solver* numerik. Menurut De Brouwer et al. (2019), selang $\Delta t_i = t_i - t_{i-1}$ dibagi

menjadi K sub-langkah berukuran $\delta t = \Delta t_i / K$, lalu *hidden state* diperbarui secara iteratif. Dua ODE Solver yang digunakan dan dibandingkan adalah:

$$\text{Euler: } h_{k+1} = h_k + \delta t \cdot f(h_k), \quad (2.28)$$

$$\text{RK4: } h_{k+1} = h_k + \frac{\delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4), \quad (2.29)$$

dengan titik-titik yang dievaluasi secara berurutan dalam *Runge-Kutta* orde 4:

$$\begin{aligned} k_1 &= f(h_k), \\ k_2 &= f\left(h_k + \frac{\delta t}{2} k_1\right), \\ k_3 &= f\left(h_k + \frac{\delta t}{2} k_2\right), \\ k_4 &= f(h_k + \delta t k_3). \end{aligned} \quad (2.30)$$

Fungsi $f(h) = (1 - z) \odot (g - h)$ adalah fungsi dinamika *autonomous* yang telah didefinisikan sebelumnya. Metode Euler memiliki kompleksitas komputasi paling rendah, sementara RK4 menawarkan stabilitas dan akurasi numerik yang jauh lebih tinggi meskipun membutuhkan biaya komputasi yang lebih besar.

Setelah seluruh observasi input dihitung melalui ODE Solver, *hidden state* akhir $h(t_N)$ digunakan sebagai representasi dari keseluruhan urutan dan diteruskan ke *output layer* untuk menghasilkan hasil *forecast* indeks harga saham LQ45 pada *timestamp* berikutnya.

2.10.2 Backpropagation pada GRU-ODE

Pelatihan model GRU-ODE dilakukan dengan meminimalkan fungsi *loss* terhadap parameter $\theta = \{W_r, W_z, W_g, b_r, b_z, b_g\}$ menggunakan metode optimisasi *gradient descent*. Tantangan utama pada pelatihan GRU-ODE adalah evolusi *hidden state* $h(t)$ tidak dihitung dalam satu langkah, melainkan melalui iterasi ODE solver sebanyak K sub-langkah per interval observasi. Oleh karena itu, perhitungan *gradient* terhadap parameter harus melewati seluruh sub-langkah tersebut, sehingga diperlukan adaptasi khusus dari mekanisme *backpropagation* konvensional.

Menurut Chen et al. (2018), terdapat dua pendekatan utama untuk menghitung *gradient* pada model berbasis Neural ODE:

2.10.2.1 *Discretize-then-Optimize (DTO)*

Pendekatan pertama adalah *Discretize-then-Optimize* (DTO) atau lebih dikenal sebagai *backpropagation through the solver*. Pada pendekatan ini, setiap sub-langkah *solver* diperlakukan sebagai operasi diferensiabel biasa. Seluruh sub-langkah direkam dalam *computational graph*, sehingga *gradient* dapat dihitung secara langsung melalui aturan rantai menggunakan algoritma *automatic differentiation* (Baydin et al., 2018). Pendekatan ini memberikan akurasi *gradient* yang tinggi karena perhitungan dilakukan secara eksak terhadap proses diskretisasi numerik, namun memerlukan memori yang proporsional terhadap jumlah sub-langkah karena seluruh *intermediate state* harus disimpan.

2.10.2.2 *Adjoint Method*

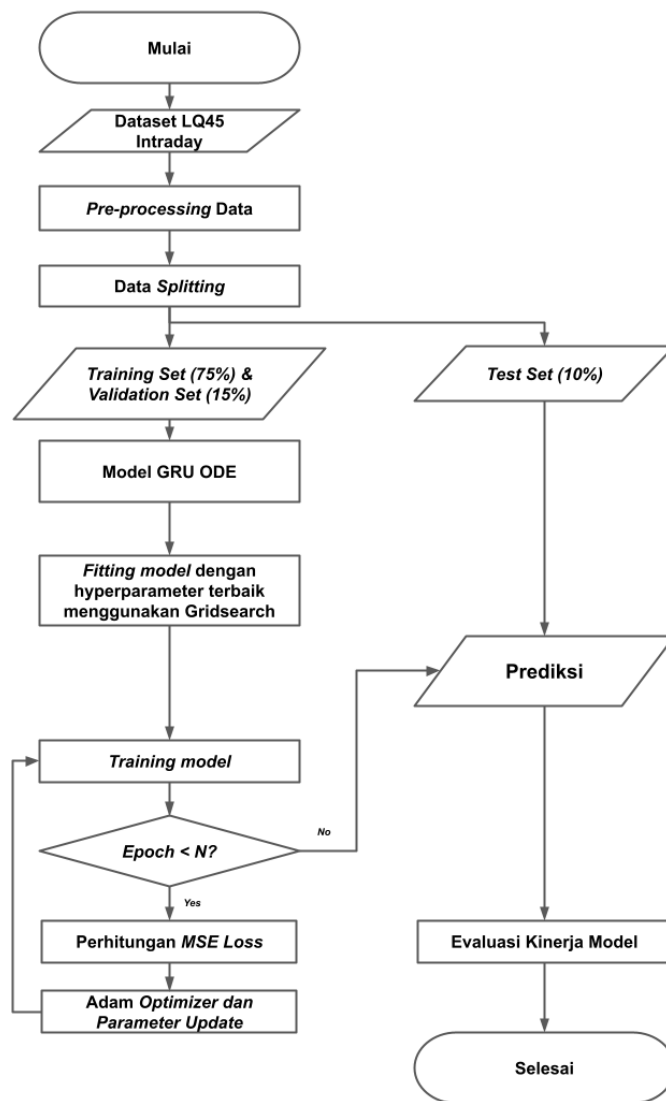
Pendekatan kedua adalah *adjoint method* yang diadopsi dalam konteks Neural ODE oleh Chen et al. (2018). Pendekatan ini tidak menyimpan *computational graph* dari sub-langkah *solver*; sebagai gantinya, *gradient* diperoleh dengan menyelesaikan persamaan *adjoint ODE* yang berjalan mundur dari waktu akhir t_1 ke waktu awal t_0 . *Adjoint method* memerlukan memori konstan $\mathcal{O}(1)$ terlepas dari jumlah sub-langkah, tetapi memiliki kelemahan berupa potensi akumulasi galat numerik dan biaya komputasi tambahan untuk integrasi *adjoint*.

Pemilihan antara kedua pendekatan tersebut bergantung pada kondisi penelitian. *Backpropagation through the solver* lebih cocok untuk pelatihan dengan jumlah sub-langkah yang relatif kecil dan kebutuhan akurasi *gradient* yang tinggi, sedangkan *adjoint method* lebih cocok untuk pelatihan dengan jumlah sub-langkah yang besar atau pada kondisi keterbatasan memori. Pada penelitian ini digunakan pendekatan *backpropagation through the solver* dengan pertimbangan jumlah sub-langkah yang relatif kecil ($K = 4$) dan akurasi *gradient* yang lebih terjamin.

BAB 3

PEMBENTUKAN MODEL *GATED RECURRENT UNIT-ORDINARY DIFFERENTIAL EQUATION* (GRU-ODE)

Pada bagian ini, akan dijelaskan konsep dasar dan langkah-langkah penelitian prediksi Indeks Harga Saham LQ45 dengan model GRU-ODE. Alur umum yang dilalui ketika melakukan penelitian ini tercermin dalam *flowchart* 3.1 berikut.



Gambar 3.1: Bagan Alur Penelitian

3.1 Konsep Dasar *Gated Recurrent Unit – Ordinary Differential Equation*

De Brouwer et al. (2019) memperkenalkan arsitektur *Gated Recurrent Unit – Ordinary Differential Equation* (GRU-ODE) pada tahun 2019 sebagai pengembangan dari *Recurrent Neural Network* (RNN) yang mampu merepresentasikan dinamika *hidden state* secara kontinu untuk data *time-series* yang diobservasi pada interval yang tidak seragam. Arsitektur ini lahir dari perpaduan dua ide, yaitu *Gated Recurrent Unit* (GRU) yang diperkenalkan oleh Cho et al. (2014) dan *Neural Ordinary Differential Equation* (*Neural ODE*) yang diperkenalkan oleh Chen et al. (2018).

GRU merupakan salah satu varian RNN yang dirancang untuk memodelkan data sekuensial dengan dependensi jangka panjang. *Neural ODE*, di sisi lain, menawarkan paradigma yang merepresentasikan evolusi *hidden state* sebagai solusi persamaan diferensial biasa (*ordinary differential equation*, ODE) yang diselesaikan secara numerik menggunakan *ODE solver*. Pendekatan ini memungkinkan *hidden state* dievaluasi pada titik waktu kapan pun, tidak lagi terbatas pada asumsi interval waktu diskrit yang seragam.

Pada GRU-ODE, evolusi *hidden state* di antara titik-titik observasi dimodelkan melalui persamaan diferensial yang diturunkan dari bentuk pembaruan diskrit GRU, sedangkan pembaruan informasi pada saat observasi baru tiba tetap mengikuti mekanisme *gating* GRU. Perpaduan ini menghasilkan model *hybrid* di mana *hidden state* berevolusi secara kontinu di antara observasi dan mengalami pembaruan diskrit (*discrete jump*) pada saat observasi diterima.

3.2 Pemodelan *Gated Recurrent Unit*

GRU adalah salah satu varian dari *Recurrent Neural Network* (RNN) yang dikembangkan oleh Cho et al. (2014) pada tahun 2014. GRU dirancang untuk mengatasi masalah *vanishing gradient* yang sering terjadi pada RNN konvensional ketika memproses data sekuensial dengan dependensi jangka panjang. GRU merupakan penyederhanaan dari *Long Short-Term Memory* (LSTM) dengan jumlah parameter yang lebih sedikit, namun tetap memiliki performa yang kompetitif. GRU menggunakan mekanisme *gating* untuk mengontrol aliran informasi dalam jaringan yaitu *update gate* dan *reset gate*, serta satu *hidden state*.

Secara matematis, algoritma model *Gated Recurrent Unit* dapat dituliskan sebagai

(Cho et al., 2014):

$$r_t = \sigma(W_r[x_t, h_{t-1}] + b_r) \quad (3.1)$$

$$z_t = \sigma(W_z[x_t, h_{t-1}] + b_z) \quad (3.2)$$

$$g_t = \tanh(W_g[x_t, r_t \odot h_{t-1}] + b_g) \quad (3.3)$$

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot g_t \quad (3.4)$$

dengan r_t adalah *reset gate*, z_t adalah *update gate*, g_t adalah kandidat *hidden state*, dan h_t adalah *hidden state* pada waktu ke- t . Notasi $[x_t, h_{t-1}]$ menyatakan operasi *concatenation* antara vektor input x_t dan *hidden state* sebelumnya h_{t-1} . Fungsi $\sigma(\cdot)$ adalah fungsi aktivasi sigmoid dan $\odot$ adalah operasi Hadamard (perkalian elemen). Matriks W_r, W_z, W_g merupakan bobot yang dipelajari selama pelatihan, sedangkan b_r, b_z, b_g adalah vektor bias.

Reset gate r_t mengontrol seberapa besar informasi dari *hidden state* sebelumnya h_{t-1} diabaikan saat menghitung kandidat *hidden state* g_t . Apabila r_t bernilai mendekati nol, model akan mengabaikan informasi historis dan hanya mempertimbangkan input saat ini. Sebaliknya, *update gate* z_t mengatur proporsi antara *hidden state* sebelumnya dan kandidat *hidden state* yang akan dipertahankan pada *hidden state* baru h_t . Nilai z_t yang besar berarti model mempertahankan sebagian besar informasi historis, sedangkan nilai z_t yang kecil berarti model lebih banyak mengadopsi informasi dari kandidat *hidden state*.

Formulasi pembaruan *hidden state* pada persamaan (3.4) mengikuti konvensi yang digunakan oleh De Brouwer et al. (2019). Konvensi ini setara dengan formulasi asli Cho et al. (2014) melalui reparameterisasi $z_t \leftrightarrow (1 - z_t)$. Selain itu, notasi *concatenation* $W_r[x_t, h_{t-1}]$ digunakan menggantikan bentuk $W_r x_t + U_r h_{t-1}$ karena lebih merepresentasikan implementasi praktis pada *framework deep learning* dan memudahkan derivasi turunan parsial pada proses Optimasi Parameter (Backpropagation). Himpunan parameter yang akan dipelajari dalam *training* adalah $\theta = \{W_r, W_z, W_g, b_r, b_z, b_g\}$.

3.3 Pemodelan GRU-ODE

Persamaan laju perubahan *hidden state* pada GRU-ODE diturunkan dari bentuk diskrit GRU melalui persamaan diferensial biasa. Persamaan pembaruan *hidden state* pada GRU adalah sebagai berikut (Cho et al., 2014):

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot g_t \quad (3.5)$$

dengan z_t adalah *update gate* dan g_t adalah kandidat *hidden state*. Dengan mengurangi h_{t-1} pada kedua ruas, persamaan tersebut dapat ditulis ulang menjadi bentuk persamaan:

$$\begin{aligned} h_t - h_{t-1} &= z_t \odot h_{t-1} + (1 - z_t) \odot g_t - h_{t-1} \\ h_t - h_{t-1} &= (1 - z_t) \odot g_t - (1 - z_t) \odot h_{t-1} \\ \Delta h &= (1 - z_t) \odot (g_t - h_{t-1}) \end{aligned} \quad (3.6)$$

Persamaan di atas secara struktural menyerupai langkah Euler forward dari suatu persamaan diferensial biasa berbentuk $\frac{dh}{dt} = f(h, x)$, yaitu $h_{t+1} - h_t = \Delta t \cdot f(h_t, x_t)$ dengan ukuran langkah $\Delta t = 1$ yang berlaku secara implisit pada GRU diskrit. Berdasarkan kemiripan struktural ini, De Brouwer et al. (2019) mendefinisikan versi waktu-kontinu dari pembaruan GRU dengan mempostulatkan persamaan diferensial yang ruas kanannya berbentuk identik dengan Δh pada formulasi diskrit. Persamaan diferensial yang mendefinisikan dinamika *hidden state* pada GRU-ODE adalah (De Brouwer et al., 2019):

$$\frac{dh(t)}{dt} = (1 - z(t)) \odot (g(t) - h(t)) \quad (3.7)$$

dengan komponen *gate* dan *candidate* yang bergantung pada waktu didefinisikan sebagai:

$$r(t) = \sigma(W_r[x_t, h(t)] + b_r) \quad (3.8)$$

$$z(t) = \sigma(W_z[x_t, h(t)] + b_z) \quad (3.9)$$

$$g(t) = \tanh(W_g[x_t, r(t) \odot h(t)] + b_g) \quad (3.10)$$

Kedua sifat ini secara fundamental berfungsi sebagai *continuity prior* yang memastikan *hidden state* berevolusi secara mulus dan tidak mengalami fluktuasi ekstrem di antara dua titik observasi. Hal ini menjadi salah satu keunggulan utama GRU-ODE dalam pemodelan data *time-series* yang tidak seragam. Untuk memudahkan notasi pada pembahasan selanjutnya, kerangka pemodelan ini mengikuti konvensi parameterisasi derivatif *hidden state* oleh *neural network* yang pertama kali diperkenalkan pada arsitektur *Neural ODE* (Chen et al., 2018):

$$\frac{dh(t)}{dt} = f(h(t), t, \theta) \quad (3.11)$$

Pada formulasi asli Chen et al. (2018) tersebut, fungsi f hanya bergantung pada *hidden state* $h(t)$ dan waktu t , dengan θ sebagai parameter jaringan. Pada arsitektur GRU-

ODE (De Brouwer et al., 2019), formulasi ini diadaptasi dengan mengadopsi vektor input observasi $x(t)$. Untuk menyederhanakan penulisan, parameter jaringan θ lazim dituliskan sebagai subskrip, sehingga persamaan diferensial GRU-ODE secara utuh dapat dinotasikan dalam bentuk sebagai berikut:

$$\frac{dh(t)}{dt} = f_{\theta}(h(t), x(t)) = (1 - z(t)) \odot (g(t) - h(t)) \quad (3.12)$$

dengan $\theta = \{W_r, W_z, W_g, b_r, b_z, b_g\}$ merupakan himpunan parameter GRU-ODE yang dipelajari selama pelatihan. Dengan formulasi tersebut, *hidden state* $h(t)$ berevolusi secara kontinu terhadap waktu.

Setelah diperoleh representasi *hidden state* final yang dinotasikan sebagai $h_N \equiv h(t_i)$, akan dipetakan representasi *hidden state* ini menjadi nilai (*closing price*) menggunakan sebuah *output layer* berupa transformasi linier sederhana:

$$\hat{y}_i = W_o h_N + b_o \quad (3.13)$$

dengan W_o merupakan matriks bobot proyeksi dan b_o adalah nilai bias pada *output layer*.

3.4 Integrasi Numerik pada GRU-ODE

Persamaan diferensial pada (3.12) tidak memiliki solusi analitik karena keberadaan fungsi aktivasi nonlinier (*sigmoid dan tanh*) di dalam f_{θ} . Oleh sebab itu, evolusi *hidden state* dari t_0 ke t_1 harus diselesaikan secara numerik menggunakan *ODE solver*, seperti metode Euler atau Runge-Kutta orde 4 (RK4). Secara umum, evolusi *hidden state* pada interval $[t_0, t_1]$ dapat dinyatakan dalam bentuk integral:

$$h(t_1) = h(t_0) + \int_{t_0}^{t_1} f_{\theta}(h(\tau), x(\tau)) d\tau \quad (3.14)$$

dengan f_{θ} sebagaimana didefinisikan pada persamaan (3.12). Pada implementasi ini, vektor input $x(\tau)$ diperlakukan konstan selama interval integrasi, yaitu $x(\tau) = x_{t_i}$ untuk $\tau \in [t_i, t_{i+1}]$, sehingga hanya *hidden state* $h(\tau)$ yang berevolusi secara kontinu di antara dua observasi.

Integrasi numerik dilakukan dengan membagi interval waktu antar observasi (*elapsed time*) $\Delta t_i = t_{i+1} - t_i$ menjadi $N = 4$ substep (De Brouwer et al., 2019). Ukuran langkah pada setiap substep dirumuskan sebagai $\delta t = \Delta t_i / N$. Pada setiap substep, *hidden state* diperbarui menggunakan *ODE solver* yang dipilih, yaitu metode Euler atau Runge-Kutta

orde 4 (RK4).

Untuk memperjelas mekanisme kedua *ODE solver* yang digunakan, pada subseksi berikut diberikan ilustrasi perhitungan dua langkah pada GRU-ODE dengan konfigurasi skalar sederhana. Diasumsikan *hidden state* berdimensi satu ($d = 1$) dengan kondisi awal $h_0 = 0$, input observasi $x = 0,5$, serta bobot dan bias *gate* sebagai berikut: $W_r = W_z = W_g = 0,5$ dan $b_r = b_z = b_g = 0$. Dengan asumsi tersebut, fungsi turunan pada persamaan (3.12) menjadi:

$$f_\theta(h, x) = (1 - \sigma(0,5(x + h))) \cdot (\tanh(0,5(x + r \cdot h)) - h) \quad (3.15)$$

dengan $r = \sigma(0,5(x + h))$.

3.4.1 Integrasi Numerik dengan Metode Euler

Metode Euler merupakan *ODE solver* yang melakukan satu evaluasi fungsi per substep. Diberikan ukuran substep $\delta t = \Delta t_i / N$, pembaruan *hidden state* pada substep ke- k dirumuskan sebagai:

$$h_{k+1} = h_k + \delta t \cdot f_\theta(h_k, x_{t_i}) \quad (3.16)$$

Metode Euler memiliki *local truncation error* sebesar $\mathcal{O}(\delta t^2)$ dan *global truncation error* sebesar $\mathcal{O}(\delta t)$. Kelebihannya adalah komputasi yang ringan, namun akurasi terbatas terutama pada dinamika yang berubah cepat atau ukuran substep yang besar.

Dengan konfigurasi yang telah ditetapkan pada (3.16), perhitungan dua langkah metode Euler dilakukan sebagai berikut. Langkah pertama dimulai dari $h_0 = 0$ dan $x = 0,5$. Komponen *reset gate* dihitung terlebih dahulu:

$$\begin{aligned} r_0 &= z_0 = \sigma(0,5(0,5 + 0)) = \sigma(0,25) = 0,5622 \\ g_0 &= \tanh(0,5(0,5 + 0,5622 \cdot 0)) = \tanh(0,25) = 0,2449 \end{aligned}$$

Substitusi menghasilkan nilai turunan:

$$f_\theta(h_0, x) = (1 - 0,5622) \cdot (0,2449 - 0) = 0,1072$$

Sehingga *hidden state* setelah langkah pertama:

$$h_1 = 0 + 0,5 \cdot 0,1072 = 0,0536$$

Langkah kedua dimulai dari $h_1 = 0,0536$. Komponen *gate* dihitung ulang dengan nilai

hidden state yang baru:

$$r_1 = z_1 = \sigma(0,5(0,5 + 0,0536)) = \sigma(0,2768) = 0,5688$$

$$g_1 = \tanh(0,5(0,5 + 0,5688 \cdot 0,0536)) = \tanh(0,2652) = 0,2592$$

$$f_\theta(h_1, x) = (1 - 0,5688) \cdot (0,2592 - 0,0536) = 0,0887$$

$$h_2 = 0,0536 + 0,5 \cdot 0,0887 = 0,0979$$

Hasil akhir setelah dua langkah Euler adalah $h_2^{\text{Euler}} = 0,0979$.

3.4.2 Integrasi Numerik dengan Metode RK4

Metode RK4 merupakan ODE *solver* orde empat yang melakukan empat evaluasi fungsi per substep untuk memperoleh aproksimasi yang lebih akurat. Pembaruan *hidden state* pada substep ke- k dirumuskan sebagai:

$$k_1 = f_\theta(h_k, x_{t_i}) \quad (3.17)$$

$$k_2 = f_\theta\left(h_k + \frac{1}{2}\delta t \cdot k_1, x_{t_i}\right) \quad (3.18)$$

$$k_3 = f_\theta\left(h_k + \frac{1}{2}\delta t \cdot k_2, x_{t_i}\right) \quad (3.19)$$

$$k_4 = f_\theta(h_k + \delta t \cdot k_3, x_{t_i}) \quad (3.20)$$

$$h_{k+1} = h_k + \frac{\delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (3.21)$$

RK4 memiliki *local truncation error* sebesar $\mathcal{O}(\delta t^5)$ dan *global truncation error* sebesar $\mathcal{O}(\delta t^4)$, sehingga jauh lebih akurat dibanding metode Euler dengan ukuran langkah yang sama, namun memerlukan empat kali lebih banyak evaluasi f_θ per substep.

Dengan asumsi yang sama seperti sebelumnya, perhitungan dua langkah metode RK4 dilakukan sebagai berikut. Langkah pertama dimulai dari $h_0 = 0$ dan $x = 0,5$. Keempat evaluasi fungsi dihitung secara berurutan:

$$k_1 = f_\theta(0; 0,5) = 0,1072$$

$$k_2 = f_\theta(0 + 0,25 \cdot 0,1072; 0,5) = f_\theta(0,0268; 0,5) = 0,0979$$

$$k_3 = f_\theta(0 + 0,25 \cdot 0,0979; 0,5) = f_\theta(0,0245; 0,5) = 0,0987$$

$$k_4 = f_\theta(0 + 0,5 \cdot 0,0987; 0,5) = f_\theta(0,0493; 0,5) = 0,0901$$

Substitusi ke persamaan (3.21):

$$h_1 = 0 + \frac{0,5}{6}(0,1072 + 2(0,0979) + 2(0,0987) + 0,0901) = 0,0492$$

Langkah kedua dimulai dari $h_1 = 0,0492$. Keempat evaluasi turunan dihitung kembali:

$$k_1 = f_\theta(0,0492; 0,5) = 0,0902$$

$$k_2 = f_\theta(0,0717; 0,5) = 0,0825$$

$$k_3 = f_\theta(0,0698; 0,5) = 0,0832$$

$$k_4 = f_\theta(0,0908; 0,5) = 0,0761$$

$$h_2 = 0,0492 + \frac{0,5}{6} (0,0902 + 2(0,0825) + 2(0,0832) + 0,0761) = 0,0907$$

Hasil akhir setelah dua langkah RK4 adalah $h_2^{\text{RK4}} = 0,0907$.

3.5 Ilustrasi Perhitungan GRU-ODE

Setelah melakukan *preprocessing* data, dilakukan proses perhitungan pada satu sel GRU-ODE. Pada ilustrasi ini, digunakan fitur input berupa harga *closing price*, jumlah substep $N = 4$, dimensi *hidden state* $d_h = 2$, dan ODE *solver* berupa metode Euler. Dengan konfigurasi tersebut, matriks bobot W_r, W_z, W_g masing-masing berukuran $(\text{dimensi input} + d_h) \times d_h = 3 \times 2$, dan vektor bias b_r, b_z, b_g berdimensi 2×1 . Bias b_r diinisialisasi dengan vektor $[1, 1]$ (De Brouwer et al., 2019), sedangkan b_z dan b_g diinisialisasi dengan vektor nol. Matriks bobot diinisialisasi secara acak menggunakan *Glorot uniform* (Glorot and Bengio, 2010) pada interval tersebut. Parameter inisialisasi yang digunakan pada ilustrasi ini ditetapkan sebagai berikut:

$$W_r = \begin{bmatrix} 0,5 & -0,3 \\ 0,2 & 0,4 \\ -0,1 & 0,6 \end{bmatrix}, \quad W_z = \begin{bmatrix} 0,1 & 0,7 \\ -0,5 & 0,2 \\ 0,3 & -0,4 \end{bmatrix}, \quad W_g = \begin{bmatrix} 0,4 & -0,2 \\ 0,6 & 0,1 \\ -0,3 & 0,5 \end{bmatrix}$$

$$b_r = \begin{bmatrix} 1,0 \\ 1,0 \end{bmatrix}, \quad b_z = b_g = \begin{bmatrix} 0,0 \\ 0,0 \end{bmatrix}$$

Vektor input gabungan awal yang digunakan adalah $[x, h] = [0,5, 0, 0]$ yang merepresentasikan nilai harga *closing price* ternormalisasi dan *hidden state* awal nol. Ukuran langkah integrasi yang digunakan adalah $\delta t = 0,25$.

3.5.1 Ilustrasi Perhitungan pada *Reset Gate*

Pertama akan melewati *reset gate* dengan fungsi aktivasi sigmoid:

$$[x, h] \cdot W_r = \begin{bmatrix} 0,5 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0,5 & -0,3 \\ 0,2 & 0,4 \\ -0,1 & 0,6 \end{bmatrix} = \begin{bmatrix} 0,25 & -0,15 \end{bmatrix}$$

Selanjutnya, hasil tersebut ditambahkan dengan vektor bias b_r dan diterapkan fungsi aktivasi sigmoid (σ):

$$\begin{aligned} [x, h] \cdot W_r + b_r &= \begin{bmatrix} 0,25 & -0,15 \end{bmatrix} + \begin{bmatrix} 1,0 & 1,0 \end{bmatrix} = \begin{bmatrix} 1,25 & 0,85 \end{bmatrix} \\ r &= \sigma \begin{bmatrix} 1,25 & 0,85 \end{bmatrix} \approx \begin{bmatrix} 0,7773 & 0,7006 \end{bmatrix} \end{aligned}$$

3.5.2 Ilustrasi Perhitungan pada *Update Gate*

Perhitungan *update gate* mengikuti langkah yang identik dengan menggunakan matriks bobot W_z dan bias b_z :

$$[x, h] \cdot W_z = \begin{bmatrix} 0,5 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0,1 & 0,7 \\ -0,5 & 0,2 \\ 0,3 & -0,4 \end{bmatrix} = \begin{bmatrix} 0,05 & 0,35 \end{bmatrix}$$

Setelah penambahan vektor bias bernilai nol, fungsi aktivasi *sigmoid* diterapkan:

$$\begin{aligned} [x, h] \cdot W_z + b_z &= \begin{bmatrix} 0,05 & 0,35 \end{bmatrix} + \begin{bmatrix} 0,0 & 0,0 \end{bmatrix} = \begin{bmatrix} 0,05 & 0,35 \end{bmatrix} \\ z &= \sigma \begin{bmatrix} 0,05 & 0,35 \end{bmatrix} \approx \begin{bmatrix} 0,5125 & 0,5866 \end{bmatrix} \end{aligned}$$

3.5.3 Ilustrasi Perhitungan *Candidate Hidden State* (g)

Pertama, dihitung perkalian elemen (*Hadamard product*) antara hasil dari *reset gate* (r) dengan *hidden state* saat ini (h):

$$r \odot h = \begin{bmatrix} 0,7773 & 0,7006 \end{bmatrix} \odot \begin{bmatrix} 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 \end{bmatrix}$$

Hasil tersebut digabungkan kembali dengan input x menjadi sebuah vektor baru yang

kemudian dikalikan dengan bobot W_g :

$$[x, r \odot h] = \begin{bmatrix} 0,5 & 0 & 0 \end{bmatrix}$$

$$[x, r \odot h] \cdot W_g + b_g = \begin{bmatrix} 0,5 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0,4 & -0,2 \\ 0,6 & 0,1 \\ -0,3 & 0,5 \end{bmatrix} + \begin{bmatrix} 0,0 & 0,0 \end{bmatrix} = \begin{bmatrix} 0,20 & -0,10 \end{bmatrix}$$

Terakhir, fungsi aktivasi *hyperbolic tangent* (tanh) diterapkan untuk mendapatkan nilai *candidate hidden state*:

$$g = \tanh \begin{bmatrix} 0,20 & -0,10 \end{bmatrix} \approx \begin{bmatrix} 0,1974 & -0,0997 \end{bmatrix}$$

3.5.4 Ilustrasi Perhitungan Laju Perubahan *Hidden State*

Setelah memperoleh nilai *reset gate*, *update gate* dan kandidat *hidden state*, akan dihitung laju perubahan *hidden state* melalui persamaan diferensial GRU-ODE. Persamaan diferensial dari GRU-ODE diformulasikan sebagai

$$\frac{dh}{dt} = (1 - z) \odot (g - h)$$

$$1 - z = \begin{bmatrix} 1 & 1 \end{bmatrix} - \begin{bmatrix} 0,5125 & 0,5866 \end{bmatrix} = \begin{bmatrix} 0,4875 & 0,4134 \end{bmatrix}$$

$$g - h = \begin{bmatrix} 0,1974 & -0,0997 \end{bmatrix} - \begin{bmatrix} 0 & 0 \end{bmatrix} = \begin{bmatrix} 0,1974 & -0,0997 \end{bmatrix}$$

Kalikan kedua hasil tersebut secara elemen (*element-wise*):

$$\frac{dh}{dt} = \begin{bmatrix} 0,4875 & 0,4134 \end{bmatrix} \odot \begin{bmatrix} 0,1974 & -0,0997 \end{bmatrix} \approx \begin{bmatrix} 0,0962 & -0,0412 \end{bmatrix}$$

3.5.5 Ilustrasi Perhitungan Penentuan *Hidden State* Baru

Terakhir, akan dihitung *hidden state* baru dengan mengalikan laju perubahan terhadap langkah waktu ($dt = 0,25$):

$$h_{\text{new}} = h + dt \cdot \frac{dh}{dt} = \begin{bmatrix} 0 & 0 \end{bmatrix} + 0,25 \cdot \begin{bmatrix} 0,0962 & -0,0412 \end{bmatrix} \approx \begin{bmatrix} 0,0241 & -0,0103 \end{bmatrix}$$

3.6 Optimasi Parameter (Backpropagation Through the Solver)

Pelatihan GRU-ODE dilakukan dengan meminimalkan fungsi *loss* $\mathcal{L}$ (pada penelitian ini *Mean Squared Error*) melalui *gradient descent*. Untuk itu, diperlukan perhitungan turunan $\partial\mathcal{L}/\partial\theta$ terhadap seluruh parameter $\theta = \{W_r, W_z, W_g, b_r, b_z, b_g\}$ dari GRU-ODE *cell*. Tantangan utamanya adalah evolusi *hidden state* $h(t)$ tidak dihitung dalam satu langkah, melainkan melalui iterasi operasi *ODE solver* (Euler/RK4) sebanyak $N = 4$ sub-step per interval observasi. Oleh sebab itu, perhitungan *gradient* harus dilakukan melalui seluruh substep tersebut.

Pada penelitian ini digunakan pendekatan *Discretized-then-Optimize (backpropagation through the solver)*, dengan pertimbangan: (a) jumlah substep per interval relatif kecil ($N = 4$), sehingga kebutuhan komputasi masih dapat dipenuhi; (b) akurasi *gradient* lebih terjamin dibanding *adjoint method* karena tidak terdapat akumulasi error pada integrasi mundur; dan (c) implementasi lebih sederhana karena memanfaatkan langsung *automatic differentiation* dari TensorFlow tanpa memerlukan implementasi *custom adjoint solver*.

Untuk memperjelas mekanisme *backpropagation*, derivasi dilakukan dengan asumsi *solver* Euler pada satu substep. Diberikan *hidden state* h_k pada substep ke- k dan input x_{t_i} , satu langkah Euler menghasilkan:

$$r_k = \sigma(W_r[x_{t_i}, h_k] + b_r) \quad (3.22)$$

$$z_k = \sigma(W_z[x_{t_i}, h_k] + b_z) \quad (3.23)$$

$$g_k = \tanh(W_g[x_{t_i}, r_k \odot h_k] + b_g) \quad (3.24)$$

$$h_{k+1} = h_k + \delta t \cdot (1 - z_k) \odot (g_k - h_k) \quad (3.25)$$

Untuk meringkas notasi pada derivasi berikut, didefinisikan pra-aktivasi (*pre-activation*) tiap *gate*, yaitu argumen linier sebelum fungsi aktivasi diterapkan:

$$a_{r,k} = W_r[x_{t_i}, h_k] + b_r \quad (3.26)$$

$$a_{z,k} = W_z[x_{t_i}, h_k] + b_z \quad (3.27)$$

$$a_{g,k} = W_g[x_{t_i}, r_k \odot h_k] + b_g \quad (3.28)$$

sehingga $r_k = \sigma(a_{r,k})$, $z_k = \sigma(a_{z,k})$, dan $g_k = \tanh(a_{g,k})$. Dengan notasi ini, turunan tiap *gate* terhadap parameternya dapat dituliskan secara ringkas pada subbab-subbab berikut.

Persamaan (3.22)–(3.25) merupakan kumpulan operasi diferensiabel, sehingga turunan parsial $\partial h_{k+1}/\partial\theta$ dan $\partial h_{k+1}/\partial h_k$ dapat dihitung melalui aturan rantai.

3.6.1 Turunan *Loss Function* terhadap *Hidden State*

Misalkan $\hat{y}_i = W_o h_N + b_o$ adalah prediksi pada akhir interval ke- i sesuai *output layer* pada Persamaan (3.13), dengan $h_N \equiv h(t_i)$ menyatakan *hidden state* setelah N substep diintegrasikan. *Loss* MSE pada satu sampel adalah $\mathcal{L} = \frac{1}{2}(\hat{y}_i - y_i)^2$.

Secara konseptual, ketergantungan $\mathcal{L}$ terhadap *hidden state* awal h_0 merupakan komposisi mundur sepanjang seluruh substep:

$$\frac{\partial \mathcal{L}}{\partial h_0} = \left(\prod_{k=0}^{N-1} \frac{\partial h_{k+1}}{\partial h_k} \right)^\top \frac{\partial \hat{y}_i}{\partial h_N} \cdot \frac{\partial \mathcal{L}}{\partial \hat{y}_i} \quad (3.29)$$

dengan komponen-komponen rantai:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = \hat{y}_i - y_i, \quad \frac{\partial \hat{y}_i}{\partial h_N} = W_o \quad (3.30)$$

Faktor $\partial h_{k+1}/\partial h_k$ pada (3.29) merupakan inti *backpropagation* antar substep. Faktor ini diperoleh dari (3.25) menggunakan aturan rantai melalui ketiga *gate*:

$$\frac{\partial h_{k+1}}{\partial h_k} = \underbrace{I}_{\text{jalur langsung}} + \underbrace{\delta t \cdot (1 - z_k) \odot \frac{\partial g_k}{\partial h_k}}_{\text{melalui } g_k} - \underbrace{\delta t \cdot (g_k - h_k) \odot \frac{\partial z_k}{\partial h_k}}_{\text{melalui } z_k} - \underbrace{\delta t \cdot (1 - z_k)}_{\text{komponen } -h_k} \quad (3.31)$$

Pada Persamaan (3.31), faktor $\partial g_k/\partial h_k$ dan $\partial z_k/\partial h_k$ merupakan matriks Jacobian, sedangkan vektor di depannya (mis. $\delta t(1 - z_k)$) bekerja sebagai matriks diagonal $\text{diag}(\cdot)$; notasi $\odot$ digunakan sebagai penyederhanaan penulisan. Persamaan tersebut memperlihatkan empat jalur sekaligus: kontribusi identitas (dari h_k sendiri), kontribusi melalui *candidate* g_k , kontribusi melalui *update gate* z_k , dan kontribusi $-h_k$ pada formulasi residual GRU-ODE.

Sinyal error pada tiap substep selanjutnya diperoleh melalui rekursi mundur (*adjoint recursion*):

$$\frac{\partial \mathcal{L}}{\partial h_k} = \left(\frac{\partial h_{k+1}}{\partial h_k} \right)^\top \frac{\partial \mathcal{L}}{\partial h_{k+1}}, \quad \text{dengan kondisi awal} \quad \frac{\partial \mathcal{L}}{\partial h_N} = W_o^\top (\hat{y}_i - y_i). \quad (3.32)$$

Rekursi (3.32) menyediakan $\partial \mathcal{L}/\partial h_{k+1}$ untuk setiap substep, yang menjadi *input* pada perhitungan *gradient* parameter berikut.

3.6.2 Turunan *Loss Function* terhadap Parameter *Gate*

Karena parameter $\theta_j \in \{W_r, W_z, W_g, b_r, b_z, b_g\}$ digunakan bersama di seluruh substep, total *gradient* diperoleh dengan menjumlahkan kontribusi tiap substep:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \theta_j} \Big|_k \quad (3.33)$$

dengan kontribusi tiap substep dihitung dari sinyal error $\partial \mathcal{L} / \partial h_{k+1}$ pada (3.32). Sisa derivasi mengarah pada faktor $\partial h_{k+1} / \partial \theta_j$ untuk masing-masing parameter.

3.6.2.1 Turunan terhadap parameter *update gate* W_z dan b_z

Parameter W_z, b_z memengaruhi h_{k+1} melalui rantai $W_z, b_z \rightarrow a_{z,k} \rightarrow z_k \rightarrow h_{k+1}$. Dengan aturan rantai:

$$\frac{\partial h_{k+1}}{\partial W_z} = \frac{\partial h_{k+1}}{\partial z_k} \cdot \frac{\partial z_k}{\partial a_{z,k}} \cdot \frac{\partial a_{z,k}}{\partial W_z} \quad (3.34)$$

Tiap faktor dievaluasi dari persamaan (3.23) dan (3.25) dan ditabulasikan pada Tabel 3.1.

Tabel 3.1: Faktor turunan parameter *update gate*.

Faktor	Nilai	Sumber
$\partial h_{k+1} / \partial z_k$	$-\delta t \cdot (g_k - h_k)$	dari (3.25)
$\partial z_k / \partial a_{z,k}$	$z_k \odot (1 - z_k)$	turunan sigmoid
$\partial a_{z,k} / \partial W_z$	$[x_{t_i}, h_k]^\top$	dari (3.23)
$\partial a_{z,k} / \partial b_z$	1	dari (3.23)

Sehingga untuk substep ke- k :

$$\frac{\partial \mathcal{L}}{\partial W_z} \Big|_k = \left[\frac{\partial \mathcal{L}}{\partial h_{k+1}} \odot (-\delta t (g_k - h_k)) \odot z_k (1 - z_k) \right] \cdot [x_{t_i}, h_k]^\top \quad (3.35)$$

$$\frac{\partial \mathcal{L}}{\partial b_z} \Big|_k = \frac{\partial \mathcal{L}}{\partial h_{k+1}} \odot (-\delta t (g_k - h_k)) \odot z_k (1 - z_k) \quad (3.36)$$

3.6.2.2 Turunan terhadap parameter *candidate gate* W_g dan b_g

Parameter W_g, b_g memengaruhi h_{k+1} melalui rantai $W_g, b_g \rightarrow a_{g,k} \rightarrow g_k \rightarrow h_{k+1}$:

$$\frac{\partial h_{k+1}}{\partial W_g} = \frac{\partial h_{k+1}}{\partial g_k} \cdot \frac{\partial g_k}{\partial a_{g,k}} \cdot \frac{\partial a_{g,k}}{\partial W_g} \quad (3.37)$$

Sehingga untuk substep ke- k :

$$\left. \frac{\partial \mathcal{L}}{\partial W_g} \right|_k = \left[\frac{\partial \mathcal{L}}{\partial h_{k+1}} \odot \delta t(1 - z_k) \odot (1 - g_k^2) \right] \cdot [x_{t_i}, r_k \odot h_k]^\top \quad (3.38)$$

$$\left. \frac{\partial \mathcal{L}}{\partial b_g} \right|_k = \frac{\partial \mathcal{L}}{\partial h_{k+1}} \odot \delta t(1 - z_k) \odot (1 - g_k^2) \quad (3.39)$$

3.6.2.3 Turunan terhadap parameter *reset gate* W_r dan b_r

Parameter W_r, b_r memengaruhi h_{k+1} secara tidak langsung yaitu melalui r_k yang masuk ke input *candidate gate* g_k , melalui rantai: $W_r, b_r \rightarrow a_{r,k} \rightarrow r_k \rightarrow a_{g,k} \rightarrow g_k \rightarrow h_{k+1}$:

$$\frac{\partial h_{k+1}}{\partial W_r} = \frac{\partial h_{k+1}}{\partial g_k} \cdot \frac{\partial g_k}{\partial a_{g,k}} \cdot \frac{\partial a_{g,k}}{\partial r_k} \cdot \frac{\partial r_k}{\partial a_{r,k}} \cdot \frac{\partial a_{r,k}}{\partial W_r} \quad (3.40)$$

Faktor $\partial a_{g,k} / \partial r_k = W_{g,h} \text{diag}(h_k)$ bersifat matriks, sehingga saat dipropagasi mundur ia masuk sebagai perkalian matriks-vektor $W_{g,h}^\top(\cdot)$, bukan operasi Hadamard. Dengan mendefinisikan sinyal error pada pra-aktivasi *reset gate*

$$\bar{a}_{r,k} = r_k(1 - r_k) \odot h_k \odot \left(W_{g,h}^\top \left[\delta t(1 - z_k) \odot (1 - g_k^2) \odot \frac{\partial \mathcal{L}}{\partial h_{k+1}} \right] \right), \quad (3.41)$$

maka *gradient* terhadap parameter *reset gate* pada substep ke- k adalah:

$$\left. \frac{\partial \mathcal{L}}{\partial W_r} \right|_k = \bar{a}_{r,k} \cdot [x_{t_i}, h_k]^\top \quad (3.42)$$

$$\left. \frac{\partial \mathcal{L}}{\partial b_r} \right|_k = \bar{a}_{r,k} \quad (3.43)$$

dengan $W_{g,h}$ adalah sub-blok bobot W_g yang dikenakan pada bagian *hidden state* dari konkatenasi, sebagaimana didefinisikan pada Persamaan (??).

Rantai pada persamaan di atas memuat lima faktor, di antaranya dua turunan aktivasi *saturating* berurutan, yaitu $(1 - g_k^2)$ dari tanh dan $r_k(1 - r_k)$ dari sigmoid. Karena nilai kedua turunan tersebut terbatas pada $(0, 1]$ dan $(0, 0,25]$, perkaliannya bersama skala $\delta t(1 - z_k)$ membuat *gradient* terhadap W_r, b_r cenderung memiliki *magnitude* yang lebih kecil dibanding *update gate* dan *candidate gate* (Pascanu et al., 2013).

3.6.2.4 Akumulasi *Gradient* Lintas Substep

Karena parameter θ digunakan bersama di seluruh substep, gradien final diperoleh dengan menjumlahkan kontribusi setiap substep. Secara eksplisit, untuk setiap parameter

$\theta_j \in \{W_r, W_z, W_g, b_r, b_z, b_g\}$:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \theta_j} \Big|_k \quad (3.44)$$

Pada konteks pelatihan dengan sekuens berisi T interval observasi, *gradient* diakumulasi lebih lanjut melalui seluruh interval, sehingga *automatic differentiation* melakukan sebanyak $T \cdot N$ komputasi mundur antar substep) per sampel sebelum parameter diperbarui satu kali pada akhir *batch*.

3.6.3 Optimasi dengan Adam

Setelah *gradient* $\partial \mathcal{L} / \partial \theta$ diperoleh melalui *backpropagation through the solver*, parameter θ diperbarui menggunakan algoritma *optimizer*. Pada penelitian ini digunakan Adam (*Adaptive Moment Estimation*) yang diperkenalkan oleh Kingma and Ba (2015).

Adam menggabungkan dua ide utama dari *optimizer* sebelumnya: estimasi momen pertama (*momentum*, Qian, 1999) dan estimasi momen kedua (*Root Mean Square Propagation*, Tieleman and Hinton, 2012). Misalkan $\mathbf{g}_t = \nabla_{\theta} \mathcal{L}(\theta_{t-1})$ menyatakan *gradient* pada iterasi ke- t (notasi tebal digunakan untuk membedakannya dari kandidat *hidden state* g_k). Adam memperbarui parameter melalui empat langkah berikut.

3.6.3.1 Estimasi Momen Pertama (Rerata *Gradient*)

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \mathbf{g}_t \quad (3.45)$$

dengan m_t merupakan *exponentially weighted moving average* dari *gradient*, dan $\beta_1 \in [0, 1)$ adalah faktor peluruhan. Persamaan tersebut menambahkan efek *momentum* yang menjaga arah pembaruan parameter tetap stabil meskipun *gradient* pada setiap *mini-batch* mengalami fluktuasi.

3.6.3.2 Estimasi Momen Kedua (Rerata Kuadrat *Gradient*)

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot \mathbf{g}_t^2 \quad (3.46)$$

dengan $\mathbf{g}_t^2$ menyatakan operasi kuadrat elemen-per-elemen pada *gradient*. Persamaan tersebut menangkap variansi *gradient* untuk setiap parameter secara individual, sehingga memungkinkan *learning rate* yang adaptif per parameter.

3.6.3.3 Koreksi Bias

Pada iterasi awal, nilai m_t dan v_t cenderung bias menuju nol karena diinisialisasi sebagai $m_0 = v_0 = 0$. Untuk mengoreksi bias ini, dilakukan normalisasi:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (3.47)$$

Faktor pembagi $\frac{1}{1 - \beta_1^t}$ dan $\frac{1}{1 - \beta_2^t}$ mendekati 1 saat t besar, sehingga koreksi bias hanya berpengaruh signifikan pada iterasi-iterasi awal pelatihan.

3.6.3.4 Pembaruan Parameter

Dengan demikian, bentuk akhir dari estimasi tak bias dan persamaan untuk memperbarui parameter diberikan oleh:

$$\theta_t = \theta_{t-1} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \quad (3.48)$$

dengan η adalah *learning rate* dan ε adalah konstanta kecil untuk menjaga kestabilan numerik.

3.7 Hyperparameter Gated Recurrent Unit - Ordinary Differential Equation

Pelatihan model GRU-ODE dilakukan melalui proses *grid search* untuk menemukan kombinasi *hyperparameter* terbaik. *Hyperparameter* adalah parameter yang nilainya ditetapkan sebelum proses pelatihan dimulai dan tidak diperbarui selama proses pembaruan bobot melalui *backpropagation* (Goodfellow et al., 2016). Pemilihan *hyperparameter* yang tepat sangat berpengaruh terhadap kinerja model, sehingga diperlukan eksplorasi sistematis terhadap berbagai kombinasi nilai. Pada penelitian ini, *hyperparameter* yang dieksplorasi beserta literatur pemilihan rentang nilainya dijelaskan sebagai berikut:

3.7.1 Sequence Length

Sequence length (L) menyatakan jumlah pengamatan historis berurutan yang digunakan model untuk memprediksi nilai berikutnya. Pada penelitian ini ditetapkan $L = 30$, yang berarti setiap prediksi didasarkan pada 30 pengamatan terakhir. Pemilihan $L = 30$ didasarkan pada pertimbangan bahwa jendela tersebut cukup panjang untuk menangkap pola pergerakan harga jangka pendek pada pasar saham, namun tidak terlalu panjang sehingga membebani komputasi pelatihan.

3.7.2 Jumlah Unit *Hidden State* (*Hidden Units*)

Jumlah unit *hidden state* (d_h) menentukan dimensi vektor *hidden state* $h(t)$ pada GRU-ODE. Nilai d_h yang lebih besar memungkinkan model menangkap representasi yang lebih kompleks, namun meningkatkan jumlah parameter yang harus dipelajari dan berisiko menyebabkan *overfitting* (Goodfellow et al., 2016). Pada penelitian ini dieksplorasi dua nilai, yaitu $d_h \in \{16, 32\}$, untuk *time-series forecasting* menggunakan GRU Kurniasari et al. (2023).

3.7.3 *Learning Rate*

Learning rate (η) adalah ukuran langkah pembaruan parameter pada algoritma *optimizer*. Nilai η yang terlalu besar menyebabkan *loss* berosilasi atau divergen, sedangkan nilai yang terlalu kecil menyebabkan konvergensi lambat dan model dapat terjebak pada minimum lokal (Kingma and Ba, 2015). Pada penelitian ini dieksplorasi dua nilai $\eta \in \{0,001, 0,01\}$, mengacu pada penelitian prediksi harga saham menggunakan GRU oleh Tshamaroh et al. (2025).

3.7.4 Jumlah *Epoch*

Epoch menyatakan satu siklus lengkap pelatihan model terhadap seluruh data pelatihan. Pada penelitian ini ditetapkan jumlah *epoch* maksimum sebesar 80. Untuk mencegah *overfitting* akibat pelatihan yang terlalu lama, digunakan mekanisme *early stopping* dengan *patience* sebesar 12 *epoch*, yaitu pelatihan dihentikan apabila *validation loss* tidak membaik selama 12 *epoch* berturut-turut. Pada saat pelatihan dihentikan, bobot model yang dikembalikan adalah bobot pada *epoch* dengan *validation loss* terbaik. Penentuan jumlah *epochs* didasarkan pada penelitian *time-series forecasting* oleh Kurniasari et al. (2023)

3.7.5 *Batch Size*

Batch size (B) menyatakan jumlah sampel yang diproses bersama dalam satu langkah pembaruan parameter. *Batch size* yang lebih kecil cenderung menghasilkan estimasi *gradient* yang lebih bervariasi (*noisy*) tetapi dapat membantu pelarian dari minimum lokal, sedangkan *batch size* yang lebih besar memberikan estimasi *gradient* yang lebih stabil dengan biaya komputasi yang lebih tinggi (Goodfellow et al., 2016). Pada penelitian ini dieksplorasi dua nilai $B \in \{32, 64\}$, mengikuti eksperimen prediksi harga saham menggunakan GRU oleh Tshamaroh et al. (2025) dan eksperimen *time-series forecasting* oleh

Kurniasari et al. (2023).

3.7.6 ODE Solver

ODE solver adalah metode numerik untuk menyelesaikan persamaan diferensial. Pada penelitian ini dieksplorasi dua metode, yaitu Euler dan Runge-Kutta orde 4 (RK4) (De Brouwer et al., 2019). Metode Euler bersifat ringan secara komputasi karena hanya memerlukan satu evaluasi turunan per substep, sedangkan RK4 lebih akurat namun memerlukan empat evaluasi turunan per substep. Eksplorasi kedua metode bertujuan untuk menyelidiki *trade-off* antara akurasi prediksi dan efisiensi komputasi pada konteks data *intraday* LQ45.

3.7.7 Jumlah Substep Integrasi (Number of Unfolds)

Jumlah *substep* integrasi (N) menyatakan banyaknya pembagian interval waktu antar observasi Δt_i menjadi langkah-langkah kecil pada *ODE solver*. Ukuran langkah pada setiap substep adalah $\delta t = \Delta t_i / N$. Nilai N yang lebih besar menghasilkan integrasi yang lebih akurat namun meningkatkan biaya komputasi. Pada penelitian ini ditetapkan $N = 4$, mengikuti konfigurasi yang digunakan oleh De Brouwer et al. (2019).

3.7.8 Optimizer dan Gradient Clipping

Optimizer yang digunakan pada penelitian ini adalah Adam (Kingma and Ba, 2015) dengan parameter *default* yaitu $\beta_1 = 0,9$, $\beta_2 = 0,999$, dan $\varepsilon = 10^{-7}$. Selain itu, diterapkan *gradient clipping* dengan nilai *clipnorm* sebesar 1,0 untuk membatasi *gradient* agar tidak melebihi nilai tersebut. Teknik ini berfungsi untuk mencegah *exploding gradient* yang umum terjadi pada arsitektur *recurrent neural network* dengan sekuens yang panjang (Pascanu et al., 2013).

3.7.9 Data Splitting

Data dibagi secara kronologis menjadi tiga bagian, yaitu data pelatihan (*training*), validasi (*validation*), dan pengujian (*testing*), dengan proporsi 75%:15%:10%. Pembagian secara kronologis diperlukan untuk menjaga sifat temporal data deret waktu dan mencegah *data leakage*, yaitu kondisi di mana informasi dari masa depan secara tidak sengaja digunakan untuk memprediksi masa lalu (Kaufman et al., 2012). Data *training* digunakan untuk pembaruan bobot model, data *validation* digunakan untuk pemilihan *hyperparameter* terbaik dan *early stopping*, sedangkan data *testing* digunakan hanya pada tahap akhir

untuk evaluasi performa model.

3.8 Perhitungan Metrik Evaluasi

Sifat bursa saham yang sangat dinamis berarti *forecasting* indeks harga saham adalah sebuah bidang penelitian yang memerlukan tingkat akurasi model yang tinggi. Oleh karena itu, dibutuhkan metrik evaluasi yang dapat menilai kemampuan model dalam memprediksi indeks harga saham LQ45. Pada penelitian ini, dipilih tiga metrik evaluasi, yaitu *Root Mean Squared Error* (RMSE), *Mean Absolute Error* (MAE), dan *Mean Absolute Percentage Error* (MAPE) (Kim et al., 2024) yang mampu mencerminkan performa model untuk data *time-series*.

3.8.1 *Root Mean Squared Error* (RMSE)

Root Mean Squared Error (RMSE) adalah salah satu metrik evaluasi yang umum digunakan dalam evaluasi model *time-series forecasting*. RMSE bekerja dengan menghitung akar kuadrat dari rata-rata kuadrat selisih nilai prediksi dan aktual. Secara matematis, RMSE didefinisikan sebagai berikut (Hyndman and Athanasopoulos, 2018):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (3.49)$$

Dengan:

N : jumlah data observasi;

y_i : nilai aktual pada waktu ke- i ;

$\hat{y}_i$: nilai prediksi pada waktu ke- i .

RMSE memberikan bobot yang lebih besar terhadap kesalahan yang bernilai besar karena proses pengkuadratan sebelum dirata-rata (Chai and Draxler, 2014). Dalam konteks *forecasting* indeks harga saham, bursa saham yang dinamis berarti sensitivitas terhadap error sangat tinggi, sehingga penggunaan RMSE sangat relevan. Semakin kecil nilai RMSE berarti kinerja model semakin baik.

3.8.2 *Mean Absolute Error* (MAE)

Mean Absolute Error (MAE) mengukur rata-rata *absolute difference* antara nilai prediksi dan aktual. MAE memperlakukan semua error secara linier, sehingga kurang sensitif terhadap *outlier*. Namun, MAE dianggap sebagai metrik evaluasi yang lebih alami

dan tidak bias dalam menghitung error (Willmott and Matsuura, 2005). MAE juga unggul dalam interpretasi karena memiliki satuan unit yang sama seperti data. Secara matematis, MAE dituliskan sebagai:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (3.50)$$

Pada penelitian ini, MAE digunakan bersama RMSE untuk menganalisis variansi kesalahan. Jika RMSE jauh lebih besar dibanding MAE, ada kemungkinan terdapat *outlier* yang ekstrem dalam hasil prediksi.

3.8.3 Mean Absolute Percentage Error (MAPE)

Mean Absolute Percentage Error (MAPE) mengukur rata-rata persentase kesalahan absolut antara nilai prediksi dan nilai aktual. Berbeda dengan RMSE dan MAE yang memiliki satuan unit yang sama dengan data, MAPE menghasilkan nilai dalam bentuk persentase sehingga bersifat *scale-independent* dan memudahkan interpretasi serta perbandingan kinerja model antar dataset dengan skala yang berbeda (Hyndman and Koehler, 2006). Secara matematis, MAPE didefinisikan sebagai berikut:

$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\% \quad (3.51)$$

Dengan:

N : jumlah data observasi;

y_i : nilai aktual pada waktu ke- i ;

$\hat{y}_i$: nilai prediksi pada waktu ke- i .

Dalam konteks *forecasting* pasar finansial, MAPE telah menjadi salah satu metrik standar yang digunakan secara luas pada penelitian *deep learning* untuk prediksi harga saham karena sifat proporsionalnya terhadap besaran nilai aktual (Hadi et al., 2021; Sethia and Raut, 2019).

BAB 4

HASIL PENELITIAN DAN PEMBAHASAN

Bab ini akan menampilkan hasil implementasi model GRU-ODE dalam memprediksi Indeks Harga Saham LQ45, pengolahan data, *splitting* data, pembangunan model, interpretasi hasil serta akurasi model GRU-ODE terhadap prediksi Indeks Harga Saham *Intraday* LQ45. Seluruh penelitian ini menggunakan *Google Colaboratory* dengan bahasa pemrograman Python.

4.1 Spesifikasi Perangkat dan Bahasa Pemrograman

Berikut adalah Tabel 4.1 dengan spesifikasi perangkat yang digunakan dalam menjalankan penelitian ini.

Tabel 4.1: Spesifikasi perangkat dan *library* yang digunakan.

Komponen	Spesifikasi
Prosesor	AMD Ryzen 5 5500U with Radeon Graphics
<i>Installed RAM</i>	16 GB
<i>Graphics Card</i>	AMD Radeon Graphics (496 MB)
<i>Software dan Library</i>	Python 3.12.13, NumPy 2.0.2, Pandas 2.2.2, Matplotlib 3.10.0, Scikit-Learn 1.6.1, Tensor-Flow 2.20.0

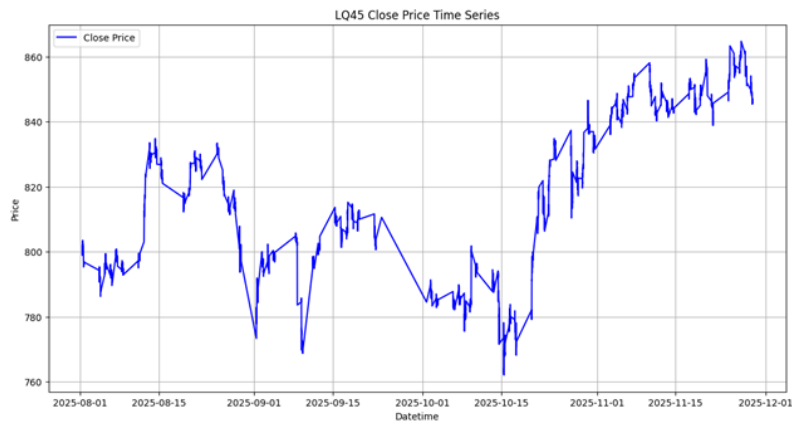
4.2 Data Indeks Harga Saham LQ45

Penelitian ini mencakup prediksi data indeks harga *closing* saham LQ45 menggunakan model GRU-ODE. Data yang digunakan dalam penelitian ini adalah data harga *closing* Indeks Harga Saham LQ45 yang didapat melalui proses *scraping* dengan menggunakan Python melalui *website* Yahoo Finance selama periode waktu 4 bulan, dari tanggal 1 Agustus 2025 sampai dengan 28 November 2025. Data yang ditarik merupakan data *intraday* yang diambil dalam jangka waktu 5 menit secara tidak teratur (*irregular time-series data*). Berdasarkan periode tersebut, terdapat 4.736 sampel data *closing price* dan *timestamp* dengan cuplikan pada Tabel 4.2.

Tabel 4.2: Sampel Data Indeks Harga Saham LQ45.

Timestamp	Close	ΔT
2025-08-01 09:00	799,715027	—
2025-08-01 09:05	800,221985	5
2025-08-01 09:10	799,195984	5
2025-08-01 09:15	798,896973	5
...	...	...
2025-11-28 15:35	846,116028	5
2025-11-28 15:40	845,401001	5
2025-11-28 15:45	847,148987	5
2025-11-28 16:00	845,762024	15

Data *time-series* yang digunakan dalam penelitian ini adalah *irregular time-series data* yang disebabkan oleh karakteristik jam operasional Bursa Efek Indonesia (BEI) yang tidak berjalan 24 jam penuh, serta adanya hari libur. Meskipun proses *scraping* ditargetkan secara *intraday* dengan interval 5 menit, jarak waktu aktual antar observasi sangat bervariasi. Dari total 4.736 sampel data, mayoritas data memiliki interval 5 menit. Namun, analisis terhadap selisih waktu menunjukkan adanya jeda istirahat sesi siang sekitar 90 menit, jeda penutupan pasar malam hari (*overnight close*) sebesar 1.020 menit (17 jam), hingga jeda akhir pekan (*weekend*) sebesar 3.900 menit (65 jam). Lebih lanjut, data ini juga merekam jeda panjang akibat libur atau kendala penarikan, dengan anomali *gap* terbesar mencapai lebih dari seminggu (11.520 menit) antara akhir September hingga awal Oktober 2025.



Gambar 4.1: Grafik Closing Price Indeks Harga Saham Intraday LQ45

4.3 Statistika Deskriptif Variabel *Closing Price* Indeks Harga Saham *Intraday* LQ45

Tabel 4.3 berisi statistika deskriptif dari variabel prediksi yaitu *Closing Price* Indeks Harga Saham LQ45 dari tanggal 1 Agustus 2025 hingga 28 November 2025.

Tabel 4.3: Statistika Deskriptif *Closing Price* Indeks LQ45.

Statistik	Close
<i>count</i>	4736
<i>mean</i>	816,44
<i>std</i>	24,86
<i>min</i>	762,04
25%	796,02
50%	814,40
75%	841,64
<i>max</i>	864,77

4.4 *Pre-processing*

Tahap *pre-processing* adalah tahapan yang dilakukan pada data mentah sebelum di-*feed* ke dalam model. Berikut adalah tahap-tahap yang dilakukan dalam proses *pre-processing* dalam penelitian ini.

4.4.1 *Data Cleaning dan Standarisasi*

Proses ini meliputi beberapa tahap, yaitu:

- *Datetime parsing*: mengubah fitur ‘Datetime’ menjadi dalam bentuk tanggal/waktu dalam Pandas dan memastikan zona waktunya sesuai (UTC).
- Pengurutan kronologis: memastikan data *time-series* diurutkan berdasarkan ‘Datetime’ berurutan dari yang paling lawas hingga terbaru.
- Seleksi fitur: membuang kolom yang tidak diperlukan dan hanya menyimpan fitur ‘Datetime’ dan ‘Close’.

4.4.2 *Feature Engineering*

Proses *feature engineering* pada penelitian ini meliputi:

- Kalkulasi ΔT (selisih waktu): menghitung selisih waktu antara data dibandingkan *timestamp* sebelumnya. Hal ini dilakukan untuk memberikan konteks iregularitas waktu kepada model pada arsitektur GRU-ODE.

- Penanganan *missing value*: melakukan imputasi nilai modus pada *missing value* pada ΔT baris pertama yaitu mengasumsikan jarak sebanyak 5 menit sebanyak nilai ΔT yang paling sering muncul.

4.4.3 Normalisasi

Tahap *pre-processing* data dilanjutkan dengan normalisasi menggunakan *Min-Max Scaler* pada rentang $[0, 1]$. Pemilihan metode ini didasarkan pada signifikansinya dalam memodelkan dinamika data finansial menggunakan arsitektur *deep learning* kontinu. Kumari and Swarnkar (2020) menunjukkan bahwa *Min-Max Scaler* adalah metode *pre-processing* yang paling superior untuk data indeks pasar saham karena mampu menyeragamkan skala komputasi tanpa menghilangkan integritas sinyal volatilitas asli. Hal ini tidak hanya menyelaraskan data dengan domain fungsi aktivasi, tetapi juga menjaga stabilitas komputasi *solver* numerik dengan mencegah nilai *hidden state* meledak (*exploding gradients*) (Pranolo et al., 2023). Parameter normalisasi $x_{\min}$ dan $x_{\max}$ dihitung hanya dari data *training* untuk mencegah *data leakage*, kemudian diterapkan ke data *validation* dan *test* menggunakan parameter yang sama.

Pada data *training* indeks LQ45 yang digunakan, diperoleh nilai minimum dan maksimum sebagai berikut:

$$x_{\min} = 762,0440, \quad x_{\max} = 864,7660$$

Sebagai ilustrasi proses normalisasi, dimisalkan terdapat tiga observasi harga *closing* indeks LQ45 dari data *training*, yaitu $x_1 = 799,7150$, $x_2 = 800,2220$, dan $x_3 = 799,1960$. Hasil normalisasi untuk ketiga observasi tersebut menggunakan persamaan (2.2) adalah:

$$\begin{aligned} x'_1 &= \frac{x_1 - x_{\min}}{x_{\max} - x_{\min}} = \frac{799,7150 - 762,0440}{864,7660 - 762,0440} = \frac{37,6710}{102,7220} \approx 0,3667 \\ x'_2 &= \frac{x_2 - x_{\min}}{x_{\max} - x_{\min}} = \frac{800,2220 - 762,0440}{864,7660 - 762,0440} = \frac{38,1780}{102,7220} \approx 0,3717 \\ x'_3 &= \frac{x_3 - x_{\min}}{x_{\max} - x_{\min}} = \frac{799,1960 - 762,0440}{864,7660 - 762,0440} = \frac{37,1520}{102,7220} \approx 0,3617 \end{aligned}$$

Hasil normalisasi tersebut berada pada rentang $[0, 1]$ sesuai dengan definisi *Min-Max Scaler*. Proses transformasi tersebut diterapkan pada seluruh observasi data *training*, *validation*, dan *test* menggunakan parameter $x_{\min}$ dan $x_{\max}$ yang sama dari data *training*, sebelum data dimasukkan ke dalam model GRU-ODE.

Setelah model menghasilkan prediksi dalam rentang ternormalisasi, hasil prediksi

dikembalikan ke skala asli (nilai indeks LQ45) melalui proses denormalisasi sebagaimana persamaan (2.3). Sebagai ilustrasi, jika model menghasilkan prediksi $\hat{x}' = 0,3667$, maka nilai prediksi pada skala asli adalah:

$$\hat{x} = \hat{x}' \cdot (x_{\max} - x_{\min}) + x_{\min} = 0,3667 \cdot 102,7220 + 762,0440 \approx 799,7150$$

4.4.4 Sequence Generation

Tahapan selanjutnya adalah pembentukan matriks fitur untuk *input* model. Data *time-series* ditransformasikan menjadi sekuens-sekuens yang saling tumpang tindih menggunakan metode *sliding window* dengan panjang jendela $L = 30$. Untuk memprediksi target pada indeks waktu ke- t , model membutuhkan sekuens historis dari harga penutupan (*closing price*) beserta sekuens jeda waktu antar observasi (*elapsed time*).

Secara matematis, input sekuensial dan target untuk satu sampel data diekstraksi dan direpresentasikan melalui persamaan berikut:

$$X_t = (x_{t-L}, x_{t-L+1}, \dots, x_{t-1}) \quad (4.1)$$

$$E_t = (\Delta t_{t-L}, \Delta t_{t-L+1}, \dots, \Delta t_{t-1}) \quad (4.2)$$

$$y_t = x_t \quad (4.3)$$

dengan X_t merepresentasikan sekuens fitur observasi harga saham, E_t merupakan sekuens jeda waktu antar observasi, dan y_t adalah nilai aktual harga saham pada waktu ke- t yang menjadi target prediksi model.

4.4.5 Data Splitting

Time-series forecasting memerlukan data untuk dibagi secara berurutan (*sequential*) untuk mencegah model mencontek informasi dari masa depan (*data leakage*). Maka, dalam penelitian ini, data di-*split* menjadi *train set* (75% pertama), *validation set* (15% berikutnya), dan *test set* (10% terakhir), sebagaimana terinci pada Tabel 4.4.

Tabel 4.4: Pembagian Data dan Periode Waktu Pemodelan.

Split	Proporsi	Jumlah Hari	Periode Waktu
<i>Train Set</i>	75%	67 hari	1 Agustus – 6 Oktober
<i>Validation Set</i>	15%	13 hari	7 Oktober – 19 Oktober
<i>Test Set</i>	10%	9 hari	20 Oktober – 28 Oktober

4.5 Model GRU-ODE

Pembentukan model GRU-ODE diterapkan di program Python menggunakan *framework* TensorFlow dan Keras. Arsitektur model yang digunakan bersumber dari modifikasi kerangka milik De Brouwer et al. (2019) dan Cho et al. (2014). Secara struktural, arsitektur jaringan pada penelitian ini adalah sebagai berikut:

- **Input Layer:** *Input layer* menerima dua buah input, yaitu sekuens harga penutupan (*closing price*) sepanjang 30 langkah waktu (*time steps*) dan sekuens jeda waktu (*elapsed time*). Kedua *input* tersebut kemudian digabungkan (*concatenate*) menjadi satu matriks gabungan berdimensi tiga. Matriks ini berfungsi sebagai kondisi awal sekaligus memberikan informasi langkah waktu integrasi numerik bagi ODE Solver.
- **Hidden Layer:** *Hidden layer* terdiri dari satu arsitektur GRU-ODE. Layer ini secara iteratif mengevaluasi fungsi f_θ untuk memperbarui *hidden state* $h(t)$ secara kontinu. Persamaan diferensial tersebut kemudian diselesaikan secara numerik menggunakan ODE solver. Representasi *hidden state* pada langkah waktu terakhir (h_N) kemudian diambil dan dilewatkan pada lapisan *Dropout* sebagai mekanisme regularisasi untuk mencegah *overfitting* selama pelatihan.
- **Output Layer:** *Output layer* bertugas memetakan representasi *hidden state* terakhir menjadi sebuah nilai prediksi melalui transformasi linier. Transformasi linier ini dilakukan melalui *Dense layer* yang hanya memiliki 1 unit. Hasil dari lapisan ini masih berada pada rentang skala normalisasi, yang selanjutnya akan dinormalisasi untuk menghasilkan nilai prediksi Indeks Harga Saham LQ45 pada waktu observasi berikutnya.

4.6 Optimisasi Hyperparameter GRU-ODE

Optimisasi *hyperparameter* dilakukan demi menemukan model dengan performa terbaik menggunakan metode *GridSearch*. Metode ini bekerja dengan mencari semua kombinasi *hyperparameter*, kemudian memilih kombinasi *hyperparameter* terbaik pada model GRU-ODE pada penelitian prediksi Indeks Harga Saham *Intraday* LQ45. Kandidat *hyperparameter* yang akan dioptimisasi beserta keterangannya ditampilkan pada Tabel 4.5.

Tabel 4.5: Konfigurasi *Hyperparameter* Model GRU-ODE.

Hyperparameter	Nilai	Referensi
Hidden	16 dan 32	(Kurniasari et al., 2023)
<i>Learning Rate</i>	0,0001 dan 0,001	(Tshamaroh et al., 2025)
<i>Epochs</i>	50	(Kurniasari et al., 2023)
<i>Batch size</i>	16 dan 32	(Kurniasari et al., 2023)
<i>ODE Solver</i>	RK4 dan Euler	(De Brouwer et al., 2019)

Pemilihan kombinasi terbaik dilakukan berdasarkan nilai *Mean Absolute Error* (MAE) pada data *validation* yang terendah yang dituangkan pada Tabel 4.6. Kombinasi terbaik tersebut kemudian dievaluasi pada *test set* untuk memperoleh estimasi performa model yang tidak bias.

Tabel 4.6: *Hyperparameter* Terbaik Hasil *Grid Search*.

Hyperparameter	Parameter Terbaik
Hidden	32
<i>Learning Rate</i>	0,001
<i>Epochs</i>	50
<i>Batch size</i>	16
<i>ODE Solver</i>	RK4

4.7 Implementasi Model GRU-ODE dalam Prediksi Indeks Harga Saham *Intraday* LQ45

Tabel 4.7: Metrik Evaluasi Model GRU-ODE pada Setiap Set Data.

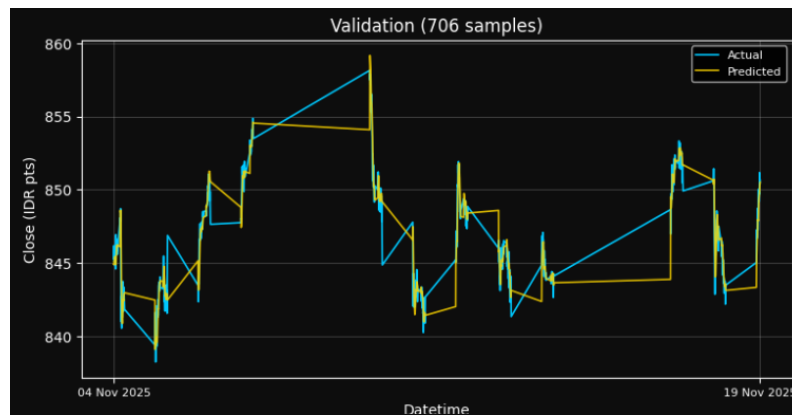
Set Data	MAE	RMSE	MAPE (%)
<i>Train</i>	1,0236	1,7417	0,1276
<i>Validation</i>	0,7736	1,0775	0,0914
<i>Test</i>	0,7831	1,0955	0,0918

Lebih lanjut, dilakukan evaluasi pada seluruh kombinasi *hyperparameter* menggunakan *Grid Search* pada model GRU-ODE. *Optimizer* yang digunakan dalam penelitian ini adalah Adam dan *Mean Squared Error* sebagai fungsi *loss*. Tingkat akurasi model diukur menggunakan *Root Mean Squared Error* (RMSE), *Mean Absolute Error* (MAE) dan *Mean Absolute Percentage Error* (MAPE) sebagai metrik evaluasi. Konfigurasi terbaik dipilih berdasarkan nilai MAE terkecil pada data *validation*. Tabel 4.7 adalah metrik evaluasi yang diperoleh untuk setiap *train*, *validation* dan *test set* pada model terbaik dengan konfigurasi *hyperparameter* pada Tabel 4.6.

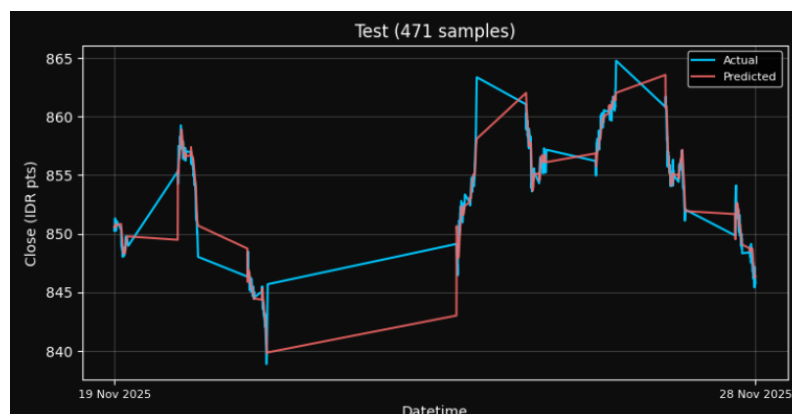
Hasil grafik visualisasi prediksi vs aktual dari masing-masing *train*, *validation*, dan *test set* dapat dilihat pada Gambar 4.2, 4.3, dan 4.4 berikut.



Gambar 4.2: Hasil Prediksi pada Train Set



Gambar 4.3: Hasil Prediksi pada Validation Set



Gambar 4.4: Hasil Prediksi pada Test Set

Berdasarkan pengujian *Grid Search* terhadap 16 kombinasi *hyperparameter*, model GRU-ODE mencapai performa paling optimal pada konfigurasi: 32 *hidden units*, *learning*

rate 0,001, ukuran *batch* 16, dan *solver* RK4. Konfigurasi ini menghasilkan prediksi yang sangat akurat dengan tingkat kesalahan MAE 0,7831, RMSE 1,0955 dan MAPE 0,0918%. Dari penafsiran tersebut, dapat disimpulkan bahwa model GRU-ODE memiliki kemampuan yang cukup baik dalam memprediksi Indeks Harga Saham *Intraday* LQ45. Hal ini berarti model GRU-ODE mampu memprediksi Indeks Harga Saham *Intraday* LQ45 yang berfluktuasi cepat dengan baik.

4.8 Perbandingan Performa Model GRU-ODE dengan Model GRU

Untuk mengevaluasi kontribusi formulasi *Neural Ordinary Differential Equation* terhadap performa prediksi Indeks Harga Saham LQ45, dilakukan studi perbandingan antara model GRU-ODE yang diusulkan dengan model GRU. Model GRU mengacu pada arsitektur GRU standar yang diperkenalkan oleh Cho et al. (2014). Untuk menjamin objektivitas perbandingan, kedua model dilatih menggunakan kondisi eksperimen yang identik, yaitu: pembagian data yang sama (75% *train*, 15% *validation*, dan 10% *test*), panjang *sliding window* $L = 30$, fungsi *loss* *Mean Squared Error*, *optimizer* Adam, serta mekanisme *EarlyStopping* dan *ReduceLROnPlateau* dengan konfigurasi yang sama. *Hyperparameter* masing-masing model dipilih melalui *Grid Search* pada ruang pencarian yang setara. Tabel 4.8 menyajikan ringkasan hasil perbandingan kedua model pada *test set*.

Tabel 4.8: Perbandingan Performa GRU-ODE vs GRU pada *Test Set*.

Model	MAE	RMSE	MAPE (%)
GRU	1,1660	1,4861	0,1368
GRU-ODE (RK4)	0,7831	1,0955	0,0918
Peningkatan (%)	29,42	24,52	29,53

Berdasarkan Tabel 4.8, model GRU-ODE secara konsisten mengungguli model GRU pada seluruh metrik evaluasi. Nilai *Mean Absolute Error* (MAE) mengalami penurunan sebesar 29,42%, yaitu dari 1,1660 pada GRU menjadi 0,7831 pada GRU-ODE. Hal ini menunjukkan bahwa rata-rata kesalahan absolut prediksi GRU-ODE terhadap nilai aktual lebih kecil dibandingkan GRU. Selanjutnya, *Root Mean Squared Error* (RMSE) menurun sebesar 24,52%, dari 1,4861 menjadi 1,0955. Karena RMSE memberikan bobot lebih besar pada kesalahan berukuran besar, penurunan ini mengindikasikan bahwa GRU-ODE lebih jarang menghasilkan prediksi dengan deviasi ekstrem terhadap nilai aktual, sehingga lebih stabil dalam menghadapi fluktuasi harga yang tajam. Adapun *Mean Absolute Percentage Error* (MAPE) turun sebesar 29,53%, dari 0,1368% menjadi 0,0918%, yang me-

nunjukkan bahwa *error* relatif GRU-ODE terhadap besaran nilai aktual juga lebih rendah secara proporsional.

Peningkatan akurasi yang konsisten pada ketiga metrik tersebut membuktikan beberapa keunggulan dari arsitektur GRU-ODE yang tidak dimiliki oleh GRU. *Pertama*, GRU-ODE memanfaatkan informasi *elapsed time* (ΔT) sebagai *input* tambahan, sehingga model mampu membedakan antara observasi yang berjarak 5 menit (interval normal dalam satu sesi perdagangan) dengan observasi yang berjarak lebih panjang akibat jeda istirahat siang (90 menit), penutupan pasar malam atau *overnight close* (1.020 menit), akhir pekan (3.900 menit), bahkan jeda akibat hari libur yang dapat mencapai 11.520 menit. Informasi ini menjadi sangat relevan untuk data Indeks Harga Saham *Intraday* LQ45 yang bersifat *irregular time-series*, namun tidak tersedia pada GRU yang secara implisit mengasumsikan interval seragam antar observasi. *Kedua*, integrasi numerik *hidden state* $h(t)$ melalui *ODE solver* RK4 dengan empat *substep* memungkinkan model untuk memodelkan evolusi *hidden state* secara kontinu di antara dua titik observasi.

Secara keseluruhan, hasil perbandingan ini menunjukkan bahwa model GRU-ODE memberikan kontribusi positif terhadap peningkatan akurasi prediksi pada data finansial yang bersifat *irregular time-series*. Reduksi *error* prediksi yang berkisar antara 24–30% pada ketiga metrik evaluasi memberikan bukti empiris bahwa pendekatan pemodelan terhadap irregularitas waktu dan dinamika kontinu *hidden state* merupakan pendekatan yang lebih sesuai untuk prediksi Indeks Harga Saham *Intraday* LQ45 dibandingkan GRU.

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil penelitian dan pembahasan pada bab sebelumnya, diperoleh beberapa kesimpulan sebagai berikut.

1. Data Indeks Harga Saham LQ45 *intraday* dengan interval 5 menit pada periode 1 Agustus 2025 sampai 28 November 2025 meliputi dinamika harga yang secara natural berlangsung dalam waktu kontinu meskipun diobservasi secara diskrit, serta keberadaan gap struktural dengan 4.736 sampel yang dicirikan oleh adanya jeda istirahat sesi siang (± 90 menit), jeda penutupan pasar malam (± 1.020 menit), serta jeda akhir pekan (± 3.900 menit). Karakteristik irregularitas ini menjadikan pendekatan model waktu kontinu sangat relevan diterapkan.
2. Model GRU-ODE berhasil dibangun dengan menggabungkan *gating mechanism* dari GRU dengan evolusi *hidden state* kontinu melalui *ODE solver*. Konstruksi model dilakukan dengan *input layer* yang menggabungkan *closing price* dan *elapsed time*, *hidden layer* arsitektur GRU-ODE dengan integrasi numerik via Euler atau RK4, serta *output layer* berupa *dense layer* dengan denormalisasi *Min-Max*.
3. Berdasarkan pengujian *Grid Search* terhadap 16 kombinasi *hyperparameter*, model GRU-ODE mencapai performa paling optimal dengan konfigurasi 32 *hidden units*, *learning rate* 0,001, *batch size* 16, dan *solver* RK4. Konfigurasi ini menghasilkan MAE 0,8229, RMSE 1,1217, dan MAPE 0,0964% pada *test set*.

Perbandingan langsung antara GRU-ODE dengan GRU pada *test set* menunjukkan bahwa GRU-ODE secara konsisten mengungguli GRU pada seluruh metrik evaluasi, dengan reduksi MAE sebesar 29,42%, RMSE 24,52%, dan MAPE 29,53%. Hasil ini menunjukkan bahwa pemodelan *hidden state* secara kontinu melalui pendekatan Neural ODE lebih efektif untuk data Indeks Harga Saham LQ45 *intraday* dibanding pendekatan *update* diskrit dari GRU standar.

5.2 Saran

Adapun saran untuk penelitian lanjutan adalah sebagai berikut.

1. Penelitian selanjutnya dapat memperluas rentang data, baik dari sisi periode pengamatan maupun jenis indeks saham yang diteliti, untuk menguji generalisasi model GRU-ODE.

2. Dapat dilakukan perbandingan menyeluruh antara model GRU-ODE dengan varian *Neural ODE* lain (misalnya *Latent ODE*, ODE-RNN, dan GRU-ODE-Bayes) untuk menguji efektivitas relatif tiap arsitektur pada data *intraday* keuangan.
3. Eksplorasi penggunaan ODE *solver* adaptif (seperti DOPRI5 atau Adams-Bashforth) dapat dipertimbangkan untuk meningkatkan akurasi pada interval waktu dengan *gap* besar tanpa membebani komputasi secara signifikan.
4. Penambahan fitur eksogen (misalnya volume perdagangan, indeks global, atau sentimen berita keuangan) dapat dipertimbangkan untuk meningkatkan akurasi prediksi.

DAFTAR PUSTAKA

- Adebiyi, A. A., Adewumi, A. O., and Ayo, C. K. (2014). Comparison of ARIMA and artificial neural networks models for stock price prediction. *Journal of Applied Mathematics*, 2014:1–7.
- Arthur, W. B. (1999). Complexity and the economy. *Science*, 284(5411):107–109.
- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153):1–43.
- Bengio, Y., Simard, P., and Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166.
- Bergmeir, C. and Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213.
- Bursa Efek Indonesia (2022). Indeks saham LQ45. <https://www.idx.co.id/>. Diakses 2025.
- Bursa Efek Indonesia (2024). Statistik pasar modal indonesia 2024. <https://www.idx.co.id/>. Diakses 2025.
- Bursa Efek Indonesia (2025). Data historis indeks LQ45 tahun 2025. <https://www.idx.co.id/>. Diakses 2025.
- Chai, T. and Draxler, R. R. (2014). Root mean square error (RMSE) or mean absolute error (MAE)? – arguments against avoiding RMSE in the literature. *Geoscientific Model Development*, 7(3):1247–1250.
- Chen, R. T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D. K. (2018). Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734.
- Chung, J., Gulcehre, C., Cho, K., and Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. In *NIPS 2014 Workshop on Deep Learning and Representation Learning*. arXiv:1412.3555.

- Cryer, J. D. and Chan, K.-S. (2008). *Time Series Analysis: With Applications in R*. Springer, 2 edition.
- De Brouwer, E., Simm, J., Arany, A., and Moreau, Y. (2019). GRU-ODE-Bayes: Continuous modeling of sporadically-observed time series. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, pages 7379–7390.
- GeeksforGeeks (2026). Derivative of the sigmoid function. Diakses pada 25 Mei 2026.
- Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256. PMLR.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, MA.
- Hadi, F., Pristyanto, Y., and Prastyo, P. H. (2021). Predicting lq45 financial sector indices using rnn-lstm. *Journal of Big Data*, 8(1):104.
- Han, J., Kamber, M., and Pei, J. (2011). *Data Mining: Concepts and Techniques*. Morgan Kaufmann, Waltham, MA, 3rd edition.
- Haykin, S. (2009). *Neural networks and learning machines*. Pearson, Upper Saddle River, NJ, 3 edition.
- Hyndman, R. J. and Athanasopoulos, G. (2018). *Forecasting: Principles and Practice*. OTexts, Melbourne, 2 edition.
- Hyndman, R. J. and Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21.
- Kim, J. et al. (2024). Neural differential equation models for financial time-series forecasting. *Expert Systems with Applications*.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR 2015)*. arXiv:1412.6980.

- Kumari, P. and Swarnkar, P. (2020). Importance of data standardization methods on stock indices prediction accuracy. In *Proceedings of the International Conference on Paradigms of Computing, Communication and Data Sciences*, pages 427–435. Springer.
- Kurniasari, D., Oskavina, S., Wamiliana, W., and Warsono, W. (2023). Forecasting the value of indonesian oil-non-oil and gas imported using the gated recurrent unit (gru). *Indonesian Journal of Artificial Intelligence and Data Mining*, 6:71.
- LeBaron, B. (2000). Agent-based computational finance: Suggested readings and early research. *Journal of Economic Dynamics and Control*, 24(5–7):679–702.
- LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521(7553):436–444.
- LeCun, Y. A., Bottou, L., Orr, G. B., and Müller, K.-R. (2012). Efficient BackProp. In *Neural Networks: Tricks of the Trade*, pages 9–48. Springer, 2 edition.
- Nair, V. and Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814.
- Pascanu, R., Mikolov, T., and Bengio, Y. (2013). On the difficulty of training recurrent neural networks. In Dasgupta, S. and McAllester, D., editors, *Proceedings of the 30th International Conference on Machine Learning (ICML)*, volume 28 of *Proceedings of Machine Learning Research*, pages 1310–1318, Atlanta, Georgia, USA. PMLR.
- Pranolo, A. et al. (2023). Enhanced multivariate time series analysis using lstm: A comparative study of min-max and z-score normalization techniques. *Journal of Big Data*, 10(1):1–25.
- Qian, N. (1999). On the momentum term in gradient descent learning algorithms. *Neural Networks*, 12(1):145–151.
- Rebala, G., Ravi, A., and Churiwala, S. (2019). *An Introduction to Machine Learning*. Springer.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.
- Sethia, A. and Raut, P. (2019). Application of lstm, gru and ica for stock price prediction. *Information and Communication Technology for Intelligent Systems*, pages 479–487.

- Shi, J., Jain, M., and Narasimhan, G. (2022). Time series forecasting (tsf) using various deep learning models.
- Simon, P. and Singh, P. (2015). *Too Big to Ignore: The Business Case for Big Data*. Wiley.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958.
- Susilowati, E. et al. (2012). Analisis saham LQ45 di bursa efek indonesia. *Jurnal Ekonomi dan Bisnis*.
- Tieleman, T. and Hinton, G. (2012). Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning.
- Tshamaroh, M., Nasution, N. S., Nadhirah, N., Alfira, R. A., and Xintong, Z. (2025). Amazon stock price prediction using long short-term memory (lstm) and gated recurrent unit (gru). *Public Research Journal of Engineering, Data Technology and Computer Science*, 3(1):1–9.
- Willmott, C. J. and Matsuura, K. (2005). Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Climatic Research*, 30(1):79–82.
- Zill, D. G. (2013). *A First Course in Differential Equations with Modeling Applications*. Brooks/Cole, 10 edition.

Lampiran 1 Kode Program Implementasi Model GRU-ODE

Lampiran ini memuat kode program lengkap implementasi model GRU-ODE untuk peramalan harga saham intraday indeks LQ45 frekuensi tinggi. Kode ditulis dalam bahasa Python menggunakan pustaka TensorFlow, Keras, NumPy, Pandas, serta scikit-learn dan dijalankan pada Google Colab. Program ini terdiri dari dua bagian utama: *Cell 1* berisi proses *setup*, pra-pemrosesan data, definisi model GRU-ODE, serta pencarian *hyperparameter* dengan *grid search*; sedangkan *Cell 2* berisi proses evaluasi model terbaik dan visualisasi hasil prediksi.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import matplotlib.gridspec as gridspec
5 import matplotlib.dates as mdates
6 from sklearn.preprocessing import MinMaxScaler
7 from sklearn.metrics import mean_absolute_error, mean_squared_error
8 import tensorflow as tf
9 from tensorflow import keras
10 from google.colab import files
11 import itertools, time, os, gc
12 tf.random.set_seed(42)
13 np.random.seed(42)
14 # -- 1. Upload Data -----
15 print("Upload your LQ45_combined_final.csv file:")
16 uploaded = files.upload()
17 filename = list(uploaded.keys())[0]
18 df = pd.read_csv(filename)
19 df["Datetime"] = pd.to_datetime(df["Datetime"], utc=True)
20 df = df.sort_values("Datetime").reset_index(drop=True)
21 df = df[["Datetime", "Close"]].copy()
22 df["elapsed"] = df["Datetime"].diff().dt.total_seconds().fillna(300)
23 / 3600.0
24 print(f"Loaded {len(df)} rows | {df['Datetime'].iloc[0]} -> {df['Datetime'].iloc[-1]}")
25 # -- 2. Scale & Build Sequences -----
26 FEATURES = ["Close"]
27 TARGET_IDX = 0
28 SEQ_LEN = 30
29 TRAIN_FRAC = 0.75
30 n_seq_total = len(df) - SEQ_LEN
31 train_seq_end = int(n_seq_total * TRAIN_FRAC)
```



```

31 raw_train_cutoff = SEQ_LEN + train_seq_end
32 scaler = MinMaxScaler()
33 scaler.fit(df[FEATURES].values[:raw_train_cutoff])
34 scaled = scaler.transform(df[FEATURES].values).astype(np.float32)
35 elapsed_arr = df["elapsed"].values.astype(np.float32)
36 def make_sequences(values, elapsed, seq_len):
37     X, E, y = [], [], []
38     for i in range(seq_len, len(values)):
39         X.append(values[i - seq_len : i])
40         E.append(elapsed[i - seq_len : i])
41         y.append(values[i, TARGET_IDX])
42     return np.array(X, np.float32), np.array(E, np.float32), np.
        array(y, np.float32)
43 X_all, E_all, y_all = make_sequences(scaled, elapsed_arr, SEQ_LEN)
44 N = len(X_all)
45 print(f"Total sequences: {N}")
46 # -- 3. GRU-ODE Cell with RK4 and Euler -----
47 class GRUODECell(tf.keras.layers.Layer):
48     """
49     GRU-ODE with selectable solver: 'euler', 'heun', 'rk4'.
50     Uses num_unfolds sub-steps for integration.
51     """
52     def __init__(self, units, num_unfolds=4, solver="euler", **
        kwargs):
53         super().__init__(**kwargs)
54         self.units = units
55         self.num_unfolds = num_unfolds
56         self.solver = solver.lower()
57     @property
58     def state_size(self): return self.units
59     @property
60     def output_size(self): return self.units
61     def build(self, input_shape):
62         feat_dim = input_shape[-1] - 1
63         combined = feat_dim + self.units
64         init_ones = tf.constant_initializer(1.0)
65         init_zeros = "zeros"
66         init_g = "glorot_uniform"
67         self.W_r = self.add_weight(name="W_r", shape=(combined, self
        .units), initializer=init_g)
68         self.b_r = self.add_weight(name="b_r", shape=(self.units,),
        initializer=init_ones)
69         self.W_z = self.add_weight(name="W_z", shape=(combined, self

```

```

        .units), initializer=init_g)
70         self.b_z = self.add_weight(name="b_z", shape=(self.units,),
            initializer=init_zeros)
71         self.W_g = self.add_weight(name="W_g", shape=(combined, self
            .units), initializer=init_g)
72         self.b_g = self.add_weight(name="b_g", shape=(self.units,),
            initializer=init_zeros)
73         self.built = True
74     def _dh_dt(self, x, h):
75         xh = tf.concat([x, h], axis=-1)
76         r = tf.sigmoid(xh @ self.W_r + self.b_r)
77         z = tf.sigmoid(xh @ self.W_z + self.b_z)
78         g = tf.tanh(tf.concat([x, r * h], axis=-1) @ self.W_g +
            self.b_g)
79         return (1.0 - z) * (g - h)
80     def _step_euler(self, x, h, dt):
81         return h + dt * self._dh_dt(x, h)
82     def _step_heun(self, x, h, dt):
83         k1 = self._dh_dt(x, h)
84         h_pred = h + dt * k1
85         k2 = self._dh_dt(x, h_pred)
86         return h + 0.5 * dt * (k1 + k2)
87     def _step_rk4(self, x, h, dt):
88         k1 = self._dh_dt(x, h)
89         k2 = self._dh_dt(x, h + 0.5 * dt * k1)
90         k3 = self._dh_dt(x, h + 0.5 * dt * k2)
91         k4 = self._dh_dt(x, h + dt * k3)
92         return h + (dt / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4)
93     def call(self, inputs, states):
94         x = inputs[:, :-1]
95         elapsed = tf.reshape(inputs[:, -1], (-1, 1))
96
97         elapsed = tf.clip_by_value(elapsed, 1e-4, 24.0)
98         elapsed = tf.math.log1p(elapsed)
99         dt = elapsed / float(self.num_unfolds)
100        h = states[0]
101        if self.solver == "rk4":
102            step = self._step_rk4
103        elif self.solver == "heun":
104            step = self._step_heun
105        else:
106            step = self._step_euler
107        for _ in range(self.num_unfolds):

```

```

108         h = step(x, h, dt)
109         return h, [h]
110     def get_config(self):
111         cfg = super().get_config()
112         cfg.update({"units": self.units, "num_unfolds": self.
num_unfolds,
113                     "solver": self.solver})
114         return cfg
115     # -- 4. Model Builder -----
116     UNFOLDS = 4
117     def build_model(seq_len, n_features, units, dropout, solver):
118         inp_x = keras.Input(shape=(seq_len, n_features), name="features"
)
119         inp_e = keras.Input(shape=(seq_len,), name="elapsed")
120         elapsed_3d = keras.layers.Reshape((seq_len, 1))(inp_e)
121         combined = keras.layers.Concatenate(axis=-1)([inp_x,
elapsed_3d])
122         cell = GRUODECell(units=units, num_unfolds=UNFOLDS, solver=
solver)
123         rnn_out = keras.layers.RNN(cell, return_sequences=False)(
combined)
124         # Simplified head: Dropout -> Dense(1)
125         x = keras.layers.Dropout(dropout)(rnn_out)
126         out = keras.layers.Dense(1)(x)
127         return keras.Model(inputs=[inp_x, inp_e], outputs=out, name="
GRU_ODE")
128     # -- 5. Helpers -----
129     def inverse_close(scaled_col):
130         return scaler.inverse_transform(scaled_col.reshape(-1, 1)).
flatten()
131     def compute_metrics(y_true, y_pred):
132         mae = mean_absolute_error(y_true, y_pred)
133         rmse = np.sqrt(mean_squared_error(y_true, y_pred))
134         mape = np.mean(np.abs((y_true - y_pred) / (y_true + 1e-8))) *
100
135         r2 = 1 - np.sum((y_true - y_pred)**2) / np.sum((y_true - np.
mean(y_true))**2)
136         return {"MAE": mae, "RMSE": rmse, "MAPE": mape, "R2": r2}
137     def split_data(tr_frac, va_frac):
138         train_end = int(N * tr_frac)
139         val_end = int(N * (tr_frac + va_frac))
140         return (X_all[:train_end], E_all[:train_end],
y_all[:train_end],

```

```

141         X_all[train_end:val_end], E_all[train_end:val_end],
        y_all[train_end:val_end],
142         X_all[val_end:], E_all[val_end:],
        y_all[val_end:],
143         train_end, val_end)
144 # -- 6. Hyperparameter Grid -----
145 SPLITS = [(0.75, 0.15, 0.10)]
146 assert abs(SPLITS[0][0] - TRAIN_FRAC) < 1e-9, "TRAIN_FRAC must match
        SPLITS train fraction (avoids leakage)"
147
148 HIDDEN = [16, 32]
149 LRS = [0.0001, 0.001]
150 EPOCHS = [50]
151 BATCHES = [16, 32]
152 SOLVERS = ["rk4", "euler"]
153 combos = list(itertools.product(SPLITS, HIDDEN, LRS, EPOCHS,
        BATCHES, DROPOUTS, SOLVERS))
154 TOTAL = len(combos)
155 print(f"\nTotal combinations to try: {TOTAL}")
156 print(f"Estimated time: ~{TOTAL * 1.5:.0f}-{TOTAL * 4:.0f} minutes (
        depends on epochs/batch/GPU)\n")
157 CHECKPOINT_CSV = "gridsearch_full_results.csv"
158 # Resume support: if file exists, load completed combos
159 if os.path.exists(CHECKPOINT_CSV):
160     done_df = pd.read_csv(CHECKPOINT_CSV)
161     done_keys = set(zip(done_df["split_label"], done_df["hidden"],
        done_df["lr"],
162                        done_df["epochs"], done_df["batch"],
        done_df["dropout"],
163                        done_df["solver"]))
164     print(f"Resuming -- {len(done_keys)} combos already completed.")
165     grid_results = done_df.to_dict("records")
166 else:
167     done_keys = set()
168     grid_results = []
169 # -- 7. Run Grid Search -----
170 print("="*78)
171 print(f" FULL GRID SEARCH ({TOTAL} combinations)")
172 print("="*78)
173 start_time = time.time()
174 best_val_mae = float("inf")
175 best_model = None
176 best_history = None

```

```

177 best_config = None
178 for idx, ((tr_f, va_f, te_f), hidden, lr, n_epochs, bs, dr, solver)
    in enumerate(combos):
179     split_label = f"{int(tr_f*100)}/{int(va_f*100)}/{int(te_f*100)}"
180     key = (split_label, hidden, lr, n_epochs, bs, dr, solver)
181     if key in done_keys:
182
183         existing = next((r for r in grid_results if (r["split_label"]
184             ], r["hidden"], r["lr"],
185             r["epochs"], r["batch"], r["dropout"], r["
186 solver"]) == key), None)
187         if existing and existing["val_MAE"] < best_val_mae:
188             best_val_mae = existing["val_MAE"]
189             best_config = existing
190             continue
191         elapsed = time.time() - start_time
192         print(f"\n[{idx+1}/{TOTAL}] split={split_label} hid={hidden} lr
193 ={lr} "
194             f"ep={n_epochs} bs={bs} dr={dr} solver={solver} "
195             f"(elapsed {elapsed/60:.1f}m)")
196         X_tr, E_tr, y_tr, X_va, E_va, y_va, X_te, E_te, y_te, _, _ =
197 split_data(tr_f, va_f)
198         tf.random.set_seed(42)
199         np.random.seed(42)
200         keras.backend.clear_session()
201         model = build_model(SEQ_LEN, len(FEATURES), hidden, dr, solver)
202         model.compile(
203             optimizer=keras.optimizers.Adam(learning_rate=lr, clipnorm
204 =1.0),
205             loss="mse",
206             metrics=["mae"]
207         )
208         callbacks = [
209             keras.callbacks.EarlyStopping(monitor="val_loss", patience
210 =12,
211
212                                     restore_best_weights=True,
213 verbose=0),
214             keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor
215 =0.5,
216
217                                     patience=5, min_lr=1e-5,
218 verbose=0),
219         ]
220     try:

```

```

210         hist = model.fit(
211             [X_tr, E_tr], y_tr,
212             validation_data=([X_va, E_va], y_va),
213             epochs=n_epochs,
214             batch_size=bs,
215             callbacks=callbacks,
216             verbose=0,
217         )
218     except Exception as e:
219         print(f" X Failed: {e}")
220         continue
221     p_tr = inverse_close(model.predict([X_tr, E_tr], verbose=0).
222                             flatten())
223     t_tr = inverse_close(y_tr)
224     tm_tr = compute_metrics(t_tr, p_tr)
225     p_va = inverse_close(model.predict([X_va, E_va], verbose=0).
226                             flatten())
227     t_va = inverse_close(y_va)
228     vm = compute_metrics(t_va, p_va)
229     entry = {
230         "split_label": split_label, "train_frac": tr_f, "val_frac":
231         va_f, "test_frac": te_f,
232         "hidden": hidden, "lr": lr, "epochs": n_epochs, "batch": bs,
233         "dropout": dr, "solver": solver,
234         "best_epoch": len(hist.history["loss"]),
235         "best_val_loss": float(min(hist.history["val_loss"])),
236         "train_MAE": tm_tr["MAE"], "train_RMSE": tm_tr["RMSE"], "
237         train_MAPE": tm_tr["MAPE"], "train_R2": tm_tr["R2"],
238         "val_MAE": vm["MAE"], "val_RMSE": vm["RMSE"], "val_MAPE": vm
239         ["MAPE"], "val_R2": vm["R2"],
240     }
241     grid_results.append(entry)
242
243     pd.DataFrame(grid_results).to_csv(CHECKPOINT_CSV, index=False)
244     print(f" Val MAE={vm['MAE']:.4f} RMSE={vm['RMSE']:.4f} "
245           f"MAPE={vm['MAPE']:.4f}% R^2={vm['R2']:.4f}")
246     if vm["MAE"] < best_val_mae:
247         best_val_mae = vm["MAE"]
248         best_model = model
249         best_history = hist
250         best_config = entry
251         print(" * New best!")
252     else:

```

```

248         del model
249         gc.collect()
250     total_time = time.time() - start_time
251     print(f"\n{'='*78}")
252     print(f"    GRID SEARCH COMPLETE    *    {total_time/60:.1f} minutes total
        ")
253     print(f"{'='*78}")
254     # -- 8. Final Evaluation on Test Set -----
255     if best_model is not None:
256         print("\nEvaluating Best Model on Unseen Test Set...")
257
258         _, _, _, _, _, _, X_te, E_te, y_te, _, _ = split_data(
            best_config["train_frac"], best_config["val_frac"])
259         p_te = inverse_close(best_model.predict([X_te, E_te], verbose=0)
            .flatten())
260         t_te = inverse_close(y_te)
261         tm = compute_metrics(t_te, p_te)
262         print(f"    Best Config: hidden={best_config['hidden']}, lr={
            best_config['lr']}, solver={best_config['solver']}, bs={
            best_config['batch']}, dr={best_config['dropout']}")
263         print(f"    FINAL TEST MAE    = {tm['MAE']:.4f}")
264         print(f"    FINAL TEST RMSE    = {tm['RMSE']:.4f}")
265         print(f"    FINAL TEST MAPE    = {tm['MAPE']:.4f}%")
266         print(f"    FINAL TEST R^2    = {tm['R2']:.4f}")
267     else:
268         print("\nNo best model was found. Check for training errors.")
269
270     # -- 8. Data Prep for Plotting -----
271     print("Preparing data for final dashboard...")
272     # Re-fetch the exact splits for the best configuration
273     X_tr, E_tr, y_tr, X_va, E_va, y_va, X_te, E_te, y_te, train_end,
        val_end = split_data(
274         best_config["train_frac"], best_config["val_frac"]
275     )
276     if best_model is None:
277         print("best_model not found in memory (likely resumed from CSV).
            Retraining the best configuration...")
278         tf.random.set_seed(42)
279         np.random.seed(42)
280         keras.backend.clear_session()
281         best_model = build_model(SEQ_LEN, len(FEATURES), best_config["
            hidden"], best_config["dropout"], best_config["solver"])
282         best_model.compile(

```

```

283         optimizer=keras.optimizers.Adam(learning_rate=best_config["
loss="mse",
284         loss="mse",
285         metrics=["mae"]
286     )
287     callbacks = [
288         keras.callbacks.EarlyStopping(monitor="val_loss", patience
=12, restore_best_weights=True, verbose=0),
289         keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor
=0.5, patience=5, min_lr=1e-5, verbose=0),
290     ]
291     best_history = best_model.fit(
292         [X_tr, E_tr], y_tr,
293         validation_data=(X_va, E_va), y_va),
294         epochs=best_config["epochs"],
295         batch_size=best_config["batch"],
296         callbacks=callbacks,
297         verbose=1
298     )
299     print("Retraining complete!")
300     # Generate final predictions with the best model
301     p_tr = inverse_close(best_model.predict([X_tr, E_tr], verbose=0).
flatten())
302     t_tr = inverse_close(y_tr)
303     tr_m = compute_metrics(t_tr, p_tr)
304     p_va = inverse_close(best_model.predict([X_va, E_va], verbose=0).
flatten())
305     t_va = inverse_close(y_va)
306     va_m = compute_metrics(t_va, p_va)
307     p_te = inverse_close(best_model.predict([X_te, E_te], verbose=0).
flatten())
308     t_te = inverse_close(y_te)
309     te_m = compute_metrics(t_te, p_te)
310     # Organize into dictionaries for the plot loops
311     trues = {"Train": t_tr, "Val": t_va, "Test": t_te}
312     preds = {"Train": p_tr, "Val": p_va, "Test": p_te}
313     results = {
314         "Train": [tr_m["MAE"], tr_m["RMSE"], tr_m["MAPE"], tr_m["R2"]],
315         "Val": [va_m["MAE"], va_m["RMSE"], va_m["MAPE"], va_m["R2"]],
316         "Test": [te_m["MAE"], te_m["RMSE"], te_m["MAPE"], te_m["R2"]],
317     }
318     # Print the Evaluation Metrics to the console
319     print("\n" + "="*78)

```



```

320 print(f" FINAL EVALUATION METRICS (Best Config: {best_config['
      split_label']} | Hid:{best_config['hidden']} | LR:{best_config['
      lr']})")
321 print("="*78)
322 print(f" [TRAIN] MAE: {tr_m['MAE']:.4f} | RMSE: {tr_m['RMSE']:.4f}
      | MAPE: {tr_m['MAPE']:.4f}% | R^2: {tr_m['R2']:.4f}")
323 print(f" [VAL] MAE: {va_m['MAE']:.4f} | RMSE: {va_m['RMSE']:.4f}
      | MAPE: {va_m['MAPE']:.4f}% | R^2: {va_m['R2']:.4f}")
324 print(f" [TEST] MAE: {te_m['MAE']:.4f} | RMSE: {te_m['RMSE']:.4f}
      | MAPE: {te_m['MAPE']:.4f}% | R^2: {te_m['R2']:.4f}")
325 print("="*78 + "\n")
326 # Prep datetime arrays based on the indices
327 dt_all = df["Datetime"].values[SEQ_LEN:]
328 dt_train = pd.to_datetime(dt_all[:train_end])
329 dt_val = pd.to_datetime(dt_all[train_end:val_end])
330 dt_test = pd.to_datetime(dt_all[val_end:])
331 # Prep Grid Search Results DataFrame (sorted by Val MAE, taking top
      15 for readability)
332 results_df = pd.DataFrame(grid_results).sort_values("val_MAE").
      reset_index(drop=True)
333 plot_df = results_df.head(15).copy() # Top 15 combos
334 config_labels = [f"H{r['hidden']}|L{r['lr']}|{r['solver']}" for _, r
      in plot_df.iterrows()]
335 # -- 9. Plots -----
336 plt.style.use("dark_background")
337 BG = "#0D0D0D"
338 C1 = "#00C9FF" # actual
339 C2 = "#FF6B6B" # predicted / test
340 C3 = "#FFD700" # val
341 def start_end_ticks(ax, dates):
342     """Show only the start and end date as tick labels."""
343     start_num = mdates.date2num(pd.Timestamp(dates[0]))
344     end_num = mdates.date2num(pd.Timestamp(dates[-1]))
345     ax.set_xticks([start_num, end_num])
346     ax.xaxis.set_major_formatter(mdates.DateFormatter("%d %b %Y"))
347     ax.tick_params(axis="x", rotation=0, labelsz=8)
348 fig = plt.figure(figsize=(20, 22), facecolor=BG)
349 fig.suptitle(f"GRU-ODE * LQ45 5-min Close * Best Split {
      best_config['split_label']}",
350             fontsize=18, fontweight="bold", color="white", y=0.92)
351 gs = gridspec.GridSpec(4, 2, figure=fig, hspace=0.40, wspace=0.25)
352 # (A) Loss curve
353 ax = fig.add_subplot(gs[0, :])

```

```

354 if best_history is not None:
355     ep = range(1, len(best_history.history["loss"]) + 1)
356     ax.plot(ep, best_history.history["loss"], color=C1, lw=2,
357            label="Train Loss")
358     ax.plot(ep, best_history.history["val_loss"], color=C2, lw=2,
359            label="Val Loss", ls="--")
360 ax.set_title("Training History (Best Config)", color="white")
361 ax.set_xlabel("Epoch"); ax.set_ylabel("MSE"); ax.legend(); ax.grid(
362     alpha=0.2)
363 ax.set_facecolor(BG)
364 # (B) Train
365 ax = fig.add_subplot(gs[1, 0])
366 ax.plot(dt_train, trues["Train"], color=C1, lw=1.2, label="Actual")
367 ax.plot(dt_train, preds["Train"], color=C2, lw=1.2, label="Predicted",
368        ", alpha=0.85)
369 ax.set_title(f"Train ({len(X_tr)} samples)", color="white")
370 ax.set_xlabel("Datetime"); ax.set_ylabel("Close (IDR pts)")
371 ax.legend(fontsize=8); ax.grid(alpha=0.2); ax.set_facecolor(BG)
372 start_end_ticks(ax, dt_train)
373 # (C) Validation
374 ax = fig.add_subplot(gs[1, 1])
375 ax.plot(dt_val, trues["Val"], color=C1, lw=1.2, label="Actual")
376 ax.plot(dt_val, preds["Val"], color=C3, lw=1.2, label="Predicted",
377        ", alpha=0.85)
378 ax.set_title(f"Validation ({len(X_va)} samples)", color="white")
379 ax.set_xlabel("Datetime"); ax.set_ylabel("Close (IDR pts)")
380 ax.legend(fontsize=8); ax.grid(alpha=0.2); ax.set_facecolor(BG)
381 start_end_ticks(ax, dt_val)
382 # (D) Test
383 ax = fig.add_subplot(gs[2, 0])
384 ax.plot(dt_test, trues["Test"], color=C1, lw=1.5, label="Actual")
385 ax.plot(dt_test, preds["Test"], color=C2, lw=1.5, label="Predicted",
386        ", alpha=0.85)
387 ax.set_title(f"Test ({len(X_te)} samples)", color="white")
388 ax.set_xlabel("Datetime"); ax.set_ylabel("Close (IDR pts)")
389 ax.legend(fontsize=8); ax.grid(alpha=0.2); ax.set_facecolor(BG)
390 start_end_ticks(ax, dt_test)
391 # (E) Metrics bar chart
392 ax = fig.add_subplot(gs[2, 1])
393 x = np.arange(3)
394 w = 0.25
395 metric_labels = ["MAE", "RMSE", "MAPE (%)"]
396 colors_b = [C1, C3, C2]

```

```

391 for i, split in enumerate(["Train", "Val", "Test"]):
392     vals = [results[split][0], results[split][1], results[split][2]]
393     ax.bar(x + (i - 1) * w, vals, w, label=split, color=colors_b[i],
            alpha=0.85)
394 ax.set_xticks(x); ax.set_xticklabels(metric_labels)
395 ax.set_title("Performance Metrics (Best Config)", color="white")
396 ax.legend(fontsize=8); ax.grid(axis="y", alpha=0.2); ax.
    set_facecolor(BG)
397 # (F) Grid Search Comparison -- Train vs Val MAE (Top 15)
398 ax = fig.add_subplot(gs[3, 0])
399 # Check if "train_MAE" exists in the CSV checkpoint, default to 0 if
    not
400 train_maes = plot_df["train_MAE"].tolist() if "train_MAE" in plot_df
    .columns else [0] * len(plot_df)
401 val_maes = plot_df["val_MAE"].tolist()
402 x_pos = np.arange(len(config_labels))
403 ax.bar(x_pos - 0.15, train_maes, 0.3, color=C1, alpha=0.85, label="
    Train MAE")
404 ax.bar(x_pos + 0.15, val_maes, 0.3, color=C3, alpha=0.85, label="
    Val MAE")
405 # Highlight best (which is index 0 since we sorted by val_MAE)
406 ax.bar(0 - 0.15, train_maes[0], 0.3, color=C1, edgecolor="white",
    linewidth=2)
407 ax.bar(0 + 0.15, val_maes[0], 0.3, color=C3, edgecolor="white",
    linewidth=2)
408 ax.set_xticks(x_pos); ax.set_xticklabels(config_labels, rotation=45,
    ha="right", fontsize=8)
409 ax.set_title("Grid Search: Train vs Val MAE (Top 15)", color="white"
    )
410 ax.set_ylabel("MAE"); ax.legend(fontsize=8); ax.grid(axis="y", alpha
    =0.2); ax.set_facecolor(BG)
411 # (G) Grid Search Comparison -- Train vs Val R^2 (Top 15)
412 ax = fig.add_subplot(gs[3, 1])
413 # Check if "train_R2" exists in the CSV checkpoint, default to 0 if
    not
414 train_r2s = plot_df["train_R2"].tolist() if "train_R2" in plot_df.
    columns else [0] * len(plot_df)
415 val_r2s = plot_df["val_R2"].tolist()
416 ax.bar(x_pos - 0.15, train_r2s, 0.3, color=C1, alpha=0.85, label="
    Train R^2")
417 ax.bar(x_pos + 0.15, val_r2s, 0.3, color=C3, alpha=0.85, label="
    Val R^2")
418 ax.bar(0 - 0.15, train_r2s[0], 0.3, color=C1, edgecolor="white",

```

```

        linewidth=2)
419 ax.bar(0 + 0.15, val_r2s[0],    0.3, color=C3, edgecolor="white",
        linewidth=2)
420 ax.set_xticks(x_pos); ax.set_xticklabels(config_labels, rotation=45,
        ha="right", fontsize=8)
421 ax.set_title("Grid Search: Train vs Val R^2 (Top 15)", color="white"
        )
422 ax.set_ylabel("R^2"); ax.legend(fontsize=8); ax.grid(axis="y", alpha
        =0.2); ax.set_facecolor(BG)
423 fig.text(0.5, 0.08,
424         f"SEQ_LEN={SEQ_LEN} | Units={best_config['hidden']} |
        Unfolds={UNFOLDS} | "
425         f"Solver={best_config['solver']} | LR={best_config['lr']} |
        "
426         f"Train={len(X_tr)} | Val={len(X_va)} | Test={len(X_te)}",
427         ha="center", fontsize=10, color="gray")
428 plt.savefig("gruode_gridsearch_results.png", dpi=150, bbox_inches="
        tight", facecolor=BG)
429 plt.show()
430 # -- 10. Save Grid Search Results to CSV -----
431 results_df.to_csv("gridsearch_split_results_sorted.csv", index=False
        )
432 print("\nTop Grid search results saved and sorted to
        gridsearch_split_results_sorted.csv")

```


Lampiran 2 Kode Program GRU

Lampiran ini memuat kode program lengkap implementasi model GRU sebagai pem-banding model GRU-ODE untuk penelitian Prediksi Indeks Harga Saham Intraday LQ45. Kode ditulis dalam bahasa pemrograman Python menggunakan pustaka TensorFlow, Keras, NumPy, Pandas, serta scikit-learn dan dijalankan pada Google Colab.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import matplotlib.gridspec as gridspec
5 import matplotlib.dates as mdates
6 from sklearn.preprocessing import MinMaxScaler
7 from sklearn.metrics import mean_absolute_error, mean_squared_error
8 import tensorflow as tf
9 from tensorflow import keras
10 from google.colab import files
11 import itertools, time, os, gc
12
13 tf.random.set_seed(42)
14 np.random.seed(42)
15
16 # -- 1. Upload Data -----
17 print("Upload your LQ45_combined_final.csv file:")
18 uploaded = files.upload()
19 filename = list(uploaded.keys())[0]
20
21 df = pd.read_csv(filename)
22 df["Datetime"] = pd.to_datetime(df["Datetime"], utc=True)
23 df = df.sort_values("Datetime").reset_index(drop=True)
24 df = df[["Datetime", "Close"]].copy()
25 df["elapsed"] = df["Datetime"].diff().dt.total_seconds().fillna(300)
26 / 3600.0
27 print(f"Loaded {len(df)} rows | {df['Datetime'].iloc[0]} -> {df['Datetime'].iloc[-1]}")
28
29 # -- 2. Scale & Build Sequences -----
30 FEATURES = ["Close"]
31 TARGET_IDX = 0
32 SEQ_LEN = 30
33 scaler = MinMaxScaler()
```

```

34 scaled = scaler.fit_transform(df[FEATURES].values).astype(np.float32
    )
35 elapsed_arr = df["elapsed"].values.astype(np.float32)
36
37 def make_sequences(values, elapsed, seq_len):
38
39     X, E, y = [], [], []
40     for i in range(seq_len, len(values)):
41         X.append(values[i - seq_len : i])
42         E.append(elapsed[i - seq_len : i])
43         y.append(values[i, TARGET_IDX])
44     return np.array(X, np.float32), np.array(E, np.float32), np.
        array(y, np.float32)
45
46 X_all, E_all, y_all = make_sequences(scaled, elapsed_arr, SEQ_LEN)
47 N = len(X_all)
48 print(f"Total sequences: {N}")
49
50 # -- 3. Model Builder-----
51 def build_model(seq_len, n_features, units, dropout):
52
53     inp_x = keras.Input(shape=(seq_len, n_features), name="features"
        )
54
55     rnn_out = keras.layers.GRU(units=units, return_sequences=False)(
        inp_x)
56
57     x = keras.layers.Dropout(dropout)(rnn_out)
58     out = keras.layers.Dense(1)(x)
59
60     return keras.Model(inputs=inp_x, outputs=out, name="GRU_Standard
        ")
61
62 # -- 4. Helpers -----
63 def inverse_close(scaled_col):
64     return scaler.inverse_transform(scaled_col.reshape(-1, 1)).
        flatten()
65
66 def compute_metrics(y_true, y_pred):
67     mae = mean_absolute_error(y_true, y_pred)
68     rmse = np.sqrt(mean_squared_error(y_true, y_pred))
69     mape = np.mean(np.abs((y_true - y_pred) / (y_true + 1e-8))) *
        100

```

```

70     r2 = 1 - np.sum((y_true - y_pred)**2) / np.sum((y_true - np.mean
71     (y_true))**2)
72     return {"MAE": mae, "RMSE": rmse, "MAPE": mape, "R2": r2}
73
74 def split_data(tr_frac, va_frac):
75     train_end = int(N * tr_frac)
76     val_end = int(N * (tr_frac + va_frac))
77     return (X_all[:train_end], E_all[:train_end], y_all[:
78     train_end],
79             X_all[train_end:val_end], E_all[train_end:val_end],
80             y_all[train_end:val_end],
81             X_all[val_end:], E_all[val_end:], y_all[
82             val_end:],
83             train_end, val_end)
84
85 # -- 5. Hyperparameter Grid -----
86
87 SPLITS = [(0.75, 0.15, 0.10)]
88 HIDDEN = [16, 32]
89 LRS = [0.0001, 0.001]
90 EPOCHS = [50]
91 BATCHES = [16, 32]
92 combos = list(itertools.product(SPLITS, HIDDEN, LRS, EPOCHS,
93     BATCHES, DROPOUTS))
94 TOTAL = len(combos)
95 print(f"\nTotal combinations to try: {TOTAL}")
96 print(f"Estimated time: ~{TOTAL * 1:.0f}-{TOTAL * 3:.0f} minutes (
97     GRU lebih cepat dari GRU-ODE)\n")
98
99 CHECKPOINT_CSV = "gru_baseline_gridsearch_results.csv"
100
101 if os.path.exists(CHECKPOINT_CSV):
102     done_df = pd.read_csv(CHECKPOINT_CSV)
103     done_keys = set(zip(done_df["split_label"], done_df["hidden"],
104     done_df["lr"],
105     done_df["epochs"], done_df["batch"], done_df
106     ["dropout"]))
107     print(f"Resuming -- {len(done_keys)} combos already completed.")
108     grid_results = done_df.to_dict("records")
109 else:
110     done_keys = set()
111     grid_results = []

```



```

105
106 # -- 6. Run Grid Search -----
107 print("="*78)
108 print(f"  GRU STANDARD BASELINE - GRID SEARCH ({TOTAL} combinations)
      ")
109 print("="*78)
110 start_time = time.time()
111 best_val_mae = float("inf")
112 best_model = None
113 best_history = None
114 best_config = None
115
116 for idx, ((tr_f, va_f, te_f), hidden, lr, n_epochs, bs, dr) in
    enumerate(combos):
117     split_label = f"{int(tr_f*100)}/{int(va_f*100)}/{int(te_f*100)}"
118     key = (split_label, hidden, lr, n_epochs, bs, dr)
119
120     if key in done_keys:
121         existing = next((r for r in grid_results if (r["split_label"
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140

```

```

=1.0),
141     loss="mse",
142     metrics=["mae"]
143 )
144
145     callbacks = [
146         keras.callbacks.EarlyStopping(monitor="val_loss", patience
=12,
147                                     restore_best_weights=True,
verbose=0),
148         keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor
=0.5,
149                                           patience=5, min_lr=1e-5,
verbose=0),
150     ]
151
152     try:
153
154         hist = model.fit(
155             X_tr, y_tr,
156             validation_data=(X_va, y_va),
157             epochs=n_epochs,
158             batch_size=bs,
159             callbacks=callbacks,
160             verbose=0,
161         )
162     except Exception as e:
163         print(f" X Failed: {e}")
164         continue
165
166     p_va = inverse_close(model.predict(X_va, verbose=0).flatten())
167     t_va = inverse_close(y_va)
168     vm = compute_metrics(t_va, p_va)
169
170     p_tr = inverse_close(model.predict(X_tr, verbose=0).flatten())
171     t_tr = inverse_close(y_tr)
172     trm = compute_metrics(t_tr, p_tr)
173
174     entry = {
175         "split_label": split_label, "train_frac": tr_f, "val_frac":
va_f, "test_frac": te_f,
176         "hidden": hidden, "lr": lr, "epochs": n_epochs, "batch": bs,
"dropout": dr,

```

```

177         "best_epoch": len(hist.history["loss"]),
178         "best_val_loss": float(min(hist.history["val_loss"])),
179         "train_MAE": trm["MAE"], "train_RMSE": trm["RMSE"], "
train_R2": trm["R2"],
180         "val_MAE": vm["MAE"], "val_RMSE": vm["RMSE"], "val_MAPE": vm
["MAPE"], "val_R2": vm["R2"],
181     }
182     grid_results.append(entry)
183     pd.DataFrame(grid_results).to_csv(CHECKPOINT_CSV, index=False)
184
185     print(f"  Val MAE={vm['MAE']:.4f}  RMSE={vm['RMSE']:.4f}  "
186           f"MAPE={vm['MAPE']:.4f}%  R^2={vm['R2']:.4f}")
187
188     if vm["MAE"] < best_val_mae:
189         best_val_mae = vm["MAE"]
190         best_model = model
191         best_history = hist
192         best_config = entry
193         print("  * New best!")
194     else:
195         del model
196         gc.collect()
197
198 total_time = time.time() - start_time
199 print(f"\n{'='*78}")
200 print(f"  GRID SEARCH COMPLETE - {total_time/60:.1f} minutes total")
201 print(f"{'='*78}")
202
203 # -- 7. Final Evaluation on Test Set -----
204 if best_model is not None:
205     print("\nEvaluating Best GRU Baseline on Unseen Test Set...")
206     _, _, _, _, _, _, X_te, E_te, y_te, _, _ = split_data(
best_config["train_frac"], best_config["val_frac"])
207     p_te = inverse_close(best_model.predict(X_te, verbose=0).flatten
())
208     t_te = inverse_close(y_te)
209     tm = compute_metrics(t_te, p_te)
210
211     print(f"  Best Config: hidden={best_config['hidden']}, lr={
best_config['lr']}, "
212           f"bs={best_config['batch']}, dr={best_config['dropout']}")
213     print(f"  FINAL TEST MAE  = {tm['MAE']:.4f}")
214     print(f"  FINAL TEST RMSE = {tm['RMSE']:.4f}")

```

```

215     print(f"    FINAL TEST MAPE = {tm['MAPE']:.4f}%")
216     print(f"    FINAL TEST R^2  = {tm['R2']:.4f}")
217 else:
218     print("\nNo best model was found. Check for training errors.")
219
220
221
222 # -- 8. Data Prep for Plotting -----
223 print("Preparing data for final dashboard...")
224
225 X_tr, E_tr, y_tr, X_va, E_va, y_va, X_te, E_te, y_te, train_end,
    val_end = split_data(
226     best_config["train_frac"], best_config["val_frac"]
227 )
228
229 if best_model is None:
230     print("best_model not found in memory (likely resumed from CSV).
        Retraining the best configuration...")
231     tf.random.set_seed(42)
232     np.random.seed(42)
233     keras.backend.clear_session()
234     best_model = build_model(SEQ_LEN, len(FEATURES), best_config["
        hidden"], best_config["dropout"])
235     best_model.compile(
236         optimizer=keras.optimizers.Adam(learning_rate=best_config["
        lr"], clipnorm=1.0),
237         loss="mse",
238         metrics=["mae"]
239     )
240     callbacks = [
241         keras.callbacks.EarlyStopping(monitor="val_loss", patience
        =12, restore_best_weights=True, verbose=0),
242         keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor
        =0.5, patience=5, min_lr=1e-5, verbose=0),
243     ]
244     best_history = best_model.fit(
245         X_tr, y_tr,
246         validation_data=(X_va, y_va),
247         epochs=best_config["epochs"],
248         batch_size=best_config["batch"],
249         callbacks=callbacks,
250         verbose=1
251     )

```

```

252     print("Retraining complete!")
253
254     # Generate final predictions
255     p_tr = inverse_close(best_model.predict(X_tr, verbose=0).flatten())
256     t_tr = inverse_close(y_tr)
257     tr_m = compute_metrics(t_tr, p_tr)
258
259     p_va = inverse_close(best_model.predict(X_va, verbose=0).flatten())
260     t_va = inverse_close(y_va)
261     va_m = compute_metrics(t_va, p_va)
262
263     p_te = inverse_close(best_model.predict(X_te, verbose=0).flatten())
264     t_te = inverse_close(y_te)
265     te_m = compute_metrics(t_te, p_te)
266
267     trues = {"Train": t_tr, "Val": t_va, "Test": t_te}
268     preds = {"Train": p_tr, "Val": p_va, "Test": p_te}
269     results = {
270         "Train": [tr_m["MAE"], tr_m["RMSE"], tr_m["MAPE"], tr_m["R2"]],
271         "Val": [va_m["MAE"], va_m["RMSE"], va_m["MAPE"], va_m["R2"]],
272         "Test": [te_m["MAE"], te_m["RMSE"], te_m["MAPE"], te_m["R2"]],
273     }
274
275     # Print Final Metrics
276     print("\n" + "="*78)
277     print(f" GRU BASELINE - FINAL EVALUATION METRICS")
278     print(f" Best Config: {best_config['split_label']} | Hid:{
279         best_config['hidden']} | LR:{best_config['lr']}")
280     print(f" [TRAIN] MAE: {tr_m['MAE']:.4f} | RMSE: {tr_m['RMSE']:.4f}
281         | MAPE: {tr_m['MAPE']:.4f}% | R^2: {tr_m['R2']:.4f}")
282     print(f" [VAL] MAE: {va_m['MAE']:.4f} | RMSE: {va_m['RMSE']:.4f}
283         | MAPE: {va_m['MAPE']:.4f}% | R^2: {va_m['R2']:.4f}")
284     print(f" [TEST] MAE: {te_m['MAE']:.4f} | RMSE: {te_m['RMSE']:.4f}
285         | MAPE: {te_m['MAPE']:.4f}% | R^2: {te_m['R2']:.4f}")
286     print("="*78 + "\n")
287
288     # Prep datetime
289     dt_all = df["Datetime"].values[SEQ_LEN:]
290     dt_train = pd.to_datetime(dt_all[:train_end])
291     dt_val = pd.to_datetime(dt_all[train_end:val_end])
292     dt_test = pd.to_datetime(dt_all[val_end:])

```

```

291 # Prep Grid Search Results
292 results_df = pd.DataFrame(grid_results).sort_values("val_MAE").
    reset_index(drop=True)
293 plot_df = results_df.head(15).copy()
294 config_labels = [f"H{r['hidden']}|L{r['lr']}|D{r['dropout']}" for _,
    r in plot_df.iterrows()]
295
296 # -- 9. Plots -----
297 plt.style.use("dark_background")
298 BG = "#0D0D0D"
299 C1 = "#00C9FF" # actual
300 C2 = "#FF6B6B" # predicted / test
301 C3 = "#FFD700" # val
302
303 def start_end_ticks(ax, dates):
304     start_num = mdates.date2num(pd.Timestamp(dates[0]))
305     end_num = mdates.date2num(pd.Timestamp(dates[-1]))
306     ax.set_xticks([start_num, end_num])
307     ax.xaxis.set_major_formatter(mdates.DateFormatter("%d %b %Y"))
308     ax.tick_params(axis="x", rotation=0, labelsz=8)
309
310 fig = plt.figure(figsize=(20, 22), facecolor=BG)
311 fig.suptitle(f"GRU STANDARD (BASELINE) - LQ45 5-min Close | Best
    Split {best_config['split_label']}",
312             fontsize=18, fontweight="bold", color="white", y=0.92)
313
314 gs = gridspec.GridSpec(4, 2, figure=fig, hspace=0.40, wspace=0.25)
315
316 # (A) Loss curve
317 ax = fig.add_subplot(gs[0, :])
318 if best_history is not None:
319     ep = range(1, len(best_history.history["loss"]) + 1)
320     ax.plot(ep, best_history.history["loss"], color=C1, lw=2, label=
        "Train Loss")
321     ax.plot(ep, best_history.history["val_loss"], color=C2, lw=2,
        label="Val Loss", ls="--")
322 ax.set_title("Training History (Best Config)", color="white")
323 ax.set_xlabel("Epoch")
324 ax.set_ylabel("MSE")
325 ax.legend()
326 ax.grid(alpha=0.2)
327 ax.set_facecolor(BG)
328

```

```

329 # (B) Train
330 ax = fig.add_subplot(gs[1, 0])
331 ax.plot(dt_train, trues["Train"], color=C1, lw=1.2, label="Actual")
332 ax.plot(dt_train, preds["Train"], color=C2, lw=1.2, label="Predicted",
333         ", alpha=0.85)
334 ax.set_title(f"Train ({len(X_tr)} samples)", color="white")
335 ax.set_xlabel("Datetime")
336 ax.set_ylabel("Close (IDR pts)")
337 ax.legend(fontsize=8)
338 ax.grid(alpha=0.2)
339 ax.set_facecolor(BG)
340 start_end_ticks(ax, dt_train)
341
342 # (C) Validation
343 ax = fig.add_subplot(gs[1, 1])
344 ax.plot(dt_val, trues["Val"], color=C1, lw=1.2, label="Actual")
345 ax.plot(dt_val, preds["Val"], color=C3, lw=1.2, label="Predicted",
346         alpha=0.85)
347 ax.set_title(f"Validation ({len(X_va)} samples)", color="white")
348 ax.set_xlabel("Datetime")
349 ax.set_ylabel("Close (IDR pts)")
350 ax.legend(fontsize=8)
351 ax.grid(alpha=0.2)
352 ax.set_facecolor(BG)
353 start_end_ticks(ax, dt_val)
354
355 # (D) Test
356 ax = fig.add_subplot(gs[2, 0])
357 ax.plot(dt_test, trues["Test"], color=C1, lw=1.5, label="Actual")
358 ax.plot(dt_test, preds["Test"], color=C2, lw=1.5, label="Predicted",
359         alpha=0.85)
360 ax.set_title(f"Test ({len(X_te)} samples)", color="white")
361 ax.set_xlabel("Datetime")
362 ax.set_ylabel("Close (IDR pts)")
363 ax.legend(fontsize=8)
364 ax.grid(alpha=0.2)
365 ax.set_facecolor(BG)
366 start_end_ticks(ax, dt_test)
367
368 # (E) Metrics bar chart
369 ax = fig.add_subplot(gs[2, 1])
370 x = np.arange(3)
371 w = 0.25

```

```

369 metric_labels = ["MAE", "RMSE", "MAPE (%)"]
370 colors_b = [C1, C3, C2]
371 for i, split in enumerate(["Train", "Val", "Test"]):
372     vals = [results[split][0], results[split][1], results[split][2]]
373     ax.bar(x + (i - 1) * w, vals, w, label=split, color=colors_b[i],
            alpha=0.85)
374 ax.set_xticks(x)
375 ax.set_xticklabels(metric_labels)
376 ax.set_title("Performance Metrics (Best Config)", color="white")
377 ax.legend(fontsize=8)
378 ax.grid(axis="y", alpha=0.2)
379 ax.set_facecolor(BG)
380
381 # (F) Grid Search - Train vs Val MAE (Top 15)
382 ax = fig.add_subplot(gs[3, 0])
383 train_maes = plot_df["train_MAE"].tolist() if "train_MAE" in plot_df
    .columns else [0] * len(plot_df)
384 val_maes = plot_df["val_MAE"].tolist()
385 x_pos = np.arange(len(config_labels))
386 ax.bar(x_pos - 0.15, train_maes, 0.3, color=C1, alpha=0.85, label="
    Train MAE")
387 ax.bar(x_pos + 0.15, val_maes, 0.3, color=C3, alpha=0.85, label="Val
    MAE")
388 ax.bar(0 - 0.15, train_maes[0], 0.3, color=C1, edgecolor="white",
        linewidth=2)
389 ax.bar(0 + 0.15, val_maes[0], 0.3, color=C3, edgecolor="white",
        linewidth=2)
390 ax.set_xticks(x_pos)
391 ax.set_xticklabels(config_labels, rotation=45, ha="right", fontsize
    =8)
392 ax.set_title("Grid Search: Train vs Val MAE (Top 15)", color="white"
    )
393 ax.set_ylabel("MAE")
394 ax.legend(fontsize=8)
395 ax.grid(axis="y", alpha=0.2)
396 ax.set_facecolor(BG)
397
398 # (G) Grid Search - Train vs Val R^2 (Top 15)
399 ax = fig.add_subplot(gs[3, 1])
400 train_r2s = plot_df["train_R2"].tolist() if "train_R2" in plot_df.
    columns else [0] * len(plot_df)
401 val_r2s = plot_df["val_R2"].tolist()
402 ax.bar(x_pos - 0.15, train_r2s, 0.3, color=C1, alpha=0.85, label="

```



```

    Train R^2")
403 ax.bar(x_pos + 0.15, val_r2s, 0.3, color=C3, alpha=0.85, label="Val
    R^2")
404 ax.bar(0 - 0.15, train_r2s[0], 0.3, color=C1, edgecolor="white",
    linewidth=2)
405 ax.bar(0 + 0.15, val_r2s[0], 0.3, color=C3, edgecolor="white",
    linewidth=2)
406 ax.set_xticks(x_pos)
407 ax.set_xticklabels(config_labels, rotation=45, ha="right", fontsize
    =8)
408 ax.set_title("Grid Search: Train vs Val R^2 (Top 15)", color="white"
    )
409 ax.set_ylabel("R^2")
410 ax.legend(fontsize=8)
411 ax.grid(axis="y", alpha=0.2)
412 ax.set_facecolor(BG)
413
414 fig.text(0.5, 0.08,
415         f"Model: GRU Standard (no ODE) | SEQ_LEN={SEQ_LEN} | Units
    ={best_config['hidden']} | "
416         f"LR={best_config['lr']} | Train={len(X_tr)} | Val={len(
    X_va)} | Test={len(X_te)}",
417         ha="center", fontsize=10, color="gray")
418
419 plt.savefig("gru_baseline_gridsearch_results.png", dpi=150,
    bbox_inches="tight", facecolor=BG)
420 plt.show()
421
422 # -- 10. Save Final Results -----
423 results_df.to_csv("gru_baseline_split_results_sorted.csv", index=
    False)
424 print("\nGRU baseline grid search results saved to
    gru_baseline_split_results_sorted.csv")
425
426
427 # -- 12. Print model summary for parameter comparison -----
428 print("\nGRU BASELINE MODEL SUMMARY:")
429 best_model.summary()
430

```